Namespace MakesCentsBackend.Controllers

Classes

AccountController

Controller for generate Account APIs, including Get All Accounts and Delete Account

BankAccountController

Controller for specified bank account APIs, including Create Bank Account and Update Bank Account

BudgetController

API controller for budgets. Persistent budget routes live directly under /api/budgets; month-specific routes live under /api/budgets/months, since a persistent budget spans many months and needs its own id space separate from any one month's.

BudgetTemplateController

API controller for budget templates - named, reusable presets of envelope funding amounts that can be applied to a budget month to bulk-create its funding queue.

DebtAccountController

EnvelopeCategoryController

API controller for envelope categories

EnvelopeFundingController

API controller for the envelope funding queue - a budget month's plan for how much to put into each envelope and in what priority order.

InvestmentAccountController

PaycheckController

API controller for paychecks. Paychecks are forecast-only - see EnvelopeFundingController for how envelopes actually get funded.

PaymentTransactionController

PlannedExpenseController

API controller for planned expenses

RolloverEnvelopeController

API controller for rollover envelopes. Create/Update/Delete act on the persistent definition; Get is month-scoped, since a rollover envelope's planned/remaining amounts are specific to one budget month.

SinkingFundEnvelopeController

API controller for sinking fund envelopes

TransactionController

API controller for transactions

TransferTransactionController

UserController

API controller for users

Class AccountController

Namespace: MakesCentsBackend.Controllers

Assembly: MakesCentsBackend.dll

Controller for generate Account APIs, including Get All Accounts and Delete Account

```
[Route("api/accounts")]
[ApiController]
public class AccountController : ControllerBase
```

Inheritance

object ← ControllerBase ← AccountController

Inherited Members

ControllerBase.StatusCode(int) , ControllerBase.StatusCode(int, object) ,
ControllerBase.Content(string) , ControllerBase.Content(string, string) ,
ControllerBase.Content(string, string, Encoding) ,
ControllerBase.Content(string, MediaTypeHeaderValue) , ControllerBase.NoContent() ,
ControllerBase.Ok() , ControllerBase.Ok(object) , ControllerBase.Redirect(string) ,
ControllerBase.RedirectPermanent(string) , ControllerBase.RedirectPreserveMethod(string) ,
ControllerBase.RedirectPermanentPreserveMethod(string) , ControllerBase.LocalRedirect(string) ,
ControllerBase.LocalRedirectPermanent(string) , ControllerBase.LocalRedirectPreserveMethod(string) ,
ControllerBase.LocalRedirectPermanentPreserveMethod(string) , ControllerBase.RedirectToAction() ,
ControllerBase.RedirectToAction(string) , ControllerBase.RedirectToAction(string, object) ,
ControllerBase.RedirectToAction(string, string) ,
ControllerBase.RedirectToAction(string, string, object) ,
ControllerBase.RedirectToAction(string, string, string) ,
ControllerBase.RedirectToAction(string, string, object, string) ,
ControllerBase.RedirectToActionPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToActionPermanent(string) ,
ControllerBase.RedirectToActionPermanent(string, object) ,
ControllerBase.RedirectToActionPermanent(string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, object) ,
ControllerBase.RedirectToActionPermanent(string, string, object, string) ,
ControllerBase.RedirectToActionPermanentPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToRoute(string) , ControllerBase.RedirectToRoute(object) ,
ControllerBase.RedirectToRoute(string, object) , ControllerBase.RedirectToRoute(string, string) ,

ControllerBase.RedirectToRoute(string, object, string) ,
ControllerBase.RedirectToRoutePreserveMethod(string, object, string) ,
ControllerBase.RedirectToRoutePermanent(string) ,
ControllerBase.RedirectToRoutePermanent(object) ,
ControllerBase.RedirectToRoutePermanent(string, object) ,
ControllerBase.RedirectToRoutePermanent(string, string) ,
ControllerBase.RedirectToRoutePermanent(string, object, string) ,
ControllerBase.RedirectToRoutePermanentPreserveMethod(string, object, string) ,
ControllerBase.RedirectToPage(string) , ControllerBase.RedirectToPage(string, object) ,
ControllerBase.RedirectToPage(string, string) , ControllerBase.RedirectToPage(string, string, object) ,
ControllerBase.RedirectToPage(string, string, string) ,
ControllerBase.RedirectToPage(string, string, object, string) ,
ControllerBase.RedirectToPagePermanent(string) ,
ControllerBase.RedirectToPagePermanent(string, object) ,
ControllerBase.RedirectToPagePermanent(string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, object, string) ,
ControllerBase.RedirectToPagePreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToPagePermanentPreserveMethod(string, string, object, string) ,
ControllerBase.File(byte[], string) , ControllerBase.File(byte[], string, bool) ,
ControllerBase.File(byte[], string, string) , ControllerBase.File(byte[], string, string, bool) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string) , ControllerBase.File(Stream, string, bool) ,
ControllerBase.File(Stream, string, string) , ControllerBase.File(Stream, string, string, bool) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string) , ControllerBase.File(string, string, bool) ,
ControllerBase.File(string, string, string) , ControllerBase.File(string, string, string, bool) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string) , ControllerBase.PhysicalFile(string, string, bool) ,
ControllerBase.PhysicalFile(string, string, string) ,
ControllerBase.PhysicalFile(string, string, string, bool) ,

ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.Unauthorized() , ControllerBase.Unauthorized(object) , ControllerBase.NotFound() ,
ControllerBase.NotFound(object) , ControllerBase.BadRequest() ,
ControllerBase.BadRequest(object) , ControllerBase.BadRequest(ModelStateDictionary) ,
ControllerBase.UnprocessableEntity() , ControllerBase.UnprocessableEntity(object) ,
ControllerBase.UnprocessableEntity(ModelStateDictionary) , ControllerBase.Conflict() ,
ControllerBase.Conflict(object) , ControllerBase.Conflict(ModelStateDictionary) ,
ControllerBase.Problem(string, string, int?, string, string) ,
ControllerBase.ValidationProblem(ValidationProblemDetails) ,
ControllerBase.ValidationProblem(ModelStateDictionary) , ControllerBase.ValidationProblem() ,
ControllerBase.ValidationProblem(string, string, int?, string, string, ModelStateDictionary) ,
ControllerBase.Created() , ControllerBase.Created(string, object) ,
ControllerBase.Created(Uri, object) , ControllerBase.CreatedAtAction(string, object) ,
ControllerBase.CreatedAtAction(string, object, object) ,
ControllerBase.CreatedAtAction(string, string, object, object) ,
ControllerBase.CreatedAtRoute(string, object) , ControllerBase.CreatedAtRoute(object, object) ,
ControllerBase.CreatedAtRoute(string, object, object) , ControllerBase.Accepted() ,
ControllerBase.Accepted(object) , ControllerBase.Accepted(Uri) , ControllerBase.Accepted(string) ,
ControllerBase.Accepted(string, object) , ControllerBase.Accepted(Uri, object) ,
ControllerBase.AcceptedAtAction(string) , ControllerBase.AcceptedAtAction(string, string) ,
ControllerBase.AcceptedAtAction(string, object) ,
ControllerBase.AcceptedAtAction(string, string, object) ,
ControllerBase.AcceptedAtAction(string, object, object) ,
ControllerBase.AcceptedAtAction(string, string, object, object) ,
ControllerBase.AcceptedAtRoute(object) , ControllerBase.AcceptedAtRoute(string) ,
ControllerBase.AcceptedAtRoute(string, object) , ControllerBase.AcceptedAtRoute(object, object) ,
ControllerBase.AcceptedAtRoute(string, object, object) , ControllerBase.Challenge() ,
ControllerBase.Challenge(params string[]) , ControllerBase.Challenge(AuthenticationProperties) ,
ControllerBase.Challenge(AuthenticationProperties, params string[]) , ControllerBase.Forbid() ,
ControllerBase.Forbid(params string[]) , ControllerBase.Forbid(AuthenticationProperties) ,
ControllerBase.Forbid(AuthenticationProperties, params string[]) ,
ControllerBase.SignIn(ClaimsPrincipal) , ControllerBase.SignIn(ClaimsPrincipal, string) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties, string) , ControllerBase.SignOut() ,
ControllerBase.SignOut(AuthenticationProperties) , ControllerBase.SignOut(params string[]) ,
ControllerBase.SignOut(AuthenticationProperties, params string[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel) ,

ControllerBase.TryUpdateModelAsync<TModel>(TModel, string) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync(object, Type, string) ,
ControllerBase.TryUpdateModelAsync(object, Type, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryValidateModel(object) , ControllerBase.TryValidateModel(object, string) ,
ControllerBase.HttpContext , ControllerBase.Request , ControllerBase.Response ,
ControllerBase.RouteData , ControllerBase.ModelState , ControllerBase.ControllerContext ,
ControllerBase.MetadataProvider , ControllerBase.ModelBinderFactory , ControllerBase.Url ,
ControllerBase.ObjectValidator , ControllerBase.ProblemDetailsFactory , ControllerBase.User ,
ControllerBase.Empty , object.Equals(object) , object.Equals(object, object) ,
object.GetHashCode() , object.GetType() , object.MemberwiseClone() ,
object.ReferenceEquals(object, object) , object.ToString()

Constructors

AccountController(AccountLogic)

Parametrized constructor to bring in Dependency Injected Account Logic

```
public AccountController(AccountLogic accountLogic)
```

Parameters

`accountLogic` AccountLogic

Methods

DeleteAccountAsync(int)

Account API to delete an account based on the given account id

```
[Authorize]
[HttpDelete("{accountId}")]
public Task<ActionResult> DeleteAccountAsync(int accountId)
```

Parameters

`accountId` int

The Id of the account to delete

Returns

Task<ActionResult>

GetAllAccounts(int)

Account API to get a list of summaries for all accounts in the given budget

```
[Authorize]
[HttpGet("budget/{budgetId}")]
public Task<ActionResult> GetAllAccounts(int budgetId)
```

Parameters

`budgetId` int

The id for the budget in question

Returns

Task<ActionResult>

Class BankAccountController

Namespace: MakesCentsBackend.Controllers

Assembly: MakesCentsBackend.dll

Controller for specified bank account APIs, including Create Bank Account and Update Bank Account

```
[Route("api/bank-accounts")]
[ApiController]
public class BankAccountController : ControllerBase
```

Inheritance

object ← ControllerBase ← BankAccountController

Inherited Members

ControllerBase.StatusCode(int) , ControllerBase.StatusCode(int, object) ,
ControllerBase.Content(string) , ControllerBase.Content(string, string) ,
ControllerBase.Content(string, string, Encoding) ,
ControllerBase.Content(string, MediaTypeHeaderValue) , ControllerBase.NoContent() ,
ControllerBase.Ok() , ControllerBase.Ok(object) , ControllerBase.Redirect(string) ,
ControllerBase.RedirectPermanent(string) , ControllerBase.RedirectPreserveMethod(string) ,
ControllerBase.RedirectPermanentPreserveMethod(string) , ControllerBase.LocalRedirect(string) ,
ControllerBase.LocalRedirectPermanent(string) , ControllerBase.LocalRedirectPreserveMethod(string) ,
ControllerBase.LocalRedirectPermanentPreserveMethod(string) , ControllerBase.RedirectToAction() ,
ControllerBase.RedirectToAction(string) , ControllerBase.RedirectToAction(string, object) ,
ControllerBase.RedirectToAction(string, string) ,
ControllerBase.RedirectToAction(string, string, object) ,
ControllerBase.RedirectToAction(string, string, string) ,
ControllerBase.RedirectToAction(string, string, object, string) ,
ControllerBase.RedirectToActionPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToActionPermanent(string) ,
ControllerBase.RedirectToActionPermanent(string, object) ,
ControllerBase.RedirectToActionPermanent(string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, object) ,
ControllerBase.RedirectToActionPermanent(string, string, object, string) ,
ControllerBase.RedirectToActionPermanentPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToRoute(string) , ControllerBase.RedirectToRoute(object) ,
ControllerBase.RedirectToRoute(string, object) , ControllerBase.RedirectToRoute(string, string) ,

ControllerBase.RedirectToRoute(string, object, string) ,
ControllerBase.RedirectToRoutePreserveMethod(string, object, string) ,
ControllerBase.RedirectToRoutePermanent(string) ,
ControllerBase.RedirectToRoutePermanent(object) ,
ControllerBase.RedirectToRoutePermanent(string, object) ,
ControllerBase.RedirectToRoutePermanent(string, string) ,
ControllerBase.RedirectToRoutePermanent(string, object, string) ,
ControllerBase.RedirectToRoutePermanentPreserveMethod(string, object, string) ,
ControllerBase.RedirectToPage(string) , ControllerBase.RedirectToPage(string, object) ,
ControllerBase.RedirectToPage(string, string) , ControllerBase.RedirectToPage(string, string, object) ,
ControllerBase.RedirectToPage(string, string, string) ,
ControllerBase.RedirectToPage(string, string, object, string) ,
ControllerBase.RedirectToPagePermanent(string) ,
ControllerBase.RedirectToPagePermanent(string, object) ,
ControllerBase.RedirectToPagePermanent(string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, object, string) ,
ControllerBase.RedirectToPagePreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToPagePermanentPreserveMethod(string, string, object, string) ,
ControllerBase.File(byte[], string) , ControllerBase.File(byte[], string, bool) ,
ControllerBase.File(byte[], string, string) , ControllerBase.File(byte[], string, string, bool) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string) , ControllerBase.File(Stream, string, bool) ,
ControllerBase.File(Stream, string, string) , ControllerBase.File(Stream, string, string, bool) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string) , ControllerBase.File(string, string, bool) ,
ControllerBase.File(string, string, string) , ControllerBase.File(string, string, string, bool) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string) , ControllerBase.PhysicalFile(string, string, bool) ,
ControllerBase.PhysicalFile(string, string, string) ,
ControllerBase.PhysicalFile(string, string, string, bool) ,

ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.Unauthorized() , ControllerBase.Unauthorized(object) , ControllerBase.NotFound() ,
ControllerBase.NotFound(object) , ControllerBase.BadRequest() ,
ControllerBase.BadRequest(object) , ControllerBase.BadRequest(ModelStateDictionary) ,
ControllerBase.UnprocessableEntity() , ControllerBase.UnprocessableEntity(object) ,
ControllerBase.UnprocessableEntity(ModelStateDictionary) , ControllerBase.Conflict() ,
ControllerBase.Conflict(object) , ControllerBase.Conflict(ModelStateDictionary) ,
ControllerBase.Problem(string, string, int?, string, string) ,
ControllerBase.ValidationProblem(ValidationProblemDetails) ,
ControllerBase.ValidationProblem(ModelStateDictionary) , ControllerBase.ValidationProblem() ,
ControllerBase.ValidationProblem(string, string, int?, string, string, ModelStateDictionary) ,
ControllerBase.Created() , ControllerBase.Created(string, object) ,
ControllerBase.Created(Uri, object) , ControllerBase.CreatedAtAction(string, object) ,
ControllerBase.CreatedAtAction(string, object, object) ,
ControllerBase.CreatedAtAction(string, string, object, object) ,
ControllerBase.CreatedAtRoute(string, object) , ControllerBase.CreatedAtRoute(object, object) ,
ControllerBase.CreatedAtRoute(string, object, object) , ControllerBase.Accepted() ,
ControllerBase.Accepted(object) , ControllerBase.Accepted(Uri) , ControllerBase.Accepted(string) ,
ControllerBase.Accepted(string, object) , ControllerBase.Accepted(Uri, object) ,
ControllerBase.AcceptedAtAction(string) , ControllerBase.AcceptedAtAction(string, string) ,
ControllerBase.AcceptedAtAction(string, object) ,
ControllerBase.AcceptedAtAction(string, string, object) ,
ControllerBase.AcceptedAtAction(string, object, object) ,
ControllerBase.AcceptedAtAction(string, string, object, object) ,
ControllerBase.AcceptedAtRoute(object) , ControllerBase.AcceptedAtRoute(string) ,
ControllerBase.AcceptedAtRoute(string, object) , ControllerBase.AcceptedAtRoute(object, object) ,
ControllerBase.AcceptedAtRoute(string, object, object) , ControllerBase.Challenge() ,
ControllerBase.Challenge(params string[]) , ControllerBase.Challenge(AuthenticationProperties) ,
ControllerBase.Challenge(AuthenticationProperties, params string[]) , ControllerBase.Forbid() ,
ControllerBase.Forbid(params string[]) , ControllerBase.Forbid(AuthenticationProperties) ,
ControllerBase.Forbid(AuthenticationProperties, params string[]) ,
ControllerBase.SignIn(ClaimsPrincipal) , ControllerBase.SignIn(ClaimsPrincipal, string) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties, string) , ControllerBase.SignOut() ,
ControllerBase.SignOut(AuthenticationProperties) , ControllerBase.SignOut(params string[]) ,
ControllerBase.SignOut(AuthenticationProperties, params string[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel) ,

ControllerBase.TryUpdateModelAsync<TModel>(TModel, string) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync(object, Type, string) ,
ControllerBase.TryUpdateModelAsync(object, Type, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryValidateModel(object) , ControllerBase.TryValidateModel(object, string) ,
ControllerBase.HttpContext , ControllerBase.Request , ControllerBase.Response ,
ControllerBase.RouteData , ControllerBase.ModelState , ControllerBase.ControllerContext ,
ControllerBase.MetadataProvider , ControllerBase.ModelBinderFactory , ControllerBase.Url ,
ControllerBase.ObjectValidator , ControllerBase.ProblemDetailsFactory , ControllerBase.User ,
ControllerBase.Empty , object.Equals(object) , object.Equals(object, object) ,
object.GetHashCode() , object.GetType() , object.MemberwiseClone() ,
object.ReferenceEquals(object, object) , object.ToString()

Constructors

BankAccountController(BankAccountLogic)

Parameterized constructor to bring in DI variables

```
public BankAccountController(BankAccountLogic bankAccountLogic)
```

Parameters

`bankAccountLogic` BankAccountLogic

Methods

CreateBankAccountAsync(CreateBankAccountRequest)

POST method to create a new bank account

```
[Authorize]
[HttpPost]
public Task<ActionResult> CreateBankAccountAsync(CreateBankAccountRequest bankAccount)
```

Parameters

`bankAccount` CreateBankAccountRequest

Returns

Task<ActionResult>

GetBankAccountAsync(int)

```
[Authorize]
[HttpGet("{bankAccountId}")]
public Task<ActionResult> GetBankAccountAsync(int bankAccountId)
```

Parameters

`bankAccountId` int

Returns

Task<ActionResult>

UpdateBankAccountAsync(int, UpdateBankAccountRequest)

```
[Authorize]
[HttpPut("{bankAccountId}")]
public Task<ActionResult> UpdateBankAccountAsync(int bankAccountId,
UpdateBankAccountRequest bankAccount)
```

Parameters

`bankAccountId` int

`bankAccount` UpdateBankAccountRequest

Returns

Task<ActionResult>

Class BudgetController

Namespace: MakesCentsBackend.Controllers

Assembly: MakesCentsBackend.dll

API controller for budgets. Persistent budget routes live directly under /api/budgets; month-specific routes live under /api/budgets/months, since a persistent budget spans many months and needs its own id space separate from any one month's.

```
[Route("api/budgets")]
[ApiController]
public class BudgetController : ControllerBase
```

Inheritance

object ← ControllerBase ← BudgetController

Inherited Members

ControllerBase.StatusCode(int) , ControllerBase.StatusCode(int, object) ,
ControllerBase.Content(string) , ControllerBase.Content(string, string) ,
ControllerBase.Content(string, string, Encoding) ,
ControllerBase.Content(string, MediaTypeHeaderValue) , ControllerBase.NoContent() ,
ControllerBase.Ok() , ControllerBase.Ok(object) , ControllerBase.Redirect(string) ,
ControllerBase.RedirectPermanent(string) , ControllerBase.RedirectPreserveMethod(string) ,
ControllerBase.RedirectPermanentPreserveMethod(string) , ControllerBase.LocalRedirect(string) ,
ControllerBase.LocalRedirectPermanent(string) , ControllerBase.LocalRedirectPreserveMethod(string) ,
ControllerBase.LocalRedirectPermanentPreserveMethod(string) , ControllerBase.RedirectToAction() ,
ControllerBase.RedirectToAction(string) , ControllerBase.RedirectToAction(string, object) ,
ControllerBase.RedirectToAction(string, string) ,
ControllerBase.RedirectToAction(string, string, object) ,
ControllerBase.RedirectToAction(string, string, string) ,
ControllerBase.RedirectToAction(string, string, object, string) ,
ControllerBase.RedirectToActionPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToActionPermanent(string) ,
ControllerBase.RedirectToActionPermanent(string, object) ,
ControllerBase.RedirectToActionPermanent(string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, object) ,
ControllerBase.RedirectToActionPermanent(string, string, object, string) ,
ControllerBase.RedirectToActionPermanentPreserveMethod(string, string, object, string) ,

ControllerBase.RedirectToRoute(string) , ControllerBase.RedirectToRoute(object) ,
ControllerBase.RedirectToRoute(string, object) , ControllerBase.RedirectToRoute(string, string) ,
ControllerBase.RedirectToRoute(string, object, string) ,
ControllerBase.RedirectToRoutePreserveMethod(string, object, string) ,
ControllerBase.RedirectToRoutePermanent(string) ,
ControllerBase.RedirectToRoutePermanent(object) ,
ControllerBase.RedirectToRoutePermanent(string, object) ,
ControllerBase.RedirectToRoutePermanent(string, string) ,
ControllerBase.RedirectToRoutePermanent(string, object, string) ,
ControllerBase.RedirectToRoutePermanentPreserveMethod(string, object, string) ,
ControllerBase.RedirectToPage(string) , ControllerBase.RedirectToPage(string, object) ,
ControllerBase.RedirectToPage(string, string) , ControllerBase.RedirectToPage(string, string, object) ,
ControllerBase.RedirectToPage(string, string, string) ,
ControllerBase.RedirectToPage(string, string, object, string) ,
ControllerBase.RedirectToPagePermanent(string) ,
ControllerBase.RedirectToPagePermanent(string, object) ,
ControllerBase.RedirectToPagePermanent(string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, object, string) ,
ControllerBase.RedirectToPagePreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToPagePermanentPreserveMethod(string, string, object, string) ,
ControllerBase.File(byte[], string) , ControllerBase.File(byte[], string, bool) ,
ControllerBase.File(byte[], string, string) , ControllerBase.File(byte[], string, string, bool) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string) , ControllerBase.File(Stream, string, bool) ,
ControllerBase.File(Stream, string, string) , ControllerBase.File(Stream, string, string, bool) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string) , ControllerBase.File(string, string, bool) ,
ControllerBase.File(string, string, string) , ControllerBase.File(string, string, string, bool) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string) , ControllerBase.PhysicalFile(string, string, bool) ,

ControllerBase.PhysicalFile(string, string, string) ,
ControllerBase.PhysicalFile(string, string, string, bool) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.Unauthorized() , ControllerBase.Unauthorized(object) , ControllerBase.NotFound() ,
ControllerBase.NotFound(object) , ControllerBase.BadRequest() ,
ControllerBase.BadRequest(object) , ControllerBase.BadRequest(ModelStateDictionary) ,
ControllerBase.UnprocessableEntity() , ControllerBase.UnprocessableEntity(object) ,
ControllerBase.UnprocessableEntity(ModelStateDictionary) , ControllerBase.Conflict() ,
ControllerBase.Conflict(object) , ControllerBase.Conflict(ModelStateDictionary) ,
ControllerBase.Problem(string, string, int?, string, string) ,
ControllerBase.ValidationProblem(ValidationProblemDetails) ,
ControllerBase.ValidationProblem(ModelStateDictionary) , ControllerBase.ValidationProblem() ,
ControllerBase.ValidationProblem(string, string, int?, string, string, ModelStateDictionary) ,
ControllerBase.Created() , ControllerBase.Created(string, object) ,
ControllerBase.Created(Uri, object) , ControllerBase.CreatedAtAction(string, object) ,
ControllerBase.CreatedAtAction(string, object, object) ,
ControllerBase.CreatedAtAction(string, string, object, object) ,
ControllerBase.CreatedAtRoute(string, object) , ControllerBase.CreatedAtRoute(object, object) ,
ControllerBase.CreatedAtRoute(string, object, object) , ControllerBase.Accepted() ,
ControllerBase.Accepted(object) , ControllerBase.Accepted(Uri) , ControllerBase.Accepted(string) ,
ControllerBase.Accepted(string, object) , ControllerBase.Accepted(Uri, object) ,
ControllerBase.AcceptedAtAction(string) , ControllerBase.AcceptedAtAction(string, string) ,
ControllerBase.AcceptedAtAction(string, object) ,
ControllerBase.AcceptedAtAction(string, string, object) ,
ControllerBase.AcceptedAtAction(string, object, object) ,
ControllerBase.AcceptedAtAction(string, string, object, object) ,
ControllerBase.AcceptedAtRoute(object) , ControllerBase.AcceptedAtRoute(string) ,
ControllerBase.AcceptedAtRoute(string, object) , ControllerBase.AcceptedAtRoute(object, object) ,
ControllerBase.AcceptedAtRoute(string, object, object) , ControllerBase.Challenge() ,
ControllerBase.Challenge(params string[]) , ControllerBase.Challenge(AuthenticationProperties) ,
ControllerBase.Challenge(AuthenticationProperties, params string[]) , ControllerBase.Forbid() ,
ControllerBase.Forbid(params string[]) , ControllerBase.Forbid(AuthenticationProperties) ,
ControllerBase.Forbid(AuthenticationProperties, params string[]) ,
ControllerBase.SignIn(ClaimsPrincipal) , ControllerBase.SignIn(ClaimsPrincipal, string) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties, string) , ControllerBase.SignOut() ,
ControllerBase.SignOut(AuthenticationProperties) , ControllerBase.SignOut(params string[]) ,

ControllerBase.SignOut(AuthenticationProperties, params string[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync(object, Type, string) ,
ControllerBase.TryUpdateModelAsync(object, Type, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryValidateModel(object) , ControllerBase.TryValidateModel(object, string) ,
ControllerBase.HttpContext , ControllerBase.Request , ControllerBase.Response ,
ControllerBase.RouteData , ControllerBase.ModelState , ControllerBase.ControllerContext ,
ControllerBase.MetadataProvider , ControllerBase.ModelBinderFactory , ControllerBase.Url ,
ControllerBase.ObjectValidator , ControllerBase.ProblemDetailsFactory , ControllerBase.User ,
ControllerBase.Empty , object.Equals(object) , object.Equals(object, object) ,
object.GetHashCode() , object.GetType() , object.MemberwiseClone() ,
object.ReferenceEquals(object, object) , object.ToString()

Constructors

BudgetController(BudgetLogic)

Parameterized constructor to bring in DI variables

```
public BudgetController(BudgetLogic budgetLogic)
```

Parameters

`budgetLogic` BudgetLogic

Methods

CreateBudgetAsync(CreateBudgetRequest)

POST method to create a new persistent budget (no month yet)

```
[Authorize]
[HttpPost]
public Task<ActionResult> CreateBudgetAsync(CreateBudgetRequest budget)
```

Parameters

`budget` CreateBudgetRequest

Returns

Task<ActionResult>

CreateBudgetMonthAsync(int, CreateBudgetMonthRequest)

POST method to create a new month under an existing persistent budget

```
[Authorize]
[HttpPost("{budgetId}/months")]
public Task<ActionResult> CreateBudgetMonthAsync(int budgetId, CreateBudgetMonthRequest
budgetMonth)
```

Parameters

`budgetId` int

`budgetMonth` CreateBudgetMonthRequest

Returns

Task<ActionResult>

DeleteBudgetAsync(int)

HTTP DELETE method to delete a persistent budget entirely - cascades every month, account, envelope category, paycheck, and planned expense it has

```
[Authorize]
[HttpDelete("{budgetId}")]
public Task<ActionResult> DeleteBudgetAsync(int budgetId)
```

Parameters

`budgetId` int

Returns

Task<ActionResult>

DeleteBudgetMonthAsync(int)

HTTP DELETE method to delete a single month's budget. Only that month's transactions go with it - accounts, envelope categories, and paychecks are unaffected.

```
[Authorize]
[HttpDelete("months/{budgetMonthId}")]
public Task<ActionResult> DeleteBudgetMonthAsync(int budgetMonthId)
```

Parameters

`budgetMonthId` int

Returns

Task<ActionResult>

GetBudgetMonthAsync(int)

Get a budget month based on its budget_month_id

```
[Authorize]
[HttpGet("months/{budgetMonthId}")]
public Task<ActionResult> GetBudgetMonthAsync(int budgetMonthId)
```

Parameters

`budgetMonthId` int

Returns

Task<ActionResult>

GetBudgetMonthAsync(int, int, Month)

Get a budget month based on a budget id, year, month, and user id

```
[Authorize]
[HttpGet("{budgetId}/months/year/{year}/month/{month}")]
public Task<ActionResult> GetBudgetMonthAsync(int budgetId, int year, Month month)
```

Parameters

`budgetId` int

`year` int

`month` Month

Returns

Task<ActionResult>

GetUserBudgetsAsync()

HTTP GET method to get every budget the logged-in user has access to (id and name only) - meant for use right after login, before any month or budget id is known.

```
[Authorize]
[HttpGet]
public Task<ActionResult> GetUserBudgetsAsync()
```

Returns

Task<ActionResult>

GetYearBudgetsAsync(int, int, Month)

Get up to a year's worth of a budget's past months, anchored at a year/month

```
[Authorize]
[HttpGet("{budgetId}/all/year/{year}/month/{month}")]
public Task<ActionResult> GetYearBudgetsAsync(int budgetId, int year, Month month)
```

Parameters

`budgetId` int

`year` int

`month` Month

Returns

Task<ActionResult>

UpdateBudgetAsync(int, EditBudgetRequest)

HTTP PUT method to update a budget's persistent fields (currently just the name)

```
[Authorize]
[HttpPut("{budgetId}")]
public Task<ActionResult> UpdateBudgetAsync(int budgetId, EditBudgetRequest budget)
```

Parameters

`budgetId` int

`budget` EditBudgetRequest

Returns

Task<ActionResult>

UpdateBudgetMonthAsync(int, EditBudgetMonthRequest)

HTTP PUT method to update a specific budget month's month/year (e.g. fixing a mistake)

```
[Authorize]
[HttpPut("months/{budgetMonthId}")]
public Task<ActionResult> UpdateBudgetMonthAsync(int budgetMonthId,
EditBudgetMonthRequest budgetMonth)
```

Parameters

`budgetMonthId` int

`budgetMonth` EditBudgetMonthRequest

Returns

Task <ActionResult>

Class BudgetTemplateController

Namespace: MakesCentsBackend.Controllers

Assembly: MakesCentsBackend.dll

API controller for budget templates - named, reusable presets of envelope funding amounts that can be applied to a budget month to bulk-create its funding queue.

```
[Route("api/budget-templates")]
[ApiController]
public class BudgetTemplateController : ControllerBase
```

Inheritance

object ← ControllerBase ← BudgetTemplateController

Inherited Members

ControllerBase.StatusCode(int) , ControllerBase.StatusCode(int, object) ,
ControllerBase.Content(string) , ControllerBase.Content(string, string) ,
ControllerBase.Content(string, string, Encoding) ,
ControllerBase.Content(string, MediaTypeHeaderValue) , ControllerBase.NoContent() ,
ControllerBase.Ok() , ControllerBase.Ok(object) , ControllerBase.Redirect(string) ,
ControllerBase.RedirectPermanent(string) , ControllerBase.RedirectPreserveMethod(string) ,
ControllerBase.RedirectPermanentPreserveMethod(string) , ControllerBase.LocalRedirect(string) ,
ControllerBase.LocalRedirectPermanent(string) , ControllerBase.LocalRedirectPreserveMethod(string) ,
ControllerBase.LocalRedirectPermanentPreserveMethod(string) , ControllerBase.RedirectToAction() ,
ControllerBase.RedirectToAction(string) , ControllerBase.RedirectToAction(string, object) ,
ControllerBase.RedirectToAction(string, string) ,
ControllerBase.RedirectToAction(string, string, object) ,
ControllerBase.RedirectToAction(string, string, string) ,
ControllerBase.RedirectToAction(string, string, object, string) ,
ControllerBase.RedirectToActionPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToActionPermanent(string) ,
ControllerBase.RedirectToActionPermanent(string, object) ,
ControllerBase.RedirectToActionPermanent(string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, object) ,
ControllerBase.RedirectToActionPermanent(string, string, object, string) ,
ControllerBase.RedirectToActionPermanentPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToRoute(string) , ControllerBase.RedirectToRoute(object) ,

ControllerBase.RedirectToRoute(string, object) , ControllerBase.RedirectToRoute(string, string) ,
ControllerBase.RedirectToRoute(string, object, string) ,
ControllerBase.RedirectToRoutePreserveMethod(string, object, string) ,
ControllerBase.RedirectToRoutePermanent(string) ,
ControllerBase.RedirectToRoutePermanent(object) ,
ControllerBase.RedirectToRoutePermanent(string, object) ,
ControllerBase.RedirectToRoutePermanent(string, string) ,
ControllerBase.RedirectToRoutePermanent(string, object, string) ,
ControllerBase.RedirectToRoutePermanentPreserveMethod(string, object, string) ,
ControllerBase.RedirectToPage(string) , ControllerBase.RedirectToPage(string, object) ,
ControllerBase.RedirectToPage(string, string) , ControllerBase.RedirectToPage(string, string, object) ,
ControllerBase.RedirectToPage(string, string, string) ,
ControllerBase.RedirectToPage(string, string, object, string) ,
ControllerBase.RedirectToPagePermanent(string) ,
ControllerBase.RedirectToPagePermanent(string, object) ,
ControllerBase.RedirectToPagePermanent(string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, object, string) ,
ControllerBase.RedirectToPagePreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToPagePermanentPreserveMethod(string, string, object, string) ,
ControllerBase.File(byte[], string) , ControllerBase.File(byte[], string, bool) ,
ControllerBase.File(byte[], string, string) , ControllerBase.File(byte[], string, string, bool) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string) , ControllerBase.File(Stream, string, bool) ,
ControllerBase.File(Stream, string, string) , ControllerBase.File(Stream, string, string, bool) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string) , ControllerBase.File(string, string, bool) ,
ControllerBase.File(string, string, string) , ControllerBase.File(string, string, string, bool) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string) , ControllerBase.PhysicalFile(string, string, bool) ,
ControllerBase.PhysicalFile(string, string, string) ,

ControllerBase.PhysicalFile(string, string, string, bool) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.Unauthorized() , ControllerBase.Unauthorized(object) , ControllerBase.NotFound() ,
ControllerBase.NotFound(object) , ControllerBase.BadRequest() ,
ControllerBase.BadRequest(object) , ControllerBase.BadRequest(ModelStateDictionary) ,
ControllerBase.UnprocessableEntity() , ControllerBase.UnprocessableEntity(object) ,
ControllerBase.UnprocessableEntity(ModelStateDictionary) , ControllerBase.Conflict() ,
ControllerBase.Conflict(object) , ControllerBase.Conflict(ModelStateDictionary) ,
ControllerBase.Problem(string, string, int?, string, string) ,
ControllerBase.ValidationProblem(ValidationProblemDetails) ,
ControllerBase.ValidationProblem(ModelStateDictionary) , ControllerBase.ValidationProblem() ,
ControllerBase.ValidationProblem(string, string, int?, string, string, ModelStateDictionary) ,
ControllerBase.Created() , ControllerBase.Created(string, object) ,
ControllerBase.Created(Uri, object) , ControllerBase.CreatedAtAction(string, object) ,
ControllerBase.CreatedAtAction(string, object, object) ,
ControllerBase.CreatedAtAction(string, string, object, object) ,
ControllerBase.CreatedAtRoute(string, object) , ControllerBase.CreatedAtRoute(object, object) ,
ControllerBase.CreatedAtRoute(string, object, object) , ControllerBase.Accepted() ,
ControllerBase.Accepted(object) , ControllerBase.Accepted(Uri) , ControllerBase.Accepted(string) ,
ControllerBase.Accepted(string, object) , ControllerBase.Accepted(Uri, object) ,
ControllerBase.AcceptedAtAction(string) , ControllerBase.AcceptedAtAction(string, string) ,
ControllerBase.AcceptedAtAction(string, object) ,
ControllerBase.AcceptedAtAction(string, string, object) ,
ControllerBase.AcceptedAtAction(string, object, object) ,
ControllerBase.AcceptedAtAction(string, string, object, object) ,
ControllerBase.AcceptedAtRoute(object) , ControllerBase.AcceptedAtRoute(string) ,
ControllerBase.AcceptedAtRoute(string, object) , ControllerBase.AcceptedAtRoute(object, object) ,
ControllerBase.AcceptedAtRoute(string, object, object) , ControllerBase.Challenge() ,
ControllerBase.Challenge(params string[]) , ControllerBase.Challenge(AuthenticationProperties) ,
ControllerBase.Challenge(AuthenticationProperties, params string[]) , ControllerBase.Forbid() ,
ControllerBase.Forbid(params string[]) , ControllerBase.Forbid(AuthenticationProperties) ,
ControllerBase.Forbid(AuthenticationProperties, params string[]) ,
ControllerBase.SignIn(ClaimsPrincipal) , ControllerBase.SignIn(ClaimsPrincipal, string) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties, string) , ControllerBase.SignOut() ,
ControllerBase.SignOut(AuthenticationProperties) , ControllerBase.SignOut(params string[]) ,
ControllerBase.SignOut(AuthenticationProperties, params string[]) ,

ControllerBase.TryUpdateModelAsync<TModel>(TModel) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync(object, Type, string) ,
ControllerBase.TryUpdateModelAsync(object, Type, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryValidateModel(object) , ControllerBase.TryValidateModel(object, string) ,
ControllerBase.HttpContext , ControllerBase.Request , ControllerBase.Response ,
ControllerBase.RouteData , ControllerBase.ModelState , ControllerBase.ControllerContext ,
ControllerBase.MetadataProvider , ControllerBase.ModelBinderFactory , ControllerBase.Url ,
ControllerBase.ObjectValidator , ControllerBase.ProblemDetailsFactory , ControllerBase.User ,
ControllerBase.Empty , object.Equals(object) , object.Equals(object, object) ,
object.GetHashCode() , object.GetType() , object.MemberwiseClone() ,
object.ReferenceEquals(object, object) , object.ToString()

Constructors

BudgetTemplateController(BudgetTemplateLogic)

Parameterized constructor to bring in DI variables

```
public BudgetTemplateController(BudgetTemplateLogic budgetTemplateLogic)
```

Parameters

`budgetTemplateLogic` BudgetTemplateLogic

Methods

ApplyBudgetTemplateAsync(int, int)

HTTP POST method to apply a template to a budget month, fully replacing that month's funding queue with the template's entries

```
[Authorize]
[HttpPost("{budgetTemplateId}/apply/budget-month/{budgetMonthId}")]
public Task<ActionResult> ApplyBudgetTemplateAsync(int budgetTemplateId, int budgetMonthId)
```

Parameters

`budgetTemplateId` int

`budgetMonthId` int

Returns

Task<ActionResult>

CreateBudgetTemplateAsync(CreateBudgetTemplateRequest)

HTTP POST method to create a new budget template

```
[Authorize]
[HttpPost]
public Task<ActionResult> CreateBudgetTemplateAsync(CreateBudgetTemplateRequest template)
```

Parameters

`template` CreateBudgetTemplateRequest

Returns

Task<ActionResult>

DeleteBudgetTemplateAsync(int)

HTTP DELETE method to delete a budget template

```
[Authorize]
[HttpDelete("{budgetTemplateId}")]
```

```
public Task<ActionResult> DeleteBudgetTemplateAsync(int budgetTemplateId)
```

Parameters

`budgetTemplateId` int

Returns

Task<ActionResult>

GetAllBudgetTemplatesAsync(int)

HTTP GET method to get every template for a budget

```
[Authorize]
[HttpGet("budget/{budgetId}")]
public Task<ActionResult> GetAllBudgetTemplatesAsync(int budgetId)
```

Parameters

`budgetId` int

Returns

Task<ActionResult>

GetBudgetTemplateAsync(int)

HTTP GET method to get a single budget template, including its envelope entries

```
[Authorize]
[HttpGet("{budgetTemplateId}")]
public Task<ActionResult> GetBudgetTemplateAsync(int budgetTemplateId)
```

Parameters

`budgetTemplateId` int

Returns

Task<ActionResult>

SetBudgetTemplateEnvelopesAsync(int, SetBudgetTemplateEnvelopesRequest)

HTTP PUT method to set (create/reorder/update/remove all at once) a template's envelope entries

```
[Authorize]
[HttpPut("{budgetTemplateId}/envelopes")]
public Task<ActionResult> SetBudgetTemplateEnvelopesAsync(int budgetTemplateId,
SetBudgetTemplateEnvelopesRequest templateEnvelopes)
```

Parameters

`budgetTemplateId` int

`templateEnvelopes` SetBudgetTemplateEnvelopesRequest

Returns

Task<ActionResult>

UpdateBudgetTemplateAsync(int, EditBudgetTemplateRequest)

HTTP PUT method to rename a budget template

```
[Authorize]
[HttpPut("{budgetTemplateId}")]
public Task<ActionResult> UpdateBudgetTemplateAsync(int budgetTemplateId,
EditBudgetTemplateRequest template)
```

Parameters

`budgetTemplateId` int

`template` EditBudgetTemplateRequest

Returns

Task<ActionResult>

Class DebtAccountController

Namespace: MakesCentsBackend.Controllers

Assembly: MakesCentsBackend.dll

```
[Route("api/debt-accounts")]
[ApiController]
public class DebtAccountController : ControllerBase
```

Inheritance

object ← ControllerBase ← DebtAccountController

Inherited Members

ControllerBase.StatusCode(int) , ControllerBase.StatusCode(int, object) ,
ControllerBase.Content(string) , ControllerBase.Content(string, string) ,
ControllerBase.Content(string, string, Encoding) ,
ControllerBase.Content(string, MediaTypeHeaderValue) , ControllerBase.NoContent() ,
ControllerBase.Ok() , ControllerBase.Ok(object) , ControllerBase.Redirect(string) ,
ControllerBase.RedirectPermanent(string) , ControllerBase.RedirectPreserveMethod(string) ,
ControllerBase.RedirectPermanentPreserveMethod(string) , ControllerBase.LocalRedirect(string) ,
ControllerBase.LocalRedirectPermanent(string) , ControllerBase.LocalRedirectPreserveMethod(string) ,
ControllerBase.LocalRedirectPermanentPreserveMethod(string) , ControllerBase.RedirectToAction() ,
ControllerBase.RedirectToAction(string) , ControllerBase.RedirectToAction(string, object) ,
ControllerBase.RedirectToAction(string, string) ,
ControllerBase.RedirectToAction(string, string, object) ,
ControllerBase.RedirectToAction(string, string, string) ,
ControllerBase.RedirectToAction(string, string, object, string) ,
ControllerBase.RedirectToActionPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToActionPermanent(string) ,
ControllerBase.RedirectToActionPermanent(string, object) ,
ControllerBase.RedirectToActionPermanent(string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, object) ,
ControllerBase.RedirectToActionPermanent(string, string, object, string) ,
ControllerBase.RedirectToActionPermanentPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToRoute(string) , ControllerBase.RedirectToRoute(object) ,
ControllerBase.RedirectToRoute(string, object) , ControllerBase.RedirectToRoute(string, string) ,
ControllerBase.RedirectToRoute(string, object, string) ,
ControllerBase.RedirectToRoutePreserveMethod(string, object, string) ,

ControllerBase.RedirectToRoutePermanent(string) ,
ControllerBase.RedirectToRoutePermanent(object) ,
ControllerBase.RedirectToRoutePermanent(string, object) ,
ControllerBase.RedirectToRoutePermanent(string, string) ,
ControllerBase.RedirectToRoutePermanent(string, object, string) ,
ControllerBase.RedirectToRoutePermanentPreserveMethod(string, object, string) ,
ControllerBase.RedirectToPage(string) , ControllerBase.RedirectToPage(string, object) ,
ControllerBase.RedirectToPage(string, string) , ControllerBase.RedirectToPage(string, string, object) ,
ControllerBase.RedirectToPage(string, string, string) ,
ControllerBase.RedirectToPage(string, string, object, string) ,
ControllerBase.RedirectToPagePermanent(string) ,
ControllerBase.RedirectToPagePermanent(string, object) ,
ControllerBase.RedirectToPagePermanent(string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, object, string) ,
ControllerBase.RedirectToPagePreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToPagePermanentPreserveMethod(string, string, object, string) ,
ControllerBase.File(byte[], string) , ControllerBase.File(byte[], string, bool) ,
ControllerBase.File(byte[], string, string) , ControllerBase.File(byte[], string, string, bool) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string) , ControllerBase.File(Stream, string, bool) ,
ControllerBase.File(Stream, string, string) , ControllerBase.File(Stream, string, string, bool) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string) , ControllerBase.File(string, string, bool) ,
ControllerBase.File(string, string, string) , ControllerBase.File(string, string, string, bool) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string) , ControllerBase.PhysicalFile(string, string, bool) ,
ControllerBase.PhysicalFile(string, string, string) ,
ControllerBase.PhysicalFile(string, string, string, bool) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,

ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.Unauthorized() , ControllerBase.Unauthorized(object) , ControllerBase.NotFound() ,
ControllerBase.NotFound(object) , ControllerBase.BadRequest() ,
ControllerBase.BadRequest(object) , ControllerBase.BadRequest(ModelStateDictionary) ,
ControllerBase.UnprocessableEntity() , ControllerBase.UnprocessableEntity(object) ,
ControllerBase.UnprocessableEntity(ModelStateDictionary) , ControllerBase.Conflict() ,
ControllerBase.Conflict(object) , ControllerBase.Conflict(ModelStateDictionary) ,
ControllerBase.Problem(string, string, int?, string, string) ,
ControllerBase.ValidationProblem(ValidationProblemDetails) ,
ControllerBase.ValidationProblem(ModelStateDictionary) , ControllerBase.ValidationProblem() ,
ControllerBase.ValidationProblem(string, string, int?, string, string, ModelStateDictionary) ,
ControllerBase.Created() , ControllerBase.Created(string, object) ,
ControllerBase.Created(Uri, object) , ControllerBase.CreatedAtAction(string, object) ,
ControllerBase.CreatedAtAction(string, object, object) ,
ControllerBase.CreatedAtAction(string, string, object, object) ,
ControllerBase.CreatedAtRoute(string, object) , ControllerBase.CreatedAtRoute(object, object) ,
ControllerBase.CreatedAtRoute(string, object, object) , ControllerBase.Accepted() ,
ControllerBase.Accepted(object) , ControllerBase.Accepted(Uri) , ControllerBase.Accepted(string) ,
ControllerBase.Accepted(string, object) , ControllerBase.Accepted(Uri, object) ,
ControllerBase.AcceptedAtAction(string) , ControllerBase.AcceptedAtAction(string, string) ,
ControllerBase.AcceptedAtAction(string, object) ,
ControllerBase.AcceptedAtAction(string, string, object) ,
ControllerBase.AcceptedAtAction(string, object, object) ,
ControllerBase.AcceptedAtAction(string, string, object, object) ,
ControllerBase.AcceptedAtRoute(object) , ControllerBase.AcceptedAtRoute(string) ,
ControllerBase.AcceptedAtRoute(string, object) , ControllerBase.AcceptedAtRoute(object, object) ,
ControllerBase.AcceptedAtRoute(string, object, object) , ControllerBase.Challenge() ,
ControllerBase.Challenge(params string[]) , ControllerBase.Challenge(AuthenticationProperties) ,
ControllerBase.Challenge(AuthenticationProperties, params string[]) , ControllerBase.Forbid() ,
ControllerBase.Forbid(params string[]) , ControllerBase.Forbid(AuthenticationProperties) ,
ControllerBase.Forbid(AuthenticationProperties, params string[]) ,
ControllerBase.SignIn(ClaimsPrincipal) , ControllerBase.SignIn(ClaimsPrincipal, string) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties, string) , ControllerBase.SignOut() ,
ControllerBase.SignOut(AuthenticationProperties) , ControllerBase.SignOut(params string[]) ,
ControllerBase.SignOut(AuthenticationProperties, params string[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider) ,

ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync(object, Type, string) ,
ControllerBase.TryUpdateModelAsync(object, Type, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryValidateModel(object) , ControllerBase.TryValidateModel(object, string) ,
ControllerBase.HttpContext , ControllerBase.Request , ControllerBase.Response ,
ControllerBase.RouteData , ControllerBase.ModelState , ControllerBase.ControllerContext ,
ControllerBase.MetadataProvider , ControllerBase.ModelBinderFactory , ControllerBase.Url ,
ControllerBase.ObjectValidator , ControllerBase.ProblemDetailsFactory , ControllerBase.User ,
ControllerBase.Empty , object.Equals(object) , object.Equals(object, object) ,
object.GetHashCode() , object.GetType() , object.MemberwiseClone() ,
object.ReferenceEquals(object, object) , object.ToString()

Constructors

DebtAccountController(DebtAccountLogic)

Parameterized constructor to bring in DI variables

```
public DebtAccountController(DebtAccountLogic debtAccountLogic)
```

Parameters

`debtAccountLogic` DebtAccountLogic

Methods

CreateDebtAccountAsync(CreateDebtAccountRequest)

HTTP POST method to create a new debt account

```
[Authorize]
[HttpPost]
public Task<ActionResult> CreateDebtAccountAsync(CreateDebtAccountRequest debtAccount)
```

Parameters

`debtAccount` CreateDebtAccountRequest

Returns

Task<ActionResult>

GetDebtAccountAsync(int)

```
[Authorize]
[HttpGet("{debtAccountId}")]
public Task<ActionResult> GetDebtAccountAsync(int debtAccountId)
```

Parameters

`debtAccountId` int

Returns

Task<ActionResult>

UpdateDebtAccountAsync(int, UpdateDebtAccountRequest)

```
[Authorize]
[HttpPut("{debtAccountId}")]
public Task<ActionResult> UpdateDebtAccountAsync(int debtAccountId,
UpdateDebtAccountRequest debtAccount)
```

Parameters

`debtAccountId` int

`debtAccount` UpdateDebtAccountRequest

Returns

Task<ActionResult>

Class EnvelopeCategoryController

Namespace: MakesCentsBackend.Controllers

Assembly: MakesCentsBackend.dll

API controller for envelope categories

```
[Route("api/envelope-categories")]
[ApiController]
public class EnvelopeCategoryController : ControllerBase
```

Inheritance

object ← ControllerBase ← EnvelopeCategoryController

Inherited Members

ControllerBase.StatusCode(int) , ControllerBase.StatusCode(int, object) ,
ControllerBase.Content(string) , ControllerBase.Content(string, string) ,
ControllerBase.Content(string, string, Encoding) ,
ControllerBase.Content(string, MediaTypeHeaderValue) , ControllerBase.NoContent() ,
ControllerBase.Ok() , ControllerBase.Ok(object) , ControllerBase.Redirect(string) ,
ControllerBase.RedirectPermanent(string) , ControllerBase.RedirectPreserveMethod(string) ,
ControllerBase.RedirectPermanentPreserveMethod(string) , ControllerBase.LocalRedirect(string) ,
ControllerBase.LocalRedirectPermanent(string) , ControllerBase.LocalRedirectPreserveMethod(string) ,
ControllerBase.LocalRedirectPermanentPreserveMethod(string) , ControllerBase.RedirectToAction() ,
ControllerBase.RedirectToAction(string) , ControllerBase.RedirectToAction(string, object) ,
ControllerBase.RedirectToAction(string, string) ,
ControllerBase.RedirectToAction(string, string, object) ,
ControllerBase.RedirectToAction(string, string, string) ,
ControllerBase.RedirectToAction(string, string, object, string) ,
ControllerBase.RedirectToActionPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToActionPermanent(string) ,
ControllerBase.RedirectToActionPermanent(string, object) ,
ControllerBase.RedirectToActionPermanent(string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, object) ,
ControllerBase.RedirectToActionPermanent(string, string, object, string) ,
ControllerBase.RedirectToActionPermanentPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToRoute(string) , ControllerBase.RedirectToRoute(object) ,
ControllerBase.RedirectToRoute(string, object) , ControllerBase.RedirectToRoute(string, string) ,

ControllerBase.RedirectToRoute(string, object, string) ,
ControllerBase.RedirectToRoutePreserveMethod(string, object, string) ,
ControllerBase.RedirectToRoutePermanent(string) ,
ControllerBase.RedirectToRoutePermanent(object) ,
ControllerBase.RedirectToRoutePermanent(string, object) ,
ControllerBase.RedirectToRoutePermanent(string, string) ,
ControllerBase.RedirectToRoutePermanent(string, object, string) ,
ControllerBase.RedirectToRoutePermanentPreserveMethod(string, object, string) ,
ControllerBase.RedirectToPage(string) , ControllerBase.RedirectToPage(string, object) ,
ControllerBase.RedirectToPage(string, string) , ControllerBase.RedirectToPage(string, string, object) ,
ControllerBase.RedirectToPage(string, string, string) ,
ControllerBase.RedirectToPage(string, string, object, string) ,
ControllerBase.RedirectToPagePermanent(string) ,
ControllerBase.RedirectToPagePermanent(string, object) ,
ControllerBase.RedirectToPagePermanent(string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, object, string) ,
ControllerBase.RedirectToPagePreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToPagePermanentPreserveMethod(string, string, object, string) ,
ControllerBase.File(byte[], string) , ControllerBase.File(byte[], string, bool) ,
ControllerBase.File(byte[], string, string) , ControllerBase.File(byte[], string, string, bool) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string) , ControllerBase.File(Stream, string, bool) ,
ControllerBase.File(Stream, string, string) , ControllerBase.File(Stream, string, string, bool) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string) , ControllerBase.File(string, string, bool) ,
ControllerBase.File(string, string, string) , ControllerBase.File(string, string, string, bool) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string) , ControllerBase.PhysicalFile(string, string, bool) ,
ControllerBase.PhysicalFile(string, string, string) ,
ControllerBase.PhysicalFile(string, string, string, bool) ,

ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.Unauthorized() , ControllerBase.Unauthorized(object) , ControllerBase.NotFound() ,
ControllerBase.NotFound(object) , ControllerBase.BadRequest() ,
ControllerBase.BadRequest(object) , ControllerBase.BadRequest(ModelStateDictionary) ,
ControllerBase.UnprocessableEntity() , ControllerBase.UnprocessableEntity(object) ,
ControllerBase.UnprocessableEntity(ModelStateDictionary) , ControllerBase.Conflict() ,
ControllerBase.Conflict(object) , ControllerBase.Conflict(ModelStateDictionary) ,
ControllerBase.Problem(string, string, int?, string, string) ,
ControllerBase.ValidationProblem(ValidationProblemDetails) ,
ControllerBase.ValidationProblem(ModelStateDictionary) , ControllerBase.ValidationProblem() ,
ControllerBase.ValidationProblem(string, string, int?, string, string, ModelStateDictionary) ,
ControllerBase.Created() , ControllerBase.Created(string, object) ,
ControllerBase.Created(Uri, object) , ControllerBase.CreatedAtAction(string, object) ,
ControllerBase.CreatedAtAction(string, object, object) ,
ControllerBase.CreatedAtAction(string, string, object, object) ,
ControllerBase.CreatedAtRoute(string, object) , ControllerBase.CreatedAtRoute(object, object) ,
ControllerBase.CreatedAtRoute(string, object, object) , ControllerBase.Accepted() ,
ControllerBase.Accepted(object) , ControllerBase.Accepted(Uri) , ControllerBase.Accepted(string) ,
ControllerBase.Accepted(string, object) , ControllerBase.Accepted(Uri, object) ,
ControllerBase.AcceptedAtAction(string) , ControllerBase.AcceptedAtAction(string, string) ,
ControllerBase.AcceptedAtAction(string, object) ,
ControllerBase.AcceptedAtAction(string, string, object) ,
ControllerBase.AcceptedAtAction(string, object, object) ,
ControllerBase.AcceptedAtAction(string, string, object, object) ,
ControllerBase.AcceptedAtRoute(object) , ControllerBase.AcceptedAtRoute(string) ,
ControllerBase.AcceptedAtRoute(string, object) , ControllerBase.AcceptedAtRoute(object, object) ,
ControllerBase.AcceptedAtRoute(string, object, object) , ControllerBase.Challenge() ,
ControllerBase.Challenge(params string[]) , ControllerBase.Challenge(AuthenticationProperties) ,
ControllerBase.Challenge(AuthenticationProperties, params string[]) , ControllerBase.Forbid() ,
ControllerBase.Forbid(params string[]) , ControllerBase.Forbid(AuthenticationProperties) ,
ControllerBase.Forbid(AuthenticationProperties, params string[]) ,
ControllerBase.SignIn(ClaimsPrincipal) , ControllerBase.SignIn(ClaimsPrincipal, string) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties, string) , ControllerBase.SignOut() ,
ControllerBase.SignOut(AuthenticationProperties) , ControllerBase.SignOut(params string[]) ,
ControllerBase.SignOut(AuthenticationProperties, params string[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel) ,

ControllerBase.TryUpdateModelAsync<TModel>(TModel, string) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync(object, Type, string) ,
ControllerBase.TryUpdateModelAsync(object, Type, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryValidateModel(object) , ControllerBase.TryValidateModel(object, string) ,
ControllerBase.HttpContext , ControllerBase.Request , ControllerBase.Response ,
ControllerBase.RouteData , ControllerBase.ModelState , ControllerBase.ControllerContext ,
ControllerBase.MetadataProvider , ControllerBase.ModelBinderFactory , ControllerBase.Url ,
ControllerBase.ObjectValidator , ControllerBase.ProblemDetailsFactory , ControllerBase.User ,
ControllerBase.Empty , object.Equals(object) , object.Equals(object, object) ,
object.GetHashCode() , object.GetType() , object.MemberwiseClone() ,
object.ReferenceEquals(object, object) , object.ToString()

Remarks

Parameterized constructor to bring in DI variables

Constructors

EnvelopeCategoryController(EnvelopeCategoryLogic)

Parameterized constructor to bring in DI variables

```
public EnvelopeCategoryController(EnvelopeCategoryLogic envelopeCategoryLogic)
```

Parameters

`envelopeCategoryLogic` EnvelopeCategoryLogic

Methods

CreateEnvelopeCategoryAsync(CreateEnvelopeCategoryRequest)

HTTP POST method to create an envelope category. Still keyed to a persistent BudgetId - categories aren't month-scoped.

```
[Authorize]
[HttpPost]
public Task<ActionResult> CreateEnvelopeCategoryAsync(CreateEnvelopeCategoryRequest
envelopeCategory)
```

Parameters

`envelopeCategory` CreateEnvelopeCategoryRequest

Returns

Task <ActionResult>

DeleteEnvelopeCategoryAsync(int)

Delete an envelope category

```
[Authorize]
[HttpDelete("{envelopeCategoryId}")]
public Task<ActionResult> DeleteEnvelopeCategoryAsync(int envelopeCategoryId)
```

Parameters

`envelopeCategoryId` int

Returns

Task <ActionResult>

GetAllEnvelopeCategoriesAsync(int)

HTTP GET method to get all envelope categories (with envelopes) for a budget month. NOTE: the route param is now a budget_month_id, not a budget_id - the DAO derives the persistent budget via a join, and needs the month to resolve rollover envelope amounts.

```
[Authorize]
[HttpGet("budget-month/{budgetMonthId}")]
public Task<ActionResult> GetAllEnvelopeCategoriesAsync(int budgetMonthId)
```

Parameters

`budgetMonthId` int

Returns

Task<ActionResult>

GetEnvelopeCategoryAsync(int)

HTTP GET method to get a single envelope category

```
[Authorize]
[HttpGet("{envelopeCategoryId}")]
public Task<ActionResult> GetEnvelopeCategoryAsync(int envelopeCategoryId)
```

Parameters

`envelopeCategoryId` int

Returns

Task<ActionResult>

UpdateEnvelopeCategoryAsync(int, EditEnvelopeCategoryRequest)

Update an envelope category based on the provided fields

```
[Authorize]
[HttpPut("{envelopeCategoryId}")]
public Task<ActionResult> UpdateEnvelopeCategoryAsync(int envelopeCategoryId,
EditEnvelopeCategoryRequest envelopeCategory)
```

Parameters

`envelopeCategoryId` int

`envelopeCategory` EditEnvelopeCategoryRequest

Returns

Task <ActionResult>

Class EnvelopeFundingController

Namespace: MakesCentsBackend.Controllers

Assembly: MakesCentsBackend.dll

API controller for the envelope funding queue - a budget month's plan for how much to put into each envelope and in what priority order.

```
[Route("api/envelope-funding")]
[ApiController]
public class EnvelopeFundingController : ControllerBase
```

Inheritance

object ← ControllerBase ← EnvelopeFundingController

Inherited Members

ControllerBase.StatusCode(int) , ControllerBase.StatusCode(int, object) ,
ControllerBase.Content(string) , ControllerBase.Content(string, string) ,
ControllerBase.Content(string, string, Encoding) ,
ControllerBase.Content(string, MediaTypeHeaderValue) , ControllerBase.NoContent() ,
ControllerBase.Ok() , ControllerBase.Ok(object) , ControllerBase.Redirect(string) ,
ControllerBase.RedirectPermanent(string) , ControllerBase.RedirectPreserveMethod(string) ,
ControllerBase.RedirectPermanentPreserveMethod(string) , ControllerBase.LocalRedirect(string) ,
ControllerBase.LocalRedirectPermanent(string) , ControllerBase.LocalRedirectPreserveMethod(string) ,
ControllerBase.LocalRedirectPermanentPreserveMethod(string) , ControllerBase.RedirectToAction() ,
ControllerBase.RedirectToAction(string) , ControllerBase.RedirectToAction(string, object) ,
ControllerBase.RedirectToAction(string, string) ,
ControllerBase.RedirectToAction(string, string, object) ,
ControllerBase.RedirectToAction(string, string, string) ,
ControllerBase.RedirectToAction(string, string, object, string) ,
ControllerBase.RedirectToActionPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToActionPermanent(string) ,
ControllerBase.RedirectToActionPermanent(string, object) ,
ControllerBase.RedirectToActionPermanent(string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, object) ,
ControllerBase.RedirectToActionPermanent(string, string, object, string) ,
ControllerBase.RedirectToActionPermanentPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToRoute(string) , ControllerBase.RedirectToRoute(object) ,

ControllerBase.RedirectToRoute(string, object) , ControllerBase.RedirectToRoute(string, string) ,
ControllerBase.RedirectToRoute(string, object, string) ,
ControllerBase.RedirectToRoutePreserveMethod(string, object, string) ,
ControllerBase.RedirectToRoutePermanent(string) ,
ControllerBase.RedirectToRoutePermanent(object) ,
ControllerBase.RedirectToRoutePermanent(string, object) ,
ControllerBase.RedirectToRoutePermanent(string, string) ,
ControllerBase.RedirectToRoutePermanent(string, object, string) ,
ControllerBase.RedirectToRoutePermanentPreserveMethod(string, object, string) ,
ControllerBase.RedirectToPage(string) , ControllerBase.RedirectToPage(string, object) ,
ControllerBase.RedirectToPage(string, string) , ControllerBase.RedirectToPage(string, string, object) ,
ControllerBase.RedirectToPage(string, string, string) ,
ControllerBase.RedirectToPage(string, string, object, string) ,
ControllerBase.RedirectToPagePermanent(string) ,
ControllerBase.RedirectToPagePermanent(string, object) ,
ControllerBase.RedirectToPagePermanent(string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, object, string) ,
ControllerBase.RedirectToPagePreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToPagePermanentPreserveMethod(string, string, object, string) ,
ControllerBase.File(byte[], string) , ControllerBase.File(byte[], string, bool) ,
ControllerBase.File(byte[], string, string) , ControllerBase.File(byte[], string, string, bool) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string) , ControllerBase.File(Stream, string, bool) ,
ControllerBase.File(Stream, string, string) , ControllerBase.File(Stream, string, string, bool) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string) , ControllerBase.File(string, string, bool) ,
ControllerBase.File(string, string, string) , ControllerBase.File(string, string, string, bool) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string) , ControllerBase.PhysicalFile(string, string, bool) ,
ControllerBase.PhysicalFile(string, string, string) ,

ControllerBase.PhysicalFile(string, string, string, bool) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.Unauthorized() , ControllerBase.Unauthorized(object) , ControllerBase.NotFound() ,
ControllerBase.NotFound(object) , ControllerBase.BadRequest() ,
ControllerBase.BadRequest(object) , ControllerBase.BadRequest(ModelStateDictionary) ,
ControllerBase.UnprocessableEntity() , ControllerBase.UnprocessableEntity(object) ,
ControllerBase.UnprocessableEntity(ModelStateDictionary) , ControllerBase.Conflict() ,
ControllerBase.Conflict(object) , ControllerBase.Conflict(ModelStateDictionary) ,
ControllerBase.Problem(string, string, int?, string, string) ,
ControllerBase.ValidationProblem(ValidationProblemDetails) ,
ControllerBase.ValidationProblem(ModelStateDictionary) , ControllerBase.ValidationProblem() ,
ControllerBase.ValidationProblem(string, string, int?, string, string, ModelStateDictionary) ,
ControllerBase.Created() , ControllerBase.Created(string, object) ,
ControllerBase.Created(Uri, object) , ControllerBase.CreatedAtAction(string, object) ,
ControllerBase.CreatedAtAction(string, object, object) ,
ControllerBase.CreatedAtAction(string, string, object, object) ,
ControllerBase.CreatedAtRoute(string, object) , ControllerBase.CreatedAtRoute(object, object) ,
ControllerBase.CreatedAtRoute(string, object, object) , ControllerBase.Accepted() ,
ControllerBase.Accepted(object) , ControllerBase.Accepted(Uri) , ControllerBase.Accepted(string) ,
ControllerBase.Accepted(string, object) , ControllerBase.Accepted(Uri, object) ,
ControllerBase.AcceptedAtAction(string) , ControllerBase.AcceptedAtAction(string, string) ,
ControllerBase.AcceptedAtAction(string, object) ,
ControllerBase.AcceptedAtAction(string, string, object) ,
ControllerBase.AcceptedAtAction(string, object, object) ,
ControllerBase.AcceptedAtAction(string, string, object, object) ,
ControllerBase.AcceptedAtRoute(object) , ControllerBase.AcceptedAtRoute(string) ,
ControllerBase.AcceptedAtRoute(string, object) , ControllerBase.AcceptedAtRoute(object, object) ,
ControllerBase.AcceptedAtRoute(string, object, object) , ControllerBase.Challenge() ,
ControllerBase.Challenge(params string[]) , ControllerBase.Challenge(AuthenticationProperties) ,
ControllerBase.Challenge(AuthenticationProperties, params string[]) , ControllerBase.Forbid() ,
ControllerBase.Forbid(params string[]) , ControllerBase.Forbid(AuthenticationProperties) ,
ControllerBase.Forbid(AuthenticationProperties, params string[]) ,
ControllerBase.SignIn(ClaimsPrincipal) , ControllerBase.SignIn(ClaimsPrincipal, string) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties, string) , ControllerBase.SignOut() ,
ControllerBase.SignOut(AuthenticationProperties) , ControllerBase.SignOut(params string[]) ,
ControllerBase.SignOut(AuthenticationProperties, params string[]) ,

ControllerBase.TryUpdateModelAsync<TModel>(TModel) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync(object, Type, string) ,
ControllerBase.TryUpdateModelAsync(object, Type, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryValidateModel(object) , ControllerBase.TryValidateModel(object, string) ,
ControllerBase.HttpContext , ControllerBase.Request , ControllerBase.Response ,
ControllerBase.RouteData , ControllerBase.ModelState , ControllerBase.ControllerContext ,
ControllerBase.MetadataProvider , ControllerBase.ModelBinderFactory , ControllerBase.Url ,
ControllerBase.ObjectValidator , ControllerBase.ProblemDetailsFactory , ControllerBase.User ,
ControllerBase.Empty , object.Equals(object) , object.Equals(object, object) ,
object.GetHashCode() , object.GetType() , object.MemberwiseClone() ,
object.ReferenceEquals(object, object) , object.ToString()

Constructors

EnvelopeFundingController(EnvelopeFundingLogic)

Parameterized constructor to bring in DI variables

```
public EnvelopeFundingController(EnvelopeFundingLogic envelopeFundingLogic)
```

Parameters

`envelopeFundingLogic` EnvelopeFundingLogic

Methods

DeleteEnvelopeFundingAsync(int)

HTTP DELETE method to delete a single envelope funding row, without touching the rest of the queue

```
[Authorize]
[HttpDelete("{envelopeFundingId}")]
public Task<ActionResult> DeleteEnvelopeFundingAsync(int envelopeFundingId)
```

Parameters

`envelopeFundingId` int

Returns

Task <ActionResult>

GetEnvelopeFundingQueueAsync(int)

HTTP GET method to get the funding queue for a budget month

```
[Authorize]
[HttpGet("budget-month/{budgetMonthId}")]
public Task<ActionResult> GetEnvelopeFundingQueueAsync(int budgetMonthId)
```

Parameters

`budgetMonthId` int

Returns

Task <ActionResult>

SetEnvelopeFundingQueueAsync(int, SetEnvelopeFundingQueueRequest)

HTTP PUT method to set (create/reorder/update/remove all at once) the funding queue for a budget month

```
[Authorize]
[HttpPut("budget-month/{budgetMonthId}")]
```

```
public Task<ActionResult> SetEnvelopeFundingQueueAsync(int budgetMonthId,
SetEnvelopeFundingQueueRequest fundingQueue)
```

Parameters

`budgetMonthId` int

`fundingQueue` SetEnvelopeFundingQueueRequest

Returns

Task <ActionResult>

Class InvestmentAccountController

Namespace: MakesCentsBackend.Controllers

Assembly: MakesCentsBackend.dll

```
[Route("api/investment-accounts")]
[ApiController]
public class InvestmentAccountController : ControllerBase
```

Inheritance

object ← ControllerBase ← InvestmentAccountController

Inherited Members

ControllerBase.StatusCode(int) , ControllerBase.StatusCode(int, object) ,
ControllerBase.Content(string) , ControllerBase.Content(string, string) ,
ControllerBase.Content(string, string, Encoding) ,
ControllerBase.Content(string, MediaTypeHeaderValue) , ControllerBase.NoContent() ,
ControllerBase.Ok() , ControllerBase.Ok(object) , ControllerBase.Redirect(string) ,
ControllerBase.RedirectPermanent(string) , ControllerBase.RedirectPreserveMethod(string) ,
ControllerBase.RedirectPermanentPreserveMethod(string) , ControllerBase.LocalRedirect(string) ,
ControllerBase.LocalRedirectPermanent(string) , ControllerBase.LocalRedirectPreserveMethod(string) ,
ControllerBase.LocalRedirectPermanentPreserveMethod(string) , ControllerBase.RedirectToAction() ,
ControllerBase.RedirectToAction(string) , ControllerBase.RedirectToAction(string, object) ,
ControllerBase.RedirectToAction(string, string) ,
ControllerBase.RedirectToAction(string, string, object) ,
ControllerBase.RedirectToAction(string, string, string) ,
ControllerBase.RedirectToAction(string, string, object, string) ,
ControllerBase.RedirectToActionPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToActionPermanent(string) ,
ControllerBase.RedirectToActionPermanent(string, object) ,
ControllerBase.RedirectToActionPermanent(string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, object) ,
ControllerBase.RedirectToActionPermanent(string, string, object, string) ,
ControllerBase.RedirectToActionPermanentPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToRoute(string) , ControllerBase.RedirectToRoute(object) ,
ControllerBase.RedirectToRoute(string, object) , ControllerBase.RedirectToRoute(string, string) ,
ControllerBase.RedirectToRoute(string, object, string) ,
ControllerBase.RedirectToRoutePreserveMethod(string, object, string) ,

ControllerBase.RedirectToRoutePermanent(string) ,
ControllerBase.RedirectToRoutePermanent(object) ,
ControllerBase.RedirectToRoutePermanent(string, object) ,
ControllerBase.RedirectToRoutePermanent(string, string) ,
ControllerBase.RedirectToRoutePermanent(string, object, string) ,
ControllerBase.RedirectToRoutePermanentPreserveMethod(string, object, string) ,
ControllerBase.RedirectToPage(string) , ControllerBase.RedirectToPage(string, object) ,
ControllerBase.RedirectToPage(string, string) , ControllerBase.RedirectToPage(string, string, object) ,
ControllerBase.RedirectToPage(string, string, string) ,
ControllerBase.RedirectToPage(string, string, object, string) ,
ControllerBase.RedirectToPagePermanent(string) ,
ControllerBase.RedirectToPagePermanent(string, object) ,
ControllerBase.RedirectToPagePermanent(string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, object, string) ,
ControllerBase.RedirectToPagePreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToPagePermanentPreserveMethod(string, string, object, string) ,
ControllerBase.File(byte[], string) , ControllerBase.File(byte[], string, bool) ,
ControllerBase.File(byte[], string, string) , ControllerBase.File(byte[], string, string, bool) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string) , ControllerBase.File(Stream, string, bool) ,
ControllerBase.File(Stream, string, string) , ControllerBase.File(Stream, string, string, bool) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string) , ControllerBase.File(string, string, bool) ,
ControllerBase.File(string, string, string) , ControllerBase.File(string, string, string, bool) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string) , ControllerBase.PhysicalFile(string, string, bool) ,
ControllerBase.PhysicalFile(string, string, string) ,
ControllerBase.PhysicalFile(string, string, string, bool) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,

ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.Unauthorized() , ControllerBase.Unauthorized(object) , ControllerBase.NotFound() ,
ControllerBase.NotFound(object) , ControllerBase.BadRequest() ,
ControllerBase.BadRequest(object) , ControllerBase.BadRequest(ModelStateDictionary) ,
ControllerBase.UnprocessableEntity() , ControllerBase.UnprocessableEntity(object) ,
ControllerBase.UnprocessableEntity(ModelStateDictionary) , ControllerBase.Conflict() ,
ControllerBase.Conflict(object) , ControllerBase.Conflict(ModelStateDictionary) ,
ControllerBase.Problem(string, string, int?, string, string) ,
ControllerBase.ValidationProblem(ValidationProblemDetails) ,
ControllerBase.ValidationProblem(ModelStateDictionary) , ControllerBase.ValidationProblem() ,
ControllerBase.ValidationProblem(string, string, int?, string, string, ModelStateDictionary) ,
ControllerBase.Created() , ControllerBase.Created(string, object) ,
ControllerBase.Created(Uri, object) , ControllerBase.CreatedAtAction(string, object) ,
ControllerBase.CreatedAtAction(string, object, object) ,
ControllerBase.CreatedAtAction(string, string, object, object) ,
ControllerBase.CreatedAtRoute(string, object) , ControllerBase.CreatedAtRoute(object, object) ,
ControllerBase.CreatedAtRoute(string, object, object) , ControllerBase.Accepted() ,
ControllerBase.Accepted(object) , ControllerBase.Accepted(Uri) , ControllerBase.Accepted(string) ,
ControllerBase.Accepted(string, object) , ControllerBase.Accepted(Uri, object) ,
ControllerBase.AcceptedAtAction(string) , ControllerBase.AcceptedAtAction(string, string) ,
ControllerBase.AcceptedAtAction(string, object) ,
ControllerBase.AcceptedAtAction(string, string, object) ,
ControllerBase.AcceptedAtAction(string, object, object) ,
ControllerBase.AcceptedAtAction(string, string, object, object) ,
ControllerBase.AcceptedAtRoute(object) , ControllerBase.AcceptedAtRoute(string) ,
ControllerBase.AcceptedAtRoute(string, object) , ControllerBase.AcceptedAtRoute(object, object) ,
ControllerBase.AcceptedAtRoute(string, object, object) , ControllerBase.Challenge() ,
ControllerBase.Challenge(params string[]) , ControllerBase.Challenge(AuthenticationProperties) ,
ControllerBase.Challenge(AuthenticationProperties, params string[]) , ControllerBase.Forbid() ,
ControllerBase.Forbid(params string[]) , ControllerBase.Forbid(AuthenticationProperties) ,
ControllerBase.Forbid(AuthenticationProperties, params string[]) ,
ControllerBase.SignIn(ClaimsPrincipal) , ControllerBase.SignIn(ClaimsPrincipal, string) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties, string) , ControllerBase.SignOut() ,
ControllerBase.SignOut(AuthenticationProperties) , ControllerBase.SignOut(params string[]) ,
ControllerBase.SignOut(AuthenticationProperties, params string[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider) ,

ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync(object, Type, string) ,
ControllerBase.TryUpdateModelAsync(object, Type, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryValidateModel(object) , ControllerBase.TryValidateModel(object, string) ,
ControllerBase.HttpContext , ControllerBase.Request , ControllerBase.Response ,
ControllerBase.RouteData , ControllerBase.ModelState , ControllerBase.ControllerContext ,
ControllerBase.MetadataProvider , ControllerBase.ModelBinderFactory , ControllerBase.Url ,
ControllerBase.ObjectValidator , ControllerBase.ProblemDetailsFactory , ControllerBase.User ,
ControllerBase.Empty , object.Equals(object) , object.Equals(object, object) ,
object.GetHashCode() , object.GetType() , object.MemberwiseClone() ,
object.ReferenceEquals(object, object) , object.ToString()

Constructors

InvestmentAccountController(InvestmentAccountLogic)

Parameterized constructor to bring in DI variables

```
public InvestmentAccountController(InvestmentAccountLogic investmentAccountLogic)
```

Parameters

`investmentAccountLogic` InvestmentAccountLogic

Methods

CreateInvestmentAccountAsync(CreateInvestmentAccountRequest)

HTTP POST method to create a new investment account

```
[Authorize]
[HttpPost]
public Task<ActionResult> CreateInvestmentAccountAsync(CreateInvestmentAccountRequest
investmentAccount)
```

Parameters

`investmentAccount` CreateInvestmentAccountRequest

Returns

Task<ActionResult>

GetInvestmentAccountAsync(int)

```
[Authorize]
[HttpGet("{investmentAccountId}")]
public Task<ActionResult> GetInvestmentAccountAsync(int investmentAccountId)
```

Parameters

`investmentAccountId` int

Returns

Task<ActionResult>

UpdateInvestmentAccountAsync(int, UpdateInvestmentAccountRequest)

```
[Authorize]
[HttpPut("{investmentAccountId}")]
public Task<ActionResult> UpdateInvestmentAccountAsync(int investmentAccountId,
UpdateInvestmentAccountRequest investmentAccount)
```

Parameters

`investmentAccountId` int

`investmentAccount` UpdateInvestmentAccountRequest

Returns

Task<ActionResult>

Class PaycheckController

Namespace: MakesCentsBackend.Controllers

Assembly: MakesCentsBackend.dll

API controller for paychecks. Paychecks are forecast-only - see EnvelopeFundingController for how envelopes actually get funded.

```
[Route("api/paychecks")]
[ApiController]
public class PaycheckController : ControllerBase
```

Inheritance

object ← ControllerBase ← PaycheckController

Inherited Members

ControllerBase.StatusCode(int) , ControllerBase.StatusCode(int, object) ,
ControllerBase.Content(string) , ControllerBase.Content(string, string) ,
ControllerBase.Content(string, string, Encoding) ,
ControllerBase.Content(string, MediaTypeHeaderValue) , ControllerBase.NoContent() ,
ControllerBase.Ok() , ControllerBase.Ok(object) , ControllerBase.Redirect(string) ,
ControllerBase.RedirectPermanent(string) , ControllerBase.RedirectPreserveMethod(string) ,
ControllerBase.RedirectPermanentPreserveMethod(string) , ControllerBase.LocalRedirect(string) ,
ControllerBase.LocalRedirectPermanent(string) , ControllerBase.LocalRedirectPreserveMethod(string) ,
ControllerBase.LocalRedirectPermanentPreserveMethod(string) , ControllerBase.RedirectToAction() ,
ControllerBase.RedirectToAction(string) , ControllerBase.RedirectToAction(string, object) ,
ControllerBase.RedirectToAction(string, string) ,
ControllerBase.RedirectToAction(string, string, object) ,
ControllerBase.RedirectToAction(string, string, string) ,
ControllerBase.RedirectToAction(string, string, object, string) ,
ControllerBase.RedirectToActionPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToActionPermanent(string) ,
ControllerBase.RedirectToActionPermanent(string, object) ,
ControllerBase.RedirectToActionPermanent(string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, object) ,
ControllerBase.RedirectToActionPermanent(string, string, object, string) ,
ControllerBase.RedirectToActionPermanentPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToRoute(string) , ControllerBase.RedirectToRoute(object) ,

ControllerBase.RedirectToRoute(string, object) , ControllerBase.RedirectToRoute(string, string) ,
ControllerBase.RedirectToRoute(string, object, string) ,
ControllerBase.RedirectToRoutePreserveMethod(string, object, string) ,
ControllerBase.RedirectToRoutePermanent(string) ,
ControllerBase.RedirectToRoutePermanent(object) ,
ControllerBase.RedirectToRoutePermanent(string, object) ,
ControllerBase.RedirectToRoutePermanent(string, string) ,
ControllerBase.RedirectToRoutePermanent(string, object, string) ,
ControllerBase.RedirectToRoutePermanentPreserveMethod(string, object, string) ,
ControllerBase.RedirectToPage(string) , ControllerBase.RedirectToPage(string, object) ,
ControllerBase.RedirectToPage(string, string) , ControllerBase.RedirectToPage(string, string, object) ,
ControllerBase.RedirectToPage(string, string, string) ,
ControllerBase.RedirectToPage(string, string, object, string) ,
ControllerBase.RedirectToPagePermanent(string) ,
ControllerBase.RedirectToPagePermanent(string, object) ,
ControllerBase.RedirectToPagePermanent(string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, object, string) ,
ControllerBase.RedirectToPagePreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToPagePermanentPreserveMethod(string, string, object, string) ,
ControllerBase.File(byte[], string) , ControllerBase.File(byte[], string, bool) ,
ControllerBase.File(byte[], string, string) , ControllerBase.File(byte[], string, string, bool) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string) , ControllerBase.File(Stream, string, bool) ,
ControllerBase.File(Stream, string, string) , ControllerBase.File(Stream, string, string, bool) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string) , ControllerBase.File(string, string, bool) ,
ControllerBase.File(string, string, string) , ControllerBase.File(string, string, string, bool) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string) , ControllerBase.PhysicalFile(string, string, bool) ,
ControllerBase.PhysicalFile(string, string, string) ,

ControllerBase.PhysicalFile(string, string, string, bool) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.Unauthorized() , ControllerBase.Unauthorized(object) , ControllerBase.NotFound() ,
ControllerBase.NotFound(object) , ControllerBase.BadRequest() ,
ControllerBase.BadRequest(object) , ControllerBase.BadRequest(ModelStateDictionary) ,
ControllerBase.UnprocessableEntity() , ControllerBase.UnprocessableEntity(object) ,
ControllerBase.UnprocessableEntity(ModelStateDictionary) , ControllerBase.Conflict() ,
ControllerBase.Conflict(object) , ControllerBase.Conflict(ModelStateDictionary) ,
ControllerBase.Problem(string, string, int?, string, string) ,
ControllerBase.ValidationProblem(ValidationProblemDetails) ,
ControllerBase.ValidationProblem(ModelStateDictionary) , ControllerBase.ValidationProblem() ,
ControllerBase.ValidationProblem(string, string, int?, string, string, ModelStateDictionary) ,
ControllerBase.Created() , ControllerBase.Created(string, object) ,
ControllerBase.Created(Uri, object) , ControllerBase.CreatedAtAction(string, object) ,
ControllerBase.CreatedAtAction(string, object, object) ,
ControllerBase.CreatedAtAction(string, string, object, object) ,
ControllerBase.CreatedAtRoute(string, object) , ControllerBase.CreatedAtRoute(object, object) ,
ControllerBase.CreatedAtRoute(string, object, object) , ControllerBase.Accepted() ,
ControllerBase.Accepted(object) , ControllerBase.Accepted(Uri) , ControllerBase.Accepted(string) ,
ControllerBase.Accepted(string, object) , ControllerBase.Accepted(Uri, object) ,
ControllerBase.AcceptedAtAction(string) , ControllerBase.AcceptedAtAction(string, string) ,
ControllerBase.AcceptedAtAction(string, object) ,
ControllerBase.AcceptedAtAction(string, string, object) ,
ControllerBase.AcceptedAtAction(string, object, object) ,
ControllerBase.AcceptedAtAction(string, string, object, object) ,
ControllerBase.AcceptedAtRoute(object) , ControllerBase.AcceptedAtRoute(string) ,
ControllerBase.AcceptedAtRoute(string, object) , ControllerBase.AcceptedAtRoute(object, object) ,
ControllerBase.AcceptedAtRoute(string, object, object) , ControllerBase.Challenge() ,
ControllerBase.Challenge(params string[]) , ControllerBase.Challenge(AuthenticationProperties) ,
ControllerBase.Challenge(AuthenticationProperties, params string[]) , ControllerBase.Forbid() ,
ControllerBase.Forbid(params string[]) , ControllerBase.Forbid(AuthenticationProperties) ,
ControllerBase.Forbid(AuthenticationProperties, params string[]) ,
ControllerBase.SignIn(ClaimsPrincipal) , ControllerBase.SignIn(ClaimsPrincipal, string) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties, string) , ControllerBase.SignOut() ,
ControllerBase.SignOut(AuthenticationProperties) , ControllerBase.SignOut(params string[]) ,
ControllerBase.SignOut(AuthenticationProperties, params string[]) ,

ControllerBase.TryUpdateModelAsync<TModel>(TModel) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync(object, Type, string) ,
ControllerBase.TryUpdateModelAsync(object, Type, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryValidateModel(object) , ControllerBase.TryValidateModel(object, string) ,
ControllerBase.HttpContext , ControllerBase.Request , ControllerBase.Response ,
ControllerBase.RouteData , ControllerBase.ModelState , ControllerBase.ControllerContext ,
ControllerBase.MetadataProvider , ControllerBase.ModelBinderFactory , ControllerBase.Url ,
ControllerBase.ObjectValidator , ControllerBase.ProblemDetailsFactory , ControllerBase.User ,
ControllerBase.Empty , object.Equals(object) , object.Equals(object, object) ,
object.GetHashCode() , object.GetType() , object.MemberwiseClone() ,
object.ReferenceEquals(object, object) , object.ToString()

Constructors

PaycheckController(PaycheckLogic)

Parameterized constructor to bring in DI variables

```
public PaycheckController(PaycheckLogic paycheckLogic)
```

Parameters

`paycheckLogic` PaycheckLogic

Methods

CreatePaycheckAsync(CreatePaycheckRequest)

HTTP POST method to create a new paycheck

```
[Authorize]
[HttpPost]
public Task<ActionResult> CreatePaycheckAsync(CreatePaycheckRequest paycheck)
```

Parameters

`paycheck` CreatePaycheckRequest

Returns

Task<ActionResult>

DeletePaycheckAsync(int)

HTTP DELETE method to delete a paycheck

```
[Authorize]
[HttpDelete("{paycheckId}")]
public Task<ActionResult> DeletePaycheckAsync(int paycheckId)
```

Parameters

`paycheckId` int

Returns

Task<ActionResult>

GetAllPaychecksAsync(int)

HTTP GET method to get every paycheck for a budget

```
[Authorize]
[HttpGet("budget/{budgetId}")]
public Task<ActionResult> GetAllPaychecksAsync(int budgetId)
```

Parameters

`budgetId` int

Returns

Task<ActionResult>

GetPaycheckAsync(int)

HTTP GET method to get a single paycheck

```
[Authorize]
[HttpGet("{paycheckId}")]
public Task<ActionResult> GetPaycheckAsync(int paycheckId)
```

Parameters

`paycheckId` int

Returns

Task<ActionResult>

UpdatePaycheckAsync(int, UpdatePaycheckRequest)

HTTP PUT method to update a paycheck

```
[Authorize]
[HttpPut("{paycheckId}")]
public Task<ActionResult> UpdatePaycheckAsync(int paycheckId, UpdatePaycheckRequest
paycheck)
```

Parameters

`paycheckId` int

`paycheck` UpdatePaycheckRequest

Returns

Task<ActionResult>

Class PaymentTransactionController

Namespace: MakesCentsBackend.Controllers

Assembly: MakesCentsBackend.dll

```
[Route("api/payment-transactions")]
[ApiController]
public class PaymentTransactionController : ControllerBase
```

Inheritance

object ← ControllerBase ← PaymentTransactionController

Inherited Members

ControllerBase.StatusCode(int) , ControllerBase.StatusCode(int, object) ,
ControllerBase.Content(string) , ControllerBase.Content(string, string) ,
ControllerBase.Content(string, string, Encoding) ,
ControllerBase.Content(string, MediaTypeHeaderValue) , ControllerBase.NoContent() ,
ControllerBase.Ok() , ControllerBase.Ok(object) , ControllerBase.Redirect(string) ,
ControllerBase.RedirectPermanent(string) , ControllerBase.RedirectPreserveMethod(string) ,
ControllerBase.RedirectPermanentPreserveMethod(string) , ControllerBase.LocalRedirect(string) ,
ControllerBase.LocalRedirectPermanent(string) , ControllerBase.LocalRedirectPreserveMethod(string) ,
ControllerBase.LocalRedirectPermanentPreserveMethod(string) , ControllerBase.RedirectToAction() ,
ControllerBase.RedirectToAction(string) , ControllerBase.RedirectToAction(string, object) ,
ControllerBase.RedirectToAction(string, string) ,
ControllerBase.RedirectToAction(string, string, object) ,
ControllerBase.RedirectToAction(string, string, string) ,
ControllerBase.RedirectToAction(string, string, object, string) ,
ControllerBase.RedirectToActionPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToActionPermanent(string) ,
ControllerBase.RedirectToActionPermanent(string, object) ,
ControllerBase.RedirectToActionPermanent(string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, object) ,
ControllerBase.RedirectToActionPermanent(string, string, object, string) ,
ControllerBase.RedirectToActionPermanentPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToRoute(string) , ControllerBase.RedirectToRoute(object) ,
ControllerBase.RedirectToRoute(string, object) , ControllerBase.RedirectToRoute(string, string) ,
ControllerBase.RedirectToRoute(string, object, string) ,
ControllerBase.RedirectToRoutePreserveMethod(string, object, string) ,

ControllerBase.RedirectToRoutePermanent(string) ,
ControllerBase.RedirectToRoutePermanent(object) ,
ControllerBase.RedirectToRoutePermanent(string, object) ,
ControllerBase.RedirectToRoutePermanent(string, string) ,
ControllerBase.RedirectToRoutePermanent(string, object, string) ,
ControllerBase.RedirectToRoutePermanentPreserveMethod(string, object, string) ,
ControllerBase.RedirectToPage(string) , ControllerBase.RedirectToPage(string, object) ,
ControllerBase.RedirectToPage(string, string) , ControllerBase.RedirectToPage(string, string, object) ,
ControllerBase.RedirectToPage(string, string, string) ,
ControllerBase.RedirectToPage(string, string, object, string) ,
ControllerBase.RedirectToPagePermanent(string) ,
ControllerBase.RedirectToPagePermanent(string, object) ,
ControllerBase.RedirectToPagePermanent(string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, object, string) ,
ControllerBase.RedirectToPagePreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToPagePermanentPreserveMethod(string, string, object, string) ,
ControllerBase.File(byte[], string) , ControllerBase.File(byte[], string, bool) ,
ControllerBase.File(byte[], string, string) , ControllerBase.File(byte[], string, string, bool) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string) , ControllerBase.File(Stream, string, bool) ,
ControllerBase.File(Stream, string, string) , ControllerBase.File(Stream, string, string, bool) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string) , ControllerBase.File(string, string, bool) ,
ControllerBase.File(string, string, string) , ControllerBase.File(string, string, string, bool) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string) , ControllerBase.PhysicalFile(string, string, bool) ,
ControllerBase.PhysicalFile(string, string, string) ,
ControllerBase.PhysicalFile(string, string, string, bool) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,

ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.Unauthorized() , ControllerBase.Unauthorized(object) , ControllerBase.NotFound() ,
ControllerBase.NotFound(object) , ControllerBase.BadRequest() ,
ControllerBase.BadRequest(object) , ControllerBase.BadRequest(ModelStateDictionary) ,
ControllerBase.UnprocessableEntity() , ControllerBase.UnprocessableEntity(object) ,
ControllerBase.UnprocessableEntity(ModelStateDictionary) , ControllerBase.Conflict() ,
ControllerBase.Conflict(object) , ControllerBase.Conflict(ModelStateDictionary) ,
ControllerBase.Problem(string, string, int?, string, string) ,
ControllerBase.ValidationProblem(ValidationProblemDetails) ,
ControllerBase.ValidationProblem(ModelStateDictionary) , ControllerBase.ValidationProblem() ,
ControllerBase.ValidationProblem(string, string, int?, string, string, ModelStateDictionary) ,
ControllerBase.Created() , ControllerBase.Created(string, object) ,
ControllerBase.Created(Uri, object) , ControllerBase.CreatedAtAction(string, object) ,
ControllerBase.CreatedAtAction(string, object, object) ,
ControllerBase.CreatedAtAction(string, string, object, object) ,
ControllerBase.CreatedAtRoute(string, object) , ControllerBase.CreatedAtRoute(object, object) ,
ControllerBase.CreatedAtRoute(string, object, object) , ControllerBase.Accepted() ,
ControllerBase.Accepted(object) , ControllerBase.Accepted(Uri) , ControllerBase.Accepted(string) ,
ControllerBase.Accepted(string, object) , ControllerBase.Accepted(Uri, object) ,
ControllerBase.AcceptedAtAction(string) , ControllerBase.AcceptedAtAction(string, string) ,
ControllerBase.AcceptedAtAction(string, object) ,
ControllerBase.AcceptedAtAction(string, string, object) ,
ControllerBase.AcceptedAtAction(string, object, object) ,
ControllerBase.AcceptedAtAction(string, string, object, object) ,
ControllerBase.AcceptedAtRoute(object) , ControllerBase.AcceptedAtRoute(string) ,
ControllerBase.AcceptedAtRoute(string, object) , ControllerBase.AcceptedAtRoute(object, object) ,
ControllerBase.AcceptedAtRoute(string, object, object) , ControllerBase.Challenge() ,
ControllerBase.Challenge(params string[]) , ControllerBase.Challenge(AuthenticationProperties) ,
ControllerBase.Challenge(AuthenticationProperties, params string[]) , ControllerBase.Forbid() ,
ControllerBase.Forbid(params string[]) , ControllerBase.Forbid(AuthenticationProperties) ,
ControllerBase.Forbid(AuthenticationProperties, params string[]) ,
ControllerBase.SignIn(ClaimsPrincipal) , ControllerBase.SignIn(ClaimsPrincipal, string) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties, string) , ControllerBase.SignOut() ,
ControllerBase.SignOut(AuthenticationProperties) , ControllerBase.SignOut(params string[]) ,
ControllerBase.SignOut(AuthenticationProperties, params string[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider) ,

ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync(object, Type, string) ,
ControllerBase.TryUpdateModelAsync(object, Type, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryValidateModel(object) , ControllerBase.TryValidateModel(object, string) ,
ControllerBase.HttpContext , ControllerBase.Request , ControllerBase.Response ,
ControllerBase.RouteData , ControllerBase.ModelState , ControllerBase.ControllerContext ,
ControllerBase.MetadataProvider , ControllerBase.ModelBinderFactory , ControllerBase.Url ,
ControllerBase.ObjectValidator , ControllerBase.ProblemDetailsFactory , ControllerBase.User ,
ControllerBase.Empty , object.Equals(object) , object.Equals(object, object) ,
object.GetHashCode() , object.GetType() , object.MemberwiseClone() ,
object.ReferenceEquals(object, object) , object.ToString()

Constructors

PaymentTransactionController(PaymentTransactionLogic)

Parameterized constructor to bring in DI variables

```
public PaymentTransactionController(PaymentTransactionLogic paymentTransactionLogic)
```

Parameters

`paymentTransactionLogic` PaymentTransactionLogic

Methods

CreatePaymentTransactionAsync(CreatePaymentTransactionRequest)

HTTP POST request to create a new payment transaction

```
[Authorize]
[HttpPost]
public Task<ActionResult> CreatePaymentTransactionAsync(CreatePaymentTransactionRequest
paymentTransaction)
```

Parameters

`paymentTransaction` CreatePaymentTransactionRequest

Returns

Task<ActionResult>

GetPaymentTransactionAsync(int)

HTTP GET method to get a payment transaction and its splits

```
[Authorize]
[HttpGet("{paymentTransactionId}")]
public Task<ActionResult> GetPaymentTransactionAsync(int paymentTransactionId)
```

Parameters

`paymentTransactionId` int

Returns

Task<ActionResult>

UpdatePaymentTransactionAsync(int, UpdatePaymentTransactionRequest)

HTTP PUT method to update a payment transaction and its splits

```
[Authorize]
[HttpPut("{paymentTransactionId}")]
```

```
public Task<ActionResult> UpdatePaymentTransactionAsync(int paymentTransactionId,
UpdatePaymentTransactionRequest paymentTransaction)
```

Parameters

`paymentTransactionId` int

`paymentTransaction` UpdatePaymentTransactionRequest

Returns

Task <ActionResult>

Class PlannedExpenseController

Namespace: MakesCentsBackend.Controllers

Assembly: MakesCentsBackend.dll

API controller for planned expenses

```
[Route("api/[controller]")]
[ApiController]
public class PlannedExpenseController : ControllerBase
```

Inheritance

object ← ControllerBase ← PlannedExpenseController

Inherited Members

ControllerBase.StatusCode(int) , ControllerBase.StatusCode(int, object) ,
ControllerBase.Content(string) , ControllerBase.Content(string, string) ,
ControllerBase.Content(string, string, Encoding) ,
ControllerBase.Content(string, MediaTypeHeaderValue) , ControllerBase.NoContent() ,
ControllerBase.Ok() , ControllerBase.Ok(object) , ControllerBase.Redirect(string) ,
ControllerBase.RedirectPermanent(string) , ControllerBase.RedirectPreserveMethod(string) ,
ControllerBase.RedirectPermanentPreserveMethod(string) , ControllerBase.LocalRedirect(string) ,
ControllerBase.LocalRedirectPermanent(string) , ControllerBase.LocalRedirectPreserveMethod(string) ,
ControllerBase.LocalRedirectPermanentPreserveMethod(string) , ControllerBase.RedirectToAction() ,
ControllerBase.RedirectToAction(string) , ControllerBase.RedirectToAction(string, object) ,
ControllerBase.RedirectToAction(string, string) ,
ControllerBase.RedirectToAction(string, string, object) ,
ControllerBase.RedirectToAction(string, string, string) ,
ControllerBase.RedirectToAction(string, string, object, string) ,
ControllerBase.RedirectToActionPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToActionPermanent(string) ,
ControllerBase.RedirectToActionPermanent(string, object) ,
ControllerBase.RedirectToActionPermanent(string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, object) ,
ControllerBase.RedirectToActionPermanent(string, string, object, string) ,
ControllerBase.RedirectToActionPermanentPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToRoute(string) , ControllerBase.RedirectToRoute(object) ,
ControllerBase.RedirectToRoute(string, object) , ControllerBase.RedirectToRoute(string, string) ,

ControllerBase.RedirectToRoute(string, object, string) ,
ControllerBase.RedirectToRoutePreserveMethod(string, object, string) ,
ControllerBase.RedirectToRoutePermanent(string) ,
ControllerBase.RedirectToRoutePermanent(object) ,
ControllerBase.RedirectToRoutePermanent(string, object) ,
ControllerBase.RedirectToRoutePermanent(string, string) ,
ControllerBase.RedirectToRoutePermanent(string, object, string) ,
ControllerBase.RedirectToRoutePermanentPreserveMethod(string, object, string) ,
ControllerBase.RedirectToPage(string) , ControllerBase.RedirectToPage(string, object) ,
ControllerBase.RedirectToPage(string, string) , ControllerBase.RedirectToPage(string, string, object) ,
ControllerBase.RedirectToPage(string, string, string) ,
ControllerBase.RedirectToPage(string, string, object, string) ,
ControllerBase.RedirectToPagePermanent(string) ,
ControllerBase.RedirectToPagePermanent(string, object) ,
ControllerBase.RedirectToPagePermanent(string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, object, string) ,
ControllerBase.RedirectToPagePreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToPagePermanentPreserveMethod(string, string, object, string) ,
ControllerBase.File(byte[], string) , ControllerBase.File(byte[], string, bool) ,
ControllerBase.File(byte[], string, string) , ControllerBase.File(byte[], string, string, bool) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string) , ControllerBase.File(Stream, string, bool) ,
ControllerBase.File(Stream, string, string) , ControllerBase.File(Stream, string, string, bool) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string) , ControllerBase.File(string, string, bool) ,
ControllerBase.File(string, string, string) , ControllerBase.File(string, string, string, bool) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string) , ControllerBase.PhysicalFile(string, string, bool) ,
ControllerBase.PhysicalFile(string, string, string) ,
ControllerBase.PhysicalFile(string, string, string, bool) ,

ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.Unauthorized() , ControllerBase.Unauthorized(object) , ControllerBase.NotFound() ,
ControllerBase.NotFound(object) , ControllerBase.BadRequest() ,
ControllerBase.BadRequest(object) , ControllerBase.BadRequest(ModelStateDictionary) ,
ControllerBase.UnprocessableEntity() , ControllerBase.UnprocessableEntity(object) ,
ControllerBase.UnprocessableEntity(ModelStateDictionary) , ControllerBase.Conflict() ,
ControllerBase.Conflict(object) , ControllerBase.Conflict(ModelStateDictionary) ,
ControllerBase.Problem(string, string, int?, string, string) ,
ControllerBase.ValidationProblem(ValidationProblemDetails) ,
ControllerBase.ValidationProblem(ModelStateDictionary) , ControllerBase.ValidationProblem() ,
ControllerBase.ValidationProblem(string, string, int?, string, string, ModelStateDictionary) ,
ControllerBase.Created() , ControllerBase.Created(string, object) ,
ControllerBase.Created(Uri, object) , ControllerBase.CreatedAtAction(string, object) ,
ControllerBase.CreatedAtAction(string, object, object) ,
ControllerBase.CreatedAtAction(string, string, object, object) ,
ControllerBase.CreatedAtRoute(string, object) , ControllerBase.CreatedAtRoute(object, object) ,
ControllerBase.CreatedAtRoute(string, object, object) , ControllerBase.Accepted() ,
ControllerBase.Accepted(object) , ControllerBase.Accepted(Uri) , ControllerBase.Accepted(string) ,
ControllerBase.Accepted(string, object) , ControllerBase.Accepted(Uri, object) ,
ControllerBase.AcceptedAtAction(string) , ControllerBase.AcceptedAtAction(string, string) ,
ControllerBase.AcceptedAtAction(string, object) ,
ControllerBase.AcceptedAtAction(string, string, object) ,
ControllerBase.AcceptedAtAction(string, object, object) ,
ControllerBase.AcceptedAtAction(string, string, object, object) ,
ControllerBase.AcceptedAtRoute(object) , ControllerBase.AcceptedAtRoute(string) ,
ControllerBase.AcceptedAtRoute(string, object) , ControllerBase.AcceptedAtRoute(object, object) ,
ControllerBase.AcceptedAtRoute(string, object, object) , ControllerBase.Challenge() ,
ControllerBase.Challenge(params string[]) , ControllerBase.Challenge(AuthenticationProperties) ,
ControllerBase.Challenge(AuthenticationProperties, params string[]) , ControllerBase.Forbid() ,
ControllerBase.Forbid(params string[]) , ControllerBase.Forbid(AuthenticationProperties) ,
ControllerBase.Forbid(AuthenticationProperties, params string[]) ,
ControllerBase.SignIn(ClaimsPrincipal) , ControllerBase.SignIn(ClaimsPrincipal, string) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties, string) , ControllerBase.SignOut() ,
ControllerBase.SignOut(AuthenticationProperties) , ControllerBase.SignOut(params string[]) ,
ControllerBase.SignOut(AuthenticationProperties, params string[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel) ,

ControllerBase.TryUpdateModelAsync<TModel>(TModel, string) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync(object, Type, string) ,
ControllerBase.TryUpdateModelAsync(object, Type, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryValidateModel(object) , ControllerBase.TryValidateModel(object, string) ,
ControllerBase.HttpContext , ControllerBase.Request , ControllerBase.Response ,
ControllerBase.RouteData , ControllerBase.ModelState , ControllerBase.ControllerContext ,
ControllerBase.MetadataProvider , ControllerBase.ModelBinderFactory , ControllerBase.Url ,
ControllerBase.ObjectValidator , ControllerBase.ProblemDetailsFactory , ControllerBase.User ,
ControllerBase.Empty , object.Equals(object) , object.Equals(object, object) ,
object.GetHashCode() , object.GetType() , object.MemberwiseClone() ,
object.ReferenceEquals(object, object) , object.ToString()

Class RolloverEnvelopeController

Namespace: MakesCentsBackend.Controllers

Assembly: MakesCentsBackend.dll

API controller for rollover envelopes. Create/Update/Delete act on the persistent definition; Get is month-scoped, since a rollover envelope's planned/remaining amounts are specific to one budget month.

```
[Route("api/rollover-envelopes")]
[ApiController]
public class RolloverEnvelopeController : ControllerBase
```

Inheritance

object ← ControllerBase ← RolloverEnvelopeController

Inherited Members

ControllerBase.StatusCode(int) , ControllerBase.StatusCode(int, object) ,
ControllerBase.Content(string) , ControllerBase.Content(string, string) ,
ControllerBase.Content(string, string, Encoding) ,
ControllerBase.Content(string, MediaTypeHeaderValue) , ControllerBase.NoContent() ,
ControllerBase.Ok() , ControllerBase.Ok(object) , ControllerBase.Redirect(string) ,
ControllerBase.RedirectPermanent(string) , ControllerBase.RedirectPreserveMethod(string) ,
ControllerBase.RedirectPermanentPreserveMethod(string) , ControllerBase.LocalRedirect(string) ,
ControllerBase.LocalRedirectPermanent(string) , ControllerBase.LocalRedirectPreserveMethod(string) ,
ControllerBase.LocalRedirectPermanentPreserveMethod(string) , ControllerBase.RedirectToAction() ,
ControllerBase.RedirectToAction(string) , ControllerBase.RedirectToAction(string, object) ,
ControllerBase.RedirectToAction(string, string) ,
ControllerBase.RedirectToAction(string, string, object) ,
ControllerBase.RedirectToAction(string, string, string) ,
ControllerBase.RedirectToAction(string, string, object, string) ,
ControllerBase.RedirectToActionPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToActionPermanent(string) ,
ControllerBase.RedirectToActionPermanent(string, object) ,
ControllerBase.RedirectToActionPermanent(string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, object) ,
ControllerBase.RedirectToActionPermanent(string, string, object, string) ,
ControllerBase.RedirectToActionPermanentPreserveMethod(string, string, object, string) ,

ControllerBase.RedirectToRoute(string) , ControllerBase.RedirectToRoute(object) ,
ControllerBase.RedirectToRoute(string, object) , ControllerBase.RedirectToRoute(string, string) ,
ControllerBase.RedirectToRoute(string, object, string) ,
ControllerBase.RedirectToRoutePreserveMethod(string, object, string) ,
ControllerBase.RedirectToRoutePermanent(string) ,
ControllerBase.RedirectToRoutePermanent(object) ,
ControllerBase.RedirectToRoutePermanent(string, object) ,
ControllerBase.RedirectToRoutePermanent(string, string) ,
ControllerBase.RedirectToRoutePermanent(string, object, string) ,
ControllerBase.RedirectToRoutePermanentPreserveMethod(string, object, string) ,
ControllerBase.RedirectToPage(string) , ControllerBase.RedirectToPage(string, object) ,
ControllerBase.RedirectToPage(string, string) , ControllerBase.RedirectToPage(string, string, object) ,
ControllerBase.RedirectToPage(string, string, string) ,
ControllerBase.RedirectToPage(string, string, object, string) ,
ControllerBase.RedirectToPagePermanent(string) ,
ControllerBase.RedirectToPagePermanent(string, object) ,
ControllerBase.RedirectToPagePermanent(string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, object, string) ,
ControllerBase.RedirectToPagePreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToPagePermanentPreserveMethod(string, string, object, string) ,
ControllerBase.File(byte[], string) , ControllerBase.File(byte[], string, bool) ,
ControllerBase.File(byte[], string, string) , ControllerBase.File(byte[], string, string, bool) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string) , ControllerBase.File(Stream, string, bool) ,
ControllerBase.File(Stream, string, string) , ControllerBase.File(Stream, string, string, bool) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string) , ControllerBase.File(string, string, bool) ,
ControllerBase.File(string, string, string) , ControllerBase.File(string, string, string, bool) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string) , ControllerBase.PhysicalFile(string, string, bool) ,

ControllerBase.PhysicalFile(string, string, string) ,
ControllerBase.PhysicalFile(string, string, string, bool) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.Unauthorized() , ControllerBase.Unauthorized(object) , ControllerBase.NotFound() ,
ControllerBase.NotFound(object) , ControllerBase.BadRequest() ,
ControllerBase.BadRequest(object) , ControllerBase.BadRequest(ModelStateDictionary) ,
ControllerBase.UnprocessableEntity() , ControllerBase.UnprocessableEntity(object) ,
ControllerBase.UnprocessableEntity(ModelStateDictionary) , ControllerBase.Conflict() ,
ControllerBase.Conflict(object) , ControllerBase.Conflict(ModelStateDictionary) ,
ControllerBase.Problem(string, string, int?, string, string) ,
ControllerBase.ValidationProblem(ValidationProblemDetails) ,
ControllerBase.ValidationProblem(ModelStateDictionary) , ControllerBase.ValidationProblem() ,
ControllerBase.ValidationProblem(string, string, int?, string, string, ModelStateDictionary) ,
ControllerBase.Created() , ControllerBase.Created(string, object) ,
ControllerBase.Created(Uri, object) , ControllerBase.CreatedAtAction(string, object) ,
ControllerBase.CreatedAtAction(string, object, object) ,
ControllerBase.CreatedAtAction(string, string, object, object) ,
ControllerBase.CreatedAtRoute(string, object) , ControllerBase.CreatedAtRoute(object, object) ,
ControllerBase.CreatedAtRoute(string, object, object) , ControllerBase.Accepted() ,
ControllerBase.Accepted(object) , ControllerBase.Accepted(Uri) , ControllerBase.Accepted(string) ,
ControllerBase.Accepted(string, object) , ControllerBase.Accepted(Uri, object) ,
ControllerBase.AcceptedAtAction(string) , ControllerBase.AcceptedAtAction(string, string) ,
ControllerBase.AcceptedAtAction(string, object) ,
ControllerBase.AcceptedAtAction(string, string, object) ,
ControllerBase.AcceptedAtAction(string, object, object) ,
ControllerBase.AcceptedAtAction(string, string, object, object) ,
ControllerBase.AcceptedAtRoute(object) , ControllerBase.AcceptedAtRoute(string) ,
ControllerBase.AcceptedAtRoute(string, object) , ControllerBase.AcceptedAtRoute(object, object) ,
ControllerBase.AcceptedAtRoute(string, object, object) , ControllerBase.Challenge() ,
ControllerBase.Challenge(params string[]) , ControllerBase.Challenge(AuthenticationProperties) ,
ControllerBase.Challenge(AuthenticationProperties, params string[]) , ControllerBase.Forbid() ,
ControllerBase.Forbid(params string[]) , ControllerBase.Forbid(AuthenticationProperties) ,
ControllerBase.Forbid(AuthenticationProperties, params string[]) ,
ControllerBase.SignIn(ClaimsPrincipal) , ControllerBase.SignIn(ClaimsPrincipal, string) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties, string) , ControllerBase.SignOut() ,
ControllerBase.SignOut(AuthenticationProperties) , ControllerBase.SignOut(params string[]) ,

ControllerBase.SignOut(AuthenticationProperties, params string[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync(object, Type, string) ,
ControllerBase.TryUpdateModelAsync(object, Type, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryValidateModel(object) , ControllerBase.TryValidateModel(object, string) ,
ControllerBase.HttpContext , ControllerBase.Request , ControllerBase.Response ,
ControllerBase.RouteData , ControllerBase.ModelState , ControllerBase.ControllerContext ,
ControllerBase.MetadataProvider , ControllerBase.ModelBinderFactory , ControllerBase.Url ,
ControllerBase.ObjectValidator , ControllerBase.ProblemDetailsFactory , ControllerBase.User ,
ControllerBase.Empty , object.Equals(object) , object.Equals(object, object) ,
object.GetHashCode() , object.GetType() , object.MemberwiseClone() ,
object.ReferenceEquals(object, object) , object.ToString()

Constructors

RolloverEnvelopeController(RolloverEnvelopeLogic)

Parameterized constructor to bring in DI variables

```
public RolloverEnvelopeController(RolloverEnvelopeLogic rolloverEnvelopeLogic)
```

Parameters

`rolloverEnvelopeLogic` RolloverEnvelopeLogic

Methods

CreateRolloverEnvelopeAsync(CreateRolloverEnvelopeRequest)

POST method to create a new rollover envelope's persistent definition

```
[Authorize]
[HttpPost]
public Task<ActionResult> CreateRolloverEnvelopeAsync(CreateRolloverEnvelopeRequest
envelope)
```

Parameters

`envelope` CreateRolloverEnvelopeRequest

Returns

Task <ActionResult>

DeleteRolloverEnvelopeAsync(int)

DELETE method to delete a rollover envelope's persistent definition (cascades every month it was ever activated for)

```
[Authorize]
[HttpDelete("{rolloverEnvelopeId}")]
public Task<ActionResult> DeleteRolloverEnvelopeAsync(int rolloverEnvelopeId)
```

Parameters

`rolloverEnvelopeId` int

Returns

Task <ActionResult>

GetRolloverEnvelopeForMonthAsync(int, int)

GET method to get a rollover envelope for a specific budget month, including its transaction history for that month

```
[Authorize]
[HttpGet("{rolloverEnvelopeId}/budget-month/{budgetMonthId}")]
public Task<ActionResult> GetRolloverEnvelopeForMonthAsync(int rolloverEnvelopeId,
int budgetMonthId)
```

Parameters

`rolloverEnvelopeId` int

`budgetMonthId` int

Returns

Task <ActionResult>

UpdateRolloverEnvelopeAsync(int, UpdateRolloverEnvelopeRequest)

PUT method to update a rollover envelope's persistent definition fields

```
[Authorize]
[HttpPut("{rolloverEnvelopeId}")]
public Task<ActionResult> UpdateRolloverEnvelopeAsync(int rolloverEnvelopeId,
UpdateRolloverEnvelopeRequest envelope)
```

Parameters

`rolloverEnvelopeId` int

`envelope` UpdateRolloverEnvelopeRequest

Returns

Task <ActionResult>

Class SinkingFundEnvelopeController

Namespace: MakesCentsBackend.Controllers

Assembly: MakesCentsBackend.dll

API controller for sinking fund envelopes

```
[Route("api/sinking-fund-envelopes")]
[ApiController]
public class SinkingFundEnvelopeController : ControllerBase
```

Inheritance

object ← ControllerBase ← SinkingFundEnvelopeController

Inherited Members

ControllerBase.StatusCode(int) , ControllerBase.StatusCode(int, object) ,
ControllerBase.Content(string) , ControllerBase.Content(string, string) ,
ControllerBase.Content(string, string, Encoding) ,
ControllerBase.Content(string, MediaTypeHeaderValue) , ControllerBase.NoContent() ,
ControllerBase.Ok() , ControllerBase.Ok(object) , ControllerBase.Redirect(string) ,
ControllerBase.RedirectPermanent(string) , ControllerBase.RedirectPreserveMethod(string) ,
ControllerBase.RedirectPermanentPreserveMethod(string) , ControllerBase.LocalRedirect(string) ,
ControllerBase.LocalRedirectPermanent(string) , ControllerBase.LocalRedirectPreserveMethod(string) ,
ControllerBase.LocalRedirectPermanentPreserveMethod(string) , ControllerBase.RedirectToAction() ,
ControllerBase.RedirectToAction(string) , ControllerBase.RedirectToAction(string, object) ,
ControllerBase.RedirectToAction(string, string) ,
ControllerBase.RedirectToAction(string, string, object) ,
ControllerBase.RedirectToAction(string, string, string) ,
ControllerBase.RedirectToAction(string, string, object, string) ,
ControllerBase.RedirectToActionPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToActionPermanent(string) ,
ControllerBase.RedirectToActionPermanent(string, object) ,
ControllerBase.RedirectToActionPermanent(string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, object) ,
ControllerBase.RedirectToActionPermanent(string, string, object, string) ,
ControllerBase.RedirectToActionPermanentPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToRoute(string) , ControllerBase.RedirectToRoute(object) ,
ControllerBase.RedirectToRoute(string, object) , ControllerBase.RedirectToRoute(string, string) ,

ControllerBase.RedirectToRoute(string, object, string) ,
ControllerBase.RedirectToRoutePreserveMethod(string, object, string) ,
ControllerBase.RedirectToRoutePermanent(string) ,
ControllerBase.RedirectToRoutePermanent(object) ,
ControllerBase.RedirectToRoutePermanent(string, object) ,
ControllerBase.RedirectToRoutePermanent(string, string) ,
ControllerBase.RedirectToRoutePermanent(string, object, string) ,
ControllerBase.RedirectToRoutePermanentPreserveMethod(string, object, string) ,
ControllerBase.RedirectToPage(string) , ControllerBase.RedirectToPage(string, object) ,
ControllerBase.RedirectToPage(string, string) , ControllerBase.RedirectToPage(string, string, object) ,
ControllerBase.RedirectToPage(string, string, string) ,
ControllerBase.RedirectToPage(string, string, object, string) ,
ControllerBase.RedirectToPagePermanent(string) ,
ControllerBase.RedirectToPagePermanent(string, object) ,
ControllerBase.RedirectToPagePermanent(string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, object, string) ,
ControllerBase.RedirectToPagePreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToPagePermanentPreserveMethod(string, string, object, string) ,
ControllerBase.File(byte[], string) , ControllerBase.File(byte[], string, bool) ,
ControllerBase.File(byte[], string, string) , ControllerBase.File(byte[], string, string, bool) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string) , ControllerBase.File(Stream, string, bool) ,
ControllerBase.File(Stream, string, string) , ControllerBase.File(Stream, string, string, bool) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string) , ControllerBase.File(string, string, bool) ,
ControllerBase.File(string, string, string) , ControllerBase.File(string, string, string, bool) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string) , ControllerBase.PhysicalFile(string, string, bool) ,
ControllerBase.PhysicalFile(string, string, string) ,
ControllerBase.PhysicalFile(string, string, string, bool) ,

ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.Unauthorized() , ControllerBase.Unauthorized(object) , ControllerBase.NotFound() ,
ControllerBase.NotFound(object) , ControllerBase.BadRequest() ,
ControllerBase.BadRequest(object) , ControllerBase.BadRequest(ModelStateDictionary) ,
ControllerBase.UnprocessableEntity() , ControllerBase.UnprocessableEntity(object) ,
ControllerBase.UnprocessableEntity(ModelStateDictionary) , ControllerBase.Conflict() ,
ControllerBase.Conflict(object) , ControllerBase.Conflict(ModelStateDictionary) ,
ControllerBase.Problem(string, string, int?, string, string) ,
ControllerBase.ValidationProblem(ValidationProblemDetails) ,
ControllerBase.ValidationProblem(ModelStateDictionary) , ControllerBase.ValidationProblem() ,
ControllerBase.ValidationProblem(string, string, int?, string, string, ModelStateDictionary) ,
ControllerBase.Created() , ControllerBase.Created(string, object) ,
ControllerBase.Created(Uri, object) , ControllerBase.CreatedAtAction(string, object) ,
ControllerBase.CreatedAtAction(string, object, object) ,
ControllerBase.CreatedAtAction(string, string, object, object) ,
ControllerBase.CreatedAtRoute(string, object) , ControllerBase.CreatedAtRoute(object, object) ,
ControllerBase.CreatedAtRoute(string, object, object) , ControllerBase.Accepted() ,
ControllerBase.Accepted(object) , ControllerBase.Accepted(Uri) , ControllerBase.Accepted(string) ,
ControllerBase.Accepted(string, object) , ControllerBase.Accepted(Uri, object) ,
ControllerBase.AcceptedAtAction(string) , ControllerBase.AcceptedAtAction(string, string) ,
ControllerBase.AcceptedAtAction(string, object) ,
ControllerBase.AcceptedAtAction(string, string, object) ,
ControllerBase.AcceptedAtAction(string, object, object) ,
ControllerBase.AcceptedAtAction(string, string, object, object) ,
ControllerBase.AcceptedAtRoute(object) , ControllerBase.AcceptedAtRoute(string) ,
ControllerBase.AcceptedAtRoute(string, object) , ControllerBase.AcceptedAtRoute(object, object) ,
ControllerBase.AcceptedAtRoute(string, object, object) , ControllerBase.Challenge() ,
ControllerBase.Challenge(params string[]) , ControllerBase.Challenge(AuthenticationProperties) ,
ControllerBase.Challenge(AuthenticationProperties, params string[]) , ControllerBase.Forbid() ,
ControllerBase.Forbid(params string[]) , ControllerBase.Forbid(AuthenticationProperties) ,
ControllerBase.Forbid(AuthenticationProperties, params string[]) ,
ControllerBase.SignIn(ClaimsPrincipal) , ControllerBase.SignIn(ClaimsPrincipal, string) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties, string) , ControllerBase.SignOut() ,
ControllerBase.SignOut(AuthenticationProperties) , ControllerBase.SignOut(params string[]) ,
ControllerBase.SignOut(AuthenticationProperties, params string[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel) ,

ControllerBase.TryUpdateModelAsync<TModel>(TModel, string) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync(object, Type, string) ,
ControllerBase.TryUpdateModelAsync(object, Type, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryValidateModel(object) , ControllerBase.TryValidateModel(object, string) ,
ControllerBase.HttpContext , ControllerBase.Request , ControllerBase.Response ,
ControllerBase.RouteData , ControllerBase.ModelState , ControllerBase.ControllerContext ,
ControllerBase.MetadataProvider , ControllerBase.ModelBinderFactory , ControllerBase.Url ,
ControllerBase.ObjectValidator , ControllerBase.ProblemDetailsFactory , ControllerBase.User ,
ControllerBase.Empty , object.Equals(object) , object.Equals(object, object) ,
object.GetHashCode() , object.GetType() , object.MemberwiseClone() ,
object.ReferenceEquals(object, object) , object.ToString()

Constructors

SinkingFundEnvelopeController(SinkingFundEnvelopeLogic)

Parameterized constructor to bring in DI variables

```
public SinkingFundEnvelopeController(SinkingFundEnvelopeLogic sinkingFundEnvelopeLogic)
```

Parameters

`sinkingFundEnvelopeLogic` SinkingFundEnvelopeLogic

Methods

CreateSinkingFundEnvelopeAsync(CreateSinkingFundEnvelopeRequest)

POST method to create a new sinking fund envelope

```
[Authorize]
[HttpPost]
public Task<ActionResult> CreateSinkingFundEnvelopeAsync(CreateSinkingFundEnvelopeRequest
envelope)
```

Parameters

`envelope` CreateSinkingFundEnvelopeRequest

Returns

Task <ActionResult>

DeleteSinkingFundEnvelopeAsync(int)

DELETE method to delete a sinking fund envelope

```
[Authorize]
[HttpDelete("{sinkingFundEnvelopeId}")]
public Task<ActionResult> DeleteSinkingFundEnvelopeAsync(int sinkingFundEnvelopeId)
```

Parameters

`sinkingFundEnvelopeId` int

Returns

Task <ActionResult>

GetRolloverDependentsAsync(int)

GET method to get the rollover envelopes that sweep into this sinking fund envelope

```
[Authorize]
[HttpGet("{sinkingFundEnvelopeId}/rollover-dependents")]
public Task<ActionResult> GetRolloverDependentsAsync(int sinkingFundEnvelopeId)
```

Parameters

`sinkingFundEnvelopeId` int

Returns

Task<ActionResult>

GetSinkingFundEnvelopeAsync(int)

GET method to get a sinking fund envelope, including its transaction history

```
[Authorize]
[HttpGet("{sinkingFundEnvelopeId}")]
public Task<ActionResult> GetSinkingFundEnvelopeAsync(int sinkingFundEnvelopeId)
```

Parameters

`sinkingFundEnvelopeId` int

Returns

Task<ActionResult>

UpdateSinkingFundEnvelopeAsync(int, UpdateSinkingFundEnvelopeRequest)

PUT method to update a sinking fund envelope based on given fields

```
[Authorize]
[HttpPut("{sinkingFundEnvelopeId}")]
public Task<ActionResult> UpdateSinkingFundEnvelopeAsync(int sinkingFundEnvelopeId,
UpdateSinkingFundEnvelopeRequest envelope)
```

Parameters

`sinkingFundEnvelopeId` int

`envelope` UpdateSinkingFundEnvelopeRequest

Returns

Task <ActionResult>

Class TransactionController

Namespace: MakesCentsBackend.Controllers

Assembly: MakesCentsBackend.dll

API controller for transactions

```
[Route("api/transactions")]
[ApiController]
public class TransactionController : ControllerBase
```

Inheritance

object ← ControllerBase ← TransactionController

Inherited Members

ControllerBase.StatusCode(int) , ControllerBase.StatusCode(int, object) ,
ControllerBase.Content(string) , ControllerBase.Content(string, string) ,
ControllerBase.Content(string, string, Encoding) ,
ControllerBase.Content(string, MediaTypeHeaderValue) , ControllerBase.NoContent() ,
ControllerBase.Ok() , ControllerBase.Ok(object) , ControllerBase.Redirect(string) ,
ControllerBase.RedirectPermanent(string) , ControllerBase.RedirectPreserveMethod(string) ,
ControllerBase.RedirectPermanentPreserveMethod(string) , ControllerBase.LocalRedirect(string) ,
ControllerBase.LocalRedirectPermanent(string) , ControllerBase.LocalRedirectPreserveMethod(string) ,
ControllerBase.LocalRedirectPermanentPreserveMethod(string) , ControllerBase.RedirectToAction() ,
ControllerBase.RedirectToAction(string) , ControllerBase.RedirectToAction(string, object) ,
ControllerBase.RedirectToAction(string, string) ,
ControllerBase.RedirectToAction(string, string, object) ,
ControllerBase.RedirectToAction(string, string, string) ,
ControllerBase.RedirectToAction(string, string, object, string) ,
ControllerBase.RedirectToActionPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToActionPermanent(string) ,
ControllerBase.RedirectToActionPermanent(string, object) ,
ControllerBase.RedirectToActionPermanent(string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, object) ,
ControllerBase.RedirectToActionPermanent(string, string, object, string) ,
ControllerBase.RedirectToActionPermanentPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToRoute(string) , ControllerBase.RedirectToRoute(object) ,
ControllerBase.RedirectToRoute(string, object) , ControllerBase.RedirectToRoute(string, string) ,

ControllerBase.RedirectToRoute(string, object, string) ,
ControllerBase.RedirectToRoutePreserveMethod(string, object, string) ,
ControllerBase.RedirectToRoutePermanent(string) ,
ControllerBase.RedirectToRoutePermanent(object) ,
ControllerBase.RedirectToRoutePermanent(string, object) ,
ControllerBase.RedirectToRoutePermanent(string, string) ,
ControllerBase.RedirectToRoutePermanent(string, object, string) ,
ControllerBase.RedirectToRoutePermanentPreserveMethod(string, object, string) ,
ControllerBase.RedirectToPage(string) , ControllerBase.RedirectToPage(string, object) ,
ControllerBase.RedirectToPage(string, string) , ControllerBase.RedirectToPage(string, string, object) ,
ControllerBase.RedirectToPage(string, string, string) ,
ControllerBase.RedirectToPage(string, string, object, string) ,
ControllerBase.RedirectToPagePermanent(string) ,
ControllerBase.RedirectToPagePermanent(string, object) ,
ControllerBase.RedirectToPagePermanent(string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, object, string) ,
ControllerBase.RedirectToPagePreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToPagePermanentPreserveMethod(string, string, object, string) ,
ControllerBase.File(byte[], string) , ControllerBase.File(byte[], string, bool) ,
ControllerBase.File(byte[], string, string) , ControllerBase.File(byte[], string, string, bool) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string) , ControllerBase.File(Stream, string, bool) ,
ControllerBase.File(Stream, string, string) , ControllerBase.File(Stream, string, string, bool) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string) , ControllerBase.File(string, string, bool) ,
ControllerBase.File(string, string, string) , ControllerBase.File(string, string, string, bool) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string) , ControllerBase.PhysicalFile(string, string, bool) ,
ControllerBase.PhysicalFile(string, string, string) ,
ControllerBase.PhysicalFile(string, string, string, bool) ,

ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.Unauthorized() , ControllerBase.Unauthorized(object) , ControllerBase.NotFound() ,
ControllerBase.NotFound(object) , ControllerBase.BadRequest() ,
ControllerBase.BadRequest(object) , ControllerBase.BadRequest(ModelStateDictionary) ,
ControllerBase.UnprocessableEntity() , ControllerBase.UnprocessableEntity(object) ,
ControllerBase.UnprocessableEntity(ModelStateDictionary) , ControllerBase.Conflict() ,
ControllerBase.Conflict(object) , ControllerBase.Conflict(ModelStateDictionary) ,
ControllerBase.Problem(string, string, int?, string, string) ,
ControllerBase.ValidationProblem(ValidationProblemDetails) ,
ControllerBase.ValidationProblem(ModelStateDictionary) , ControllerBase.ValidationProblem() ,
ControllerBase.ValidationProblem(string, string, int?, string, string, ModelStateDictionary) ,
ControllerBase.Created() , ControllerBase.Created(string, object) ,
ControllerBase.Created(Uri, object) , ControllerBase.CreatedAtAction(string, object) ,
ControllerBase.CreatedAtAction(string, object, object) ,
ControllerBase.CreatedAtAction(string, string, object, object) ,
ControllerBase.CreatedAtRoute(string, object) , ControllerBase.CreatedAtRoute(object, object) ,
ControllerBase.CreatedAtRoute(string, object, object) , ControllerBase.Accepted() ,
ControllerBase.Accepted(object) , ControllerBase.Accepted(Uri) , ControllerBase.Accepted(string) ,
ControllerBase.Accepted(string, object) , ControllerBase.Accepted(Uri, object) ,
ControllerBase.AcceptedAtAction(string) , ControllerBase.AcceptedAtAction(string, string) ,
ControllerBase.AcceptedAtAction(string, object) ,
ControllerBase.AcceptedAtAction(string, string, object) ,
ControllerBase.AcceptedAtAction(string, object, object) ,
ControllerBase.AcceptedAtAction(string, string, object, object) ,
ControllerBase.AcceptedAtRoute(object) , ControllerBase.AcceptedAtRoute(string) ,
ControllerBase.AcceptedAtRoute(string, object) , ControllerBase.AcceptedAtRoute(object, object) ,
ControllerBase.AcceptedAtRoute(string, object, object) , ControllerBase.Challenge() ,
ControllerBase.Challenge(params string[]) , ControllerBase.Challenge(AuthenticationProperties) ,
ControllerBase.Challenge(AuthenticationProperties, params string[]) , ControllerBase.Forbid() ,
ControllerBase.Forbid(params string[]) , ControllerBase.Forbid(AuthenticationProperties) ,
ControllerBase.Forbid(AuthenticationProperties, params string[]) ,
ControllerBase.SignIn(ClaimsPrincipal) , ControllerBase.SignIn(ClaimsPrincipal, string) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties, string) , ControllerBase.SignOut() ,
ControllerBase.SignOut(AuthenticationProperties) , ControllerBase.SignOut(params string[]) ,
ControllerBase.SignOut(AuthenticationProperties, params string[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel) ,

ControllerBase.TryUpdateModelAsync<TModel>(TModel, string) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync(object, Type, string) ,
ControllerBase.TryUpdateModelAsync(object, Type, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryValidateModel(object) , ControllerBase.TryValidateModel(object, string) ,
ControllerBase.HttpContext , ControllerBase.Request , ControllerBase.Response ,
ControllerBase.RouteData , ControllerBase.ModelState , ControllerBase.ControllerContext ,
ControllerBase.MetadataProvider , ControllerBase.ModelBinderFactory , ControllerBase.Url ,
ControllerBase.ObjectValidator , ControllerBase.ProblemDetailsFactory , ControllerBase.User ,
ControllerBase.Empty , object.Equals(object) , object.Equals(object, object) ,
object.GetHashCode() , object.GetType() , object.MemberwiseClone() ,
object.ReferenceEquals(object, object) , object.ToString()

Constructors

TransactionController(TransactionLogic)

Parameterized constructor to bring in DI variables

```
public TransactionController(TransactionLogic transactionLogic)
```

Parameters

`transactionLogic` TransactionLogic

Methods

GetAllTransactionsAsync(int)

HTTP GET method to get every transaction for a budget month. NOTE: the route param is now a budget_month_id, not a budget_id - transactions are scoped to one specific month.

```
[Authorize]
[HttpGet("budget-month/{budgetMonthId}")]
public Task<ActionResult> GetAllTransactionsAsync(int budgetMonthId)
```

Parameters

`budgetMonthId` int

Returns

Task <ActionResult>

HardDeleteTransactionAsync(int)

HTTP DELETE method to permanently delete a transaction

```
[Authorize]
[HttpDelete("{transactionId}/permanent")]
public Task<ActionResult> HardDeleteTransactionAsync(int transactionId)
```

Parameters

`transactionId` int

Returns

Task <ActionResult>

RestoreTransactionAsync(int)

HTTP PATCH method to restore a soft-deleted transaction

```
[Authorize]
[HttpPatch("{transactionId}/restore")]
public Task<ActionResult> RestoreTransactionAsync(int transactionId)
```

Parameters

`transactionId` int

Returns

Task<ActionResult>

SoftDeleteTransactionAsync(int)

HTTP DELETE method to soft delete a transaction

```
[Authorize]
[HttpDelete("{transactionId}")]
public Task<ActionResult> SoftDeleteTransactionAsync(int transactionId)
```

Parameters

`transactionId` int

Returns

Task<ActionResult>

Class TransferTransactionController

Namespace: MakesCentsBackend.Controllers

Assembly: MakesCentsBackend.dll

```
[Route("api/transfer-transactions")]
[ApiController]
public class TransferTransactionController : ControllerBase
```

Inheritance

object ← ControllerBase ← TransferTransactionController

Inherited Members

ControllerBase.StatusCode(int) , ControllerBase.StatusCode(int, object) ,
ControllerBase.Content(string) , ControllerBase.Content(string, string) ,
ControllerBase.Content(string, string, Encoding) ,
ControllerBase.Content(string, MediaTypeHeaderValue) , ControllerBase.NoContent() ,
ControllerBase.Ok() , ControllerBase.Ok(object) , ControllerBase.Redirect(string) ,
ControllerBase.RedirectPermanent(string) , ControllerBase.RedirectPreserveMethod(string) ,
ControllerBase.RedirectPermanentPreserveMethod(string) , ControllerBase.LocalRedirect(string) ,
ControllerBase.LocalRedirectPermanent(string) , ControllerBase.LocalRedirectPreserveMethod(string) ,
ControllerBase.LocalRedirectPermanentPreserveMethod(string) , ControllerBase.RedirectToAction() ,
ControllerBase.RedirectToAction(string) , ControllerBase.RedirectToAction(string, object) ,
ControllerBase.RedirectToAction(string, string) ,
ControllerBase.RedirectToAction(string, string, object) ,
ControllerBase.RedirectToAction(string, string, string) ,
ControllerBase.RedirectToAction(string, string, object, string) ,
ControllerBase.RedirectToActionPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToActionPermanent(string) ,
ControllerBase.RedirectToActionPermanent(string, object) ,
ControllerBase.RedirectToActionPermanent(string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, object) ,
ControllerBase.RedirectToActionPermanent(string, string, object, string) ,
ControllerBase.RedirectToActionPermanentPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToRoute(string) , ControllerBase.RedirectToRoute(object) ,
ControllerBase.RedirectToRoute(string, object) , ControllerBase.RedirectToRoute(string, string) ,
ControllerBase.RedirectToRoute(string, object, string) ,
ControllerBase.RedirectToRoutePreserveMethod(string, object, string) ,

ControllerBase.RedirectToRoutePermanent(string) ,
ControllerBase.RedirectToRoutePermanent(object) ,
ControllerBase.RedirectToRoutePermanent(string, object) ,
ControllerBase.RedirectToRoutePermanent(string, string) ,
ControllerBase.RedirectToRoutePermanent(string, object, string) ,
ControllerBase.RedirectToRoutePermanentPreserveMethod(string, object, string) ,
ControllerBase.RedirectToPage(string) , ControllerBase.RedirectToPage(string, object) ,
ControllerBase.RedirectToPage(string, string) , ControllerBase.RedirectToPage(string, string, object) ,
ControllerBase.RedirectToPage(string, string, string) ,
ControllerBase.RedirectToPage(string, string, object, string) ,
ControllerBase.RedirectToPagePermanent(string) ,
ControllerBase.RedirectToPagePermanent(string, object) ,
ControllerBase.RedirectToPagePermanent(string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, object, string) ,
ControllerBase.RedirectToPagePreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToPagePermanentPreserveMethod(string, string, object, string) ,
ControllerBase.File(byte[], string) , ControllerBase.File(byte[], string, bool) ,
ControllerBase.File(byte[], string, string) , ControllerBase.File(byte[], string, string, bool) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string) , ControllerBase.File(Stream, string, bool) ,
ControllerBase.File(Stream, string, string) , ControllerBase.File(Stream, string, string, bool) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string) , ControllerBase.File(string, string, bool) ,
ControllerBase.File(string, string, string) , ControllerBase.File(string, string, string, bool) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string) , ControllerBase.PhysicalFile(string, string, bool) ,
ControllerBase.PhysicalFile(string, string, string) ,
ControllerBase.PhysicalFile(string, string, string, bool) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,

ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.Unauthorized() , ControllerBase.Unauthorized(object) , ControllerBase.NotFound() ,
ControllerBase.NotFound(object) , ControllerBase.BadRequest() ,
ControllerBase.BadRequest(object) , ControllerBase.BadRequest(ModelStateDictionary) ,
ControllerBase.UnprocessableEntity() , ControllerBase.UnprocessableEntity(object) ,
ControllerBase.UnprocessableEntity(ModelStateDictionary) , ControllerBase.Conflict() ,
ControllerBase.Conflict(object) , ControllerBase.Conflict(ModelStateDictionary) ,
ControllerBase.Problem(string, string, int?, string, string) ,
ControllerBase.ValidationProblem(ValidationProblemDetails) ,
ControllerBase.ValidationProblem(ModelStateDictionary) , ControllerBase.ValidationProblem() ,
ControllerBase.ValidationProblem(string, string, int?, string, string, ModelStateDictionary) ,
ControllerBase.Created() , ControllerBase.Created(string, object) ,
ControllerBase.Created(Uri, object) , ControllerBase.CreatedAtAction(string, object) ,
ControllerBase.CreatedAtAction(string, object, object) ,
ControllerBase.CreatedAtAction(string, string, object, object) ,
ControllerBase.CreatedAtRoute(string, object) , ControllerBase.CreatedAtRoute(object, object) ,
ControllerBase.CreatedAtRoute(string, object, object) , ControllerBase.Accepted() ,
ControllerBase.Accepted(object) , ControllerBase.Accepted(Uri) , ControllerBase.Accepted(string) ,
ControllerBase.Accepted(string, object) , ControllerBase.Accepted(Uri, object) ,
ControllerBase.AcceptedAtAction(string) , ControllerBase.AcceptedAtAction(string, string) ,
ControllerBase.AcceptedAtAction(string, object) ,
ControllerBase.AcceptedAtAction(string, string, object) ,
ControllerBase.AcceptedAtAction(string, object, object) ,
ControllerBase.AcceptedAtAction(string, string, object, object) ,
ControllerBase.AcceptedAtRoute(object) , ControllerBase.AcceptedAtRoute(string) ,
ControllerBase.AcceptedAtRoute(string, object) , ControllerBase.AcceptedAtRoute(object, object) ,
ControllerBase.AcceptedAtRoute(string, object, object) , ControllerBase.Challenge() ,
ControllerBase.Challenge(params string[]) , ControllerBase.Challenge(AuthenticationProperties) ,
ControllerBase.Challenge(AuthenticationProperties, params string[]) , ControllerBase.Forbid() ,
ControllerBase.Forbid(params string[]) , ControllerBase.Forbid(AuthenticationProperties) ,
ControllerBase.Forbid(AuthenticationProperties, params string[]) ,
ControllerBase.SignIn(ClaimsPrincipal) , ControllerBase.SignIn(ClaimsPrincipal, string) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties, string) , ControllerBase.SignOut() ,
ControllerBase.SignOut(AuthenticationProperties) , ControllerBase.SignOut(params string[]) ,
ControllerBase.SignOut(AuthenticationProperties, params string[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider) ,

ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync(object, Type, string) ,
ControllerBase.TryUpdateModelAsync(object, Type, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryValidateModel(object) , ControllerBase.TryValidateModel(object, string) ,
ControllerBase.HttpContext , ControllerBase.Request , ControllerBase.Response ,
ControllerBase.RouteData , ControllerBase.ModelState , ControllerBase.ControllerContext ,
ControllerBase.MetadataProvider , ControllerBase.ModelBinderFactory , ControllerBase.Url ,
ControllerBase.ObjectValidator , ControllerBase.ProblemDetailsFactory , ControllerBase.User ,
ControllerBase.Empty , object.Equals(object) , object.Equals(object, object) ,
object.GetHashCode() , object.GetType() , object.MemberwiseClone() ,
object.ReferenceEquals(object, object) , object.ToString()

Constructors

TransferTransactionController(TransferTransactionLogic)

Parameterized constructor to bring in DI variables

```
public TransferTransactionController(TransferTransactionLogic transferTransactionLogic)
```

Parameters

`transferTransactionLogic` TransferTransactionLogic

Methods

CreateTransferTransactionAsync(CreateTransferTransactionRequest)

HTTP POST method to create a new transfer transaction (account-to-account or envelope-to-envelope)

```
[Authorize]
[HttpPost]
public Task<ActionResult> CreateTransferTransactionAsync(CreateTransferTransactionRequest
transferTransaction)
```

Parameters

`transferTransaction` CreateTransferTransactionRequest

Returns

Task<ActionResult>

GetTransferTransactionAsync(int)

HTTP GET method to get a transfer transaction

```
[Authorize]
[HttpGet("{transactionId}")]
public Task<ActionResult> GetTransferTransactionAsync(int transactionId)
```

Parameters

`transactionId` int

Returns

Task<ActionResult>

UpdateTransferTransactionAsync(int, UpdateTransferTransactionRequest)

HTTP PUT method to update a transfer transaction

```
[Authorize]
[HttpPut("{transferTransactionId}")]
```

```
public Task<ActionResult> UpdateTransferTransactionAsync(int transferTransactionId,
UpdateTransferTransactionRequest transferTransaction)
```

Parameters

`transferTransactionId` int

`transferTransaction` UpdateTransferTransactionRequest

Returns

Task <ActionResult>

Class UserController

Namespace: MakesCentsBackend.Controllers

Assembly: MakesCentsBackend.dll

API controller for users

```
[Route("api/user")]
[ApiController]
public class UserController : ControllerBase
```

Inheritance

object ← ControllerBase ← UserController

Inherited Members

ControllerBase.StatusCode(int) , ControllerBase.StatusCode(int, object) ,
ControllerBase.Content(string) , ControllerBase.Content(string, string) ,
ControllerBase.Content(string, string, Encoding) ,
ControllerBase.Content(string, MediaTypeHeaderValue) , ControllerBase.NoContent() ,
ControllerBase.Ok() , ControllerBase.Ok(object) , ControllerBase.Redirect(string) ,
ControllerBase.RedirectPermanent(string) , ControllerBase.RedirectPreserveMethod(string) ,
ControllerBase.RedirectPermanentPreserveMethod(string) , ControllerBase.LocalRedirect(string) ,
ControllerBase.LocalRedirectPermanent(string) , ControllerBase.LocalRedirectPreserveMethod(string) ,
ControllerBase.LocalRedirectPermanentPreserveMethod(string) , ControllerBase.RedirectToAction() ,
ControllerBase.RedirectToAction(string) , ControllerBase.RedirectToAction(string, object) ,
ControllerBase.RedirectToAction(string, string) ,
ControllerBase.RedirectToAction(string, string, object) ,
ControllerBase.RedirectToAction(string, string, string) ,
ControllerBase.RedirectToAction(string, string, object, string) ,
ControllerBase.RedirectToActionPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToActionPermanent(string) ,
ControllerBase.RedirectToActionPermanent(string, object) ,
ControllerBase.RedirectToActionPermanent(string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, string) ,
ControllerBase.RedirectToActionPermanent(string, string, object) ,
ControllerBase.RedirectToActionPermanent(string, string, object, string) ,
ControllerBase.RedirectToActionPermanentPreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToRoute(string) , ControllerBase.RedirectToRoute(object) ,
ControllerBase.RedirectToRoute(string, object) , ControllerBase.RedirectToRoute(string, string) ,

ControllerBase.RedirectToRoute(string, object, string) ,
ControllerBase.RedirectToRoutePreserveMethod(string, object, string) ,
ControllerBase.RedirectToRoutePermanent(string) ,
ControllerBase.RedirectToRoutePermanent(object) ,
ControllerBase.RedirectToRoutePermanent(string, object) ,
ControllerBase.RedirectToRoutePermanent(string, string) ,
ControllerBase.RedirectToRoutePermanent(string, object, string) ,
ControllerBase.RedirectToRoutePermanentPreserveMethod(string, object, string) ,
ControllerBase.RedirectToPage(string) , ControllerBase.RedirectToPage(string, object) ,
ControllerBase.RedirectToPage(string, string) , ControllerBase.RedirectToPage(string, string, object) ,
ControllerBase.RedirectToPage(string, string, string) ,
ControllerBase.RedirectToPage(string, string, object, string) ,
ControllerBase.RedirectToPagePermanent(string) ,
ControllerBase.RedirectToPagePermanent(string, object) ,
ControllerBase.RedirectToPagePermanent(string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, string) ,
ControllerBase.RedirectToPagePermanent(string, string, object, string) ,
ControllerBase.RedirectToPagePreserveMethod(string, string, object, string) ,
ControllerBase.RedirectToPagePermanentPreserveMethod(string, string, object, string) ,
ControllerBase.File(byte[], string) , ControllerBase.File(byte[], string, bool) ,
ControllerBase.File(byte[], string, string) , ControllerBase.File(byte[], string, string, bool) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(byte[], string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string) , ControllerBase.File(Stream, string, bool) ,
ControllerBase.File(Stream, string, string) , ControllerBase.File(Stream, string, string, bool) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(Stream, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string) , ControllerBase.File(string, string, bool) ,
ControllerBase.File(string, string, string) , ControllerBase.File(string, string, string, bool) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.File(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string) , ControllerBase.PhysicalFile(string, string, bool) ,
ControllerBase.PhysicalFile(string, string, string) ,
ControllerBase.PhysicalFile(string, string, string, bool) ,

ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue) ,
ControllerBase.PhysicalFile(string, string, string, DateTimeOffset?, EntityTagHeaderValue, bool) ,
ControllerBase.Unauthorized() , ControllerBase.Unauthorized(object) , ControllerBase.NotFound() ,
ControllerBase.NotFound(object) , ControllerBase.BadRequest() ,
ControllerBase.BadRequest(object) , ControllerBase.BadRequest(ModelStateDictionary) ,
ControllerBase.UnprocessableEntity() , ControllerBase.UnprocessableEntity(object) ,
ControllerBase.UnprocessableEntity(ModelStateDictionary) , ControllerBase.Conflict() ,
ControllerBase.Conflict(object) , ControllerBase.Conflict(ModelStateDictionary) ,
ControllerBase.Problem(string, string, int?, string, string) ,
ControllerBase.ValidationProblem(ValidationProblemDetails) ,
ControllerBase.ValidationProblem(ModelStateDictionary) , ControllerBase.ValidationProblem() ,
ControllerBase.ValidationProblem(string, string, int?, string, string, ModelStateDictionary) ,
ControllerBase.Created() , ControllerBase.Created(string, object) ,
ControllerBase.Created(Uri, object) , ControllerBase.CreatedAtAction(string, object) ,
ControllerBase.CreatedAtAction(string, object, object) ,
ControllerBase.CreatedAtAction(string, string, object, object) ,
ControllerBase.CreatedAtRoute(string, object) , ControllerBase.CreatedAtRoute(object, object) ,
ControllerBase.CreatedAtRoute(string, object, object) , ControllerBase.Accepted() ,
ControllerBase.Accepted(object) , ControllerBase.Accepted(Uri) , ControllerBase.Accepted(string) ,
ControllerBase.Accepted(string, object) , ControllerBase.Accepted(Uri, object) ,
ControllerBase.AcceptedAtAction(string) , ControllerBase.AcceptedAtAction(string, string) ,
ControllerBase.AcceptedAtAction(string, object) ,
ControllerBase.AcceptedAtAction(string, string, object) ,
ControllerBase.AcceptedAtAction(string, object, object) ,
ControllerBase.AcceptedAtAction(string, string, object, object) ,
ControllerBase.AcceptedAtRoute(object) , ControllerBase.AcceptedAtRoute(string) ,
ControllerBase.AcceptedAtRoute(string, object) , ControllerBase.AcceptedAtRoute(object, object) ,
ControllerBase.AcceptedAtRoute(string, object, object) , ControllerBase.Challenge() ,
ControllerBase.Challenge(params string[]) , ControllerBase.Challenge(AuthenticationProperties) ,
ControllerBase.Challenge(AuthenticationProperties, params string[]) , ControllerBase.Forbid() ,
ControllerBase.Forbid(params string[]) , ControllerBase.Forbid(AuthenticationProperties) ,
ControllerBase.Forbid(AuthenticationProperties, params string[]) ,
ControllerBase.SignIn(ClaimsPrincipal) , ControllerBase.SignIn(ClaimsPrincipal, string) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties) ,
ControllerBase.SignIn(ClaimsPrincipal, AuthenticationProperties, string) , ControllerBase.SignOut() ,
ControllerBase.SignOut(AuthenticationProperties) , ControllerBase.SignOut(params string[]) ,
ControllerBase.SignOut(AuthenticationProperties, params string[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel) ,

ControllerBase.TryUpdateModelAsync<TModel>(TModel, string) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, params Expression<Func<TModel, object>>[]) ,
ControllerBase.TryUpdateModelAsync<TModel>(TModel, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryUpdateModelAsync(object, Type, string) ,
ControllerBase.TryUpdateModelAsync(object, Type, string, IValueProvider, Func<ModelMetadata, bool>) ,
ControllerBase.TryValidateModel(object) , ControllerBase.TryValidateModel(object, string) ,
ControllerBase.HttpContext , ControllerBase.Request , ControllerBase.Response ,
ControllerBase.RouteData , ControllerBase.ModelState , ControllerBase.ControllerContext ,
ControllerBase.MetadataProvider , ControllerBase.ModelBinderFactory , ControllerBase.Url ,
ControllerBase.ObjectValidator , ControllerBase.ProblemDetailsFactory , ControllerBase.User ,
ControllerBase.Empty , object.Equals(object) , object.Equals(object, object) ,
object.GetHashCode() , object.GetType() , object.MemberwiseClone() ,
object.ReferenceEquals(object, object) , object.ToString()

Constructors

UserController(UserLogic)

Parameterized constructor to bring in DI variables

```
public UserController(UserLogic userLogic)
```

Parameters

`userLogic` UserLogic

Methods

DeleteUserAsync()

Delete a user based on a userId

```
[HttpDelete]
[Authorize]
public Task<ActionResult> DeleteUserAsync()
```

Returns

Task <ActionResult>

GetUserAsync()

Method to get a user based on its id

```
[HttpGet]
[Authorize]
public Task<ActionResult> GetUserAsync()
```

Returns

Task <ActionResult>

LoginUserAsync(LoginRequest)

Method to login an existing user

```
[AllowAnonymous]
[HttpPost("login")]
public Task<ActionResult> LoginUserAsync(LoginRequest user)
```

Parameters

`user` LoginRequest

Returns

Task <ActionResult>

The code status, user id, token, and message

RegisterUserAsync(RegisterRequest)

Method to register a new user

```
[AllowAnonymous]
[HttpPost("register")]
public Task<ActionResult> RegisterUserAsync(RegisterRequest user)
```

Parameters

`user` RegisterRequest

Returns

Task<ActionResult>

The code status, user id, token, and message

UpdateUserAsync(EditUserRequest)

Method to update a user based on the given fields

```
[HttpPut]
[Authorize]
public Task<ActionResult> UpdateUserAsync(EditUserRequest user)
```

Parameters

`user` EditUserRequest

Returns

Task<ActionResult>

Namespace MakesCentsBackend.Models

Classes

AccountEntityModel
: Model for an abstract account

ApplyBudgetTemplateRequest
: Request model to apply a template to a budget month, bulk-creating/updating that month's envelope funding queue from the template's entries

BankAccountEntityModel
: Model for a bank account

BankAccountSummaryDTOModel
: DTO model for getting all accounts

BaseIdRequest

BaseIdResponse
: Base Response for a create or edit function

BaseResponse
: A base response model with only a status and message

BudgetEntityModel
: Entity model for a budget

BudgetTemplateEntityModel
: Model for a budget template - a named, reusable preset of envelope funding amounts (e.g. "Summer", "Holiday") that can be applied to a budget month to bulk-create its funding queue. Templates aren't tied to any specific month - their envelope entries reference persistent envelopes, not a month's instance of one.

BudgetTemplateEnvelopeLookupModel
: Internal helper: one template envelope entry as read back for applying it to a month - not exposed to the API directly.

CreateAccountRequest

CreateAccountResponse
: Response model for an account

CreateBankAccountRequest
: Request model for creating a bank account model

CreateBankAccountResponse
Response model for creating a bank account

CreateBudgetMonthRequest
Request model for creating a new month under an existing persistent budget

CreateBudgetRequest
Request model for creating the persistent budget entity (no month yet - see CreateBudgetMonthRequest)

CreateBudgetTemplateRequest
Request model to create a budget template. No envelopes here yet - use SetBudgetTemplateEnvelopesRequest to add/edit its entries afterward.

CreateDebtAccountRequest
Request model for creating a debt account model

CreateDebtAccountResponse
Response model for creating a debt account

CreateEnvelopeCategoryRequest
Request model for creating an envelope category

CreateInvestmentAccountRequest
Request model to create an investment account

CreateInvestmentAccountResponse
Response model for creating an investment account

CreatePaycheckRequest
Request model to create a paycheck

CreatePaymentTransactionRequest
Request model to create a new payment transaction

CreatePaymentTransactionResponse
Response model for creating a payment transaction

CreateRolloverEnvelopeRequest
Request class for creating a rollover envelope's persistent definition

CreateSinkingFundEnvelopeRequest
Request class for creating a sinking fund envelope

CreateTransactionRequest

Request model to create a new transaction

CreateTransactionSplitRequest
Request model for creating a new transaction split

CreateTransferTransactionRequest
Request model to create a new transfer transaction

CreateTransferTransactionResponse

DebtAccountEntityModel
Model for a debt account

DebtAccountSummaryDTOModel

EditBudgetMonthRequest
Request model for updating a specific budget month's month/year (e.g. correcting a month created under the wrong month/year)

EditBudgetRequest
Request model for updating a budget's persistent fields (currently just the name)

EditBudgetTemplateRequest
Request model to rename a budget template

EditEnvelopeCategoryRequest
Request model for editing an envelope category

EditUserRequest
Request model for editing a user

EnvelopeCategoryEntityModel
Model for an envelope category

EnvelopeFundingEntityModel
Model for a single envelope funding row - one envelope's planned funding amount and priority order for a specific budget month. This is a plan, not a money movement - actual money still moves into envelopes via transactions. EnvelopeId is a display/API-level id - IsSinkingFund says whether it refers to a sinking_fund_envelope_id or the persistent rollover_envelope_id (not the month-specific rollover_envelope_month_id, which is an internal DAO detail).

FindUserResponse

GetAccountBaseModel
Base class for getting a specific account

GetAllAccountsResponse
Response model for getting all accounts

GetAllBudgetTemplatesResponse

GetAllEnvelopeCategoriesResponse
Response model for getting all envelope categories

GetAllPaychecksResponse

GetAllTransactionsDTOResponse

GetAllTransactionsEntityResponse

GetBankAccountDTOModel
DTO model to get a specific bank account

GetBankAccountDTOResponse
Response model for getting a specific bank account

GetBankAccountEntityModel
Entity model to get a specific bank account

GetBankAccountEntityResponse
Response model for getting a specific bank account

GetBudgetMonthDTOModel
DTO model for sending a budget month to the API

GetBudgetMonthResponse
Response model for getting a budget month

GetBudgetRequest
Shared lookup key: a user id plus a year/month anchor point. Used both to fetch a single budget month (GetBudgetMonthAsync) and as the anchor for a rolling 12-month range (GetYearBudgetsAsync) - not itself a response, just the query key for both.

GetBudgetTemplateDTOModel

GetBudgetTemplateEnvelopeDTOModel
Model for a single envelope entry within a budget template. EnvelopeId is a display/API-level id - IsSinkingFund says whether it refers to a sinking_fund_envelope_id or the persistent rollover_envelope_id.

GetBudgetTemplateResponse

GetDebtAccountDTOModel

DTO model to get a specific debt account

GetDebtAccountDTOResponse

Response model for getting a specific debt account

GetDebtAccountEntityModel

Entity model to get a specific debt account

GetDebtAccountEntityResponse

Response model for getting a specific debt account

GetEnvelopeCategoryDTOModel

GetEnvelopeCategoryResponse

GetEnvelopeFundingDTOModel

DTO model for a single envelope funding row

GetEnvelopeFundingQueueResponse

Response model for a budget month's funding queue, in priority order

GetInvestmentAccountDTOModel

DTO model to get a specific investment account

GetInvestmentAccountDTOResponse

Response model for getting a specific investment account

GetInvestmentAccountEntityModel

Entity model to get a specific investment account

GetInvestmentAccountEntityResponse

Response model for getting a specific investment account

GetPaycheckDTOModel

GetPaycheckResponse

GetPaymentTransactionDTOModel

GetPaymentTransactionResponse

GetRolloverEnvelopeBaseModel

Base fields returned for a rollover envelope in the context of one budget month. RemainingAmount/RolloverEnvelopeMonthId are null if not yet activated for that month. Planned funding for the month lives on envelope_funding, not here.

GetRolloverEnvelopeDTOModel

DTO model to get a specific rollover envelope for a given month

GetRolloverEnvelopeDTOResponse

Response model for getting a specific rollover envelope

GetRolloverEnvelopeEntityModel

Entity model for getting a specific rollover envelope

GetRolloverEnvelopeEntityResponse

Entity response model for getting a specific rollover envelope

GetRolloverEnvelopeMonthRequest

Request used to look up a rollover envelope in the context of one specific budget month. Built server-side from the route/JWT, same as BaseIdRequest - not sent by the client directly.

GetSinkingFundDependentsResponse

Response model for the rollover envelopes that use a sinking fund as their transfer target

GetSinkingFundEnvelopeBaseModel

Base fields returned for a sinking fund envelope

GetSinkingFundEnvelopeDTOModel

DTO model to get a specific sinking fund envelope

GetSinkingFundEnvelopeDTOResponse

Response model for getting a specific sinking fund envelope

GetSinkingFundEnvelopeEntityModel

Entity model for getting a specific sinking fund envelope

GetSinkingFundEnvelopeEntityResponse

Entity response model for getting a specific sinking fund envelope

GetTransactionDTOModel

GetTransactionEntityModel

GetTransactionSplitDTOModel

GetTransferTransactionDTOModel

GetTransferTransactionDTOResponse

GetTransferTransactionEntityModel

Entity model for a specific transfer transaction. TransferFromEnvelopeId/ToEnvelopeId and their IsSinkingFund flags are only populated (via a follow-up transaction_split lookup) when TransferTransactionType is Envelope - they aren't columns on transfer_transaction itself.

GetTransferTransactionEntityResponse

GetUserBudgetsResponse
Response model for every budget a user has access to

GetUserDTOModel
DTO model for getting a user

GetUserResponse

GetYearBudgetDTOModel

GetYearBudgetsResponse

InvestmentAccountEntityModel
Model for an investment account

InvestmentAccountSummaryDTOModel

LoginRequest
Request model for a user

LoginResponse
Response model for the login process

PaycheckEntityModel
Model for a paycheck. Forecast-only - a paycheck no longer funds envelopes directly (see envelope_funding for that) and isn't tied to any specific month. It exists purely to represent expected income on a recurring schedule.

PaymentTransactionEntityModel
Model for a payment transaction

PaymentTransactionSnapshotModel
Internal helper: the current account_id/total_amount/budget_month_id for a payment transaction, read before an update is applied so the old account-balance effect can be reversed and the new one applied cleanly.

RegisterRequest

RegisterResponse

Response model for registration

RolloverEnvelopeDependentModel

A rollover envelope that sweeps into a given sinking fund envelope

RolloverEnvelopeEntityModel

Model for a rollover envelope's persistent definition. No dollar amounts live here - a rollover envelope isn't guaranteed to be active every month, so amounts live on the per-month instance (see RolloverEnvelopeMonthEntityModel).

RolloverEnvelopeMonthEntityModel

Model for a single month's instance of a rollover envelope. Absence of a row for a given (RolloverEnvelopeId, BudgetMonthId) pair means the envelope was inactive that month.

SetBudgetTemplateEnvelopeItemRequest

Request model for a single item within a template's envelope list being set. BudgetTemplateEnvelopeId is null for a brand new entry, or the existing entry's id to update it in place.

SetBudgetTemplateEnvelopesRequest

Request model to set (create/reorder/update/remove all at once) a template's envelope entries. The list's order is the funding priority for whenever this template is applied.

SetEnvelopeFundingItemRequest

Request model for a single item within a funding queue being set. EnvelopeFundingId is null for a brand new row, or the existing row's id to update it in place.

SetEnvelopeFundingQueueRequest

Request model to set (create/reorder/update/remove all at once) the funding queue for a budget month. The list's order is the funding priority - item 0 is funded first.

SinkingFundEnvelopeEntityModel

Model for a sinking fund envelope. Sinking fund envelopes are persistent - one row holds a live remaining_amount that carries forward month to month, unlike rollover envelopes.

SummaryAccountDTOModel

SummaryBudgetResponse

Response model for a single budget in a user's budget list - just enough to let them pick which one to open, before any month/id is known.

SummaryBudgetTemplateResponse

Response model for a single template in a budget's template list

SummaryEnvelopeCategoryResponse

Response model for an envelope category for the Get Budget method

SummaryEnvelopeResponse
Response model for a single envelope for the Get Budget method. Sinking fund and rollover envelopes come from two different tables with two different id spaces, so IsSinkingFund tells the frontend which table EnvelopeId belongs to (same pattern SummaryAccountDTOModel uses to distinguish bank/debt/investment accounts). RemainingAmount is nullable because a rollover envelope may not be activated for the month being viewed.

SummaryPaycheckResponse

SummaryTransactionDTOModel

SummaryTransactionEntityModel

TransactionEffectSnapshotModel
Internal helper: what a transaction currently does to account balances, plus whether it is soft-deleted. Used by the delete/restore methods to reverse or re-apply its effect.

TransactionEntityModel
Model for an abstract transaction

TransactionSplitEntityModel
Model for a transaction split model. EnvelopeId is a display/API-level id - IsSinkingFund says whether it refers to a sinking_fund_envelope_id or the persistent rollover_envelope_id (not the month-specific rollover_envelope_month_id, which is an internal DAO detail).

TransactionSplitSnapshotModel
Internal helper: the raw current state of one transaction_split row, used to reverse its old effect on an envelope's remaining amount before an update applies the new one.

TransactionSplitTransferSideModel
Internal helper for reading one side of an envelope-to-envelope transfer back out of transaction_split (used by TransferTransactionDAO/TransactionDAO, not exposed to the API).

TransferTransactionEntityModel
Model for a transfer transaction. v3's transfer_transaction table only carries account columns - an envelope-to-envelope transfer is a transfer_transaction row of type Envelope with null account columns, paired with two opposite-signed transaction_split rows that carry the actual envelope references.

TransferTransactionSnapshotModel
Internal helper: the current type/accounts/total_amount/budget_month_id for a transfer transaction, read before an update is applied so the old effect (account balances or envelope remaining amounts,

depending on type) can be reversed and the new one applied.

UpdateAccountRequest

UpdateBankAccountRequest

UpdateDebtAccountRequest

UpdateInvestmentAccountRequest

UpdatePaycheckRequest

UpdatePaymentTransactionRequest

UpdatePaymentTransactionResponse

UpdateRolloverEnvelopeRequest
Request class for updating a rollover envelope's persistent definition

UpdateSinkingFundEnvelopeRequest
Request class for updating a sinking fund envelope

UpdateTransactionRequest

UpdateTransactionSplitRequest

UpdateTransferTransactionRequest

UserDTOModel
Model for a user

UserDTOResponse
Response model for a DTO user

UserEntityModel
Model for a user

UserEntityResponse
Response model for an entity user

Structs

Optional<T>
Optional class for creating and updating nulls

Interfaces

IOptional

Class AccountEntityModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Model for an abstract account

```
public abstract class AccountEntityModel
```

Inheritance

object ← AccountEntityModel

Derived

BankAccountEntityModel, DebtAccountEntityModel, InvestmentAccountEntityModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

AccountId

```
public int AccountId { get; set; }
```

Property Value

int

AccountName

```
public string AccountName { get; set; }
```

Property Value

string

AccountType

```
public AccountType AccountType { get; set; }
```

Property Value

AccountType

Balance

```
public decimal Balance { get; set; }
```

Property Value

decimal

BudgetId

```
public int BudgetId { get; set; }
```

Property Value

int

CreatedAt

```
public DateTime CreatedAt { get; set; }
```

Property Value

DateTime

Institution

```
public string Institution { get; set; }
```

Property Value

string

LastUpdatedAt

```
public DateTime LastUpdatedAt { get; set; }
```

Property Value

DateTime

Transactions

```
public List<TransactionEntityModel> Transactions { get; set; }
```

Property Value

List <TransactionEntityModel>

Class ApplyBudgetTemplateRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Request model to apply a template to a budget month, bulk-creating/updating that month's envelope funding queue from the template's entries

```
[ExportTsClass]
public class ApplyBudgetTemplateRequest
```

Inheritance

object ← ApplyBudgetTemplateRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetMonthId

```
public int BudgetMonthId { get; set; }
```

Property Value

int

BudgetTemplateId

```
public int BudgetTemplateId { get; set; }
```

Property Value

int

UserId

```
public int UserId { get; set; }
```

Property Value

int

Class BankAccountEntityModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Model for a bank account

```
public class BankAccountEntityModel : AccountEntityModel
```

Inheritance

object ← AccountEntityModel ← BankAccountEntityModel

Inherited Members

AccountEntityModel.AccountId , AccountEntityModel.BudgetId , AccountEntityModel.AccountType , AccountEntityModel.AccountName , AccountEntityModel.Institution , AccountEntityModel.Balance , AccountEntityModel.CreatedAt , AccountEntityModel.LastUpdatedAt , AccountEntityModel.Transactions , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BankAccountId

```
public int BankAccountId { get; set; }
```

Property Value

int

BankAccountType

```
public BankAccountType BankAccountType { get; set; }
```

Property Value

BankAccountType

Class BankAccountSummaryDTOModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

DTO model for getting all accounts

```
[ExportTsInterface]
public class BankAccountSummaryDTOModel : SummaryAccountDTOModel
```

Inheritance

object ← SummaryAccountDTOModel ← BankAccountSummaryDTOModel

Inherited Members

SummaryAccountDTOModel.AccountId , SummaryAccountDTOModel.BudgetId , SummaryAccountDTOModel.AccountType , SummaryAccountDTOModel.AccountName , SummaryAccountDTOModel.Balance , SummaryAccountDTOModel.Institution , SummaryAccountDTOModel.BankAccountType , SummaryAccountDTOModel.DebtAccountType , SummaryAccountDTOModel.InvestmentAccountType , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BankAccountId

```
public int BankAccountId { get; set; }
```

Property Value

int

Class BaseIdRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
public class BaseIdRequest
```

Inheritance

object ← BaseIdRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

BaseIdRequest()

```
public BaseIdRequest()
```

BaseIdRequest(int, int)

```
public BaseIdRequest(int userId, int entityId)
```

Parameters

`userId` int

`entityId` int

Properties

EntityId

```
public int EntityId { get; set; }
```

Property Value

int

UserId

```
public int UserId { get; set; }
```

Property Value

int

Class BaseIdResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Base Response for a create or edit function

```
[ExportTsInterface]
public class BaseIdResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← BaseIdResponse

Derived

CreatePaymentTransactionResponse, CreateTransferTransactionResponse, FindUserResponse, GetEnvelopeCategoryResponse, GetYearBudgetsResponse, LoginResponse, RegisterResponse, UpdatePaymentTransactionResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

BaseIdResponse()

Default constructor for a base Id response

```
public BaseIdResponse()
```

BaseIdResponse(int, string)

Parameterized constructor for BaseIdResponse

```
public BaseIdResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

`message` string

BaseIdResponse(int, string, int?)

Parameterized constructor for BaseIdResponse

```
public BaseIdResponse(int httpStatus, string message, int? id)
```

Parameters

`httpStatus` int

`message` string

`id` int?

Properties

Id

```
public int Id { get; set; }
```

Property Value

int

Class BaseResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

A base response model with only a status and message

```
[ExportTsInterface]
public class BaseResponse
```

Inheritance

object ← BaseResponse

Derived

BaseIdResponse, CreateAccountResponse, GetAllAccountsResponse, GetAllBudgetTemplatesResponse, GetAllEnvelopeCategoriesResponse, GetAllPaychecksResponse, GetAllTransactionsDTOResponse, GetAllTransactionsEntityResponse, GetBankAccountDTOResponse, GetBankAccountEntityResponse, GetBudgetMonthResponse, GetBudgetTemplateResponse, GetDebtAccountDTOResponse, GetDebtAccountEntityResponse, GetEnvelopeFundingQueueResponse, GetInvestmentAccountDTOResponse, GetInvestmentAccountEntityResponse, GetPaycheckResponse, GetPaymentTransactionResponse, GetRolloverEnvelopeDTOResponse, GetRolloverEnvelopeEntityResponse, GetSinkingFundDependentsResponse, GetSinkingFundEnvelopeDTOResponse, GetSinkingFundEnvelopeEntityResponse, GetTransferTransactionDTOResponse, GetTransferTransactionEntityResponse, GetUserBudgetsResponse, GetUserResponse, UserDTOResponse, UserEntityResponse

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

BaseResponse()

Default constructor for a base response

```
public BaseResponse()
```

BaseResponse(int, string)

Parameterized constructor for a base response

```
public BaseResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

`message` string

Properties

HttpStatus

```
public int HttpStatus { get; set; }
```

Property Value

int

Message

```
public string Message { get; set; }
```

Property Value

string

Class BudgetEntityModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Entity model for a budget

```
public class BudgetEntityModel
```

Inheritance

object ← BudgetEntityModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Accounts

```
public List<AccountEntityModel> Accounts { get; set; }
```

Property Value

List<AccountEntityModel>

BudgetId

```
public int BudgetId { get; set; }
```

Property Value

int

BudgetName

```
public string BudgetName { get; set; }
```

Property Value

string

CreatedAt

```
public DateTime CreatedAt { get; set; }
```

Property Value

DateTime

EnvelopeCategories

```
public List<EnvelopeCategoryEntityModel> EnvelopeCategories { get; set; }
```

Property Value

List<EnvelopeCategoryEntityModel>

LastUpdatedAt

```
public DateTime LastUpdatedAt { get; set; }
```

Property Value

DateTime

Month

```
public Month Month { get; set; }
```

Property Value

Month

Paychecks

```
public List<PaycheckEntityModel> Paychecks { get; set; }
```

Property Value

List <PaycheckEntityModel>

PlannedExpenses

```
public List<PlannedExpenseEntity> PlannedExpenses { get; set; }
```

Property Value

List <PlannedExpenseEntity>

Transactions

```
public List<TransactionEntityModel> Transactions { get; set; }
```

Property Value

List <TransactionEntityModel>

UserId

```
public int UserId { get; set; }
```

Property Value

int

Year

```
public int Year { get; set; }
```

Property Value

int

Class BudgetTemplateEntityModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Model for a budget template - a named, reusable preset of envelope funding amounts (e.g. "Summer", "Holiday") that can be applied to a budget month to bulk-create its funding queue. Templates aren't tied to any specific month - their envelope entries reference persistent envelopes, not a month's instance of one.

```
public class BudgetTemplateEntityModel
```

Inheritance

object ← BudgetTemplateEntityModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetId

```
public int BudgetId { get; set; }
```

Property Value

int

BudgetTemplateId

```
public int BudgetTemplateId { get; set; }
```

Property Value

int

CreatedAt

```
public DateTime CreatedAt { get; set; }
```

Property Value

DateTime

LastUpdatedAt

```
public DateTime LastUpdatedAt { get; set; }
```

Property Value

DateTime

TemplateName

```
public string TemplateName { get; set; }
```

Property Value

string

Class BudgetTemplateEnvelopeLookupModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Internal helper: one template envelope entry as read back for applying it to a month - not exposed to the API directly.

```
public class BudgetTemplateEnvelopeLookupModel
```

Inheritance

object ← BudgetTemplateEnvelopeLookupModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

EnvelopeId

```
public int EnvelopeId { get; set; }
```

Property Value

int

IsSinkingFund

```
public bool IsSinkingFund { get; set; }
```

Property Value

bool

PlannedAmount

```
public decimal PlannedAmount { get; set; }
```

Property Value

decimal

Class CreateAccountRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsClass]
public abstract class CreateAccountRequest
```

Inheritance

object ← CreateAccountRequest

Derived

CreateBankAccountRequest, CreateDebtAccountRequest, CreateInvestmentAccountRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

AccountId

```
public int AccountId { get; set; }
```

Property Value

int

AccountName

```
[JsonConverter(typeof(SafeDefaultConverter<string>))]
public string AccountName { get; set; }
```

Property Value

string

Balance

```
[JsonConverter(typeof(SafeDefaultConverter<decimal>))]
public decimal Balance { get; set; }
```

Property Value

decimal

BudgetId

```
[JsonConverter(typeof(SafeDefaultConverter<int>))]
public int BudgetId { get; set; }
```

Property Value

int

Institution

```
[JsonConverter(typeof(SafeDefaultConverter<string>))]
public string Institution { get; set; }
```

Property Value

string

UserId

```
public int UserId { get; set; }
```

Property Value

int

Class CreateAccountResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Response model for an account

```
[ExportTsInterface]
public class CreateAccountResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← CreateAccountResponse

Derived

CreateBankAccountResponse, CreateDebtAccountResponse, CreateInvestmentAccountResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

CreateAccountResponse()

Default constructor for an account response

```
public CreateAccountResponse()
```

CreateAccountResponse(int, string)

Parameterized constructor for an account response

```
public CreateAccountResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

`message` string

Properties

AccountId

```
public int? AccountId { get; set; }
```

Property Value

int?

Class CreateBankAccountRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Request model for creating a bank account model

```
[ExportTsClass]
public class CreateBankAccountRequest : CreateAccountRequest
```

Inheritance

object ← CreateAccountRequest ← CreateBankAccountRequest

Inherited Members

CreateAccountRequest.BudgetId , CreateAccountRequest.UserId , CreateAccountRequest.AccountId , CreateAccountRequest.AccountName , CreateAccountRequest.Institution , CreateAccountRequest.Balance , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BankAccountType

```
[JsonConverter(typeof(SafeDefaultConverter<BankAccountType>))]
public BankAccountType BankAccountType { get; set; }
```

Property Value

BankAccountType

Class CreateBankAccountResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Response model for creating a bank account

```
[ExportTsInterface]
public class CreateBankAccountResponse : CreateAccountResponse
```

Inheritance

object ← BaseResponse ← CreateAccountResponse ← CreateBankAccountResponse

Inherited Members

CreateAccountResponse.AccountId , BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

CreateBankAccountResponse()

Default constructor for a bank account response

```
public CreateBankAccountResponse()
```

CreateBankAccountResponse(int, string)

Parameterized constructor for a bank account response

```
public CreateBankAccountResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

`message` string

Properties

BankAccountId

```
public int? BankAccountId { get; set; }
```

Property Value

int?

Class CreateBudgetMonthRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Request model for creating a new month under an existing persistent budget

```
[ExportTsClass]
public class CreateBudgetMonthRequest
```

Inheritance

object ← CreateBudgetMonthRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetId

```
public int BudgetId { get; set; }
```

Property Value

int

BudgetTemplateId

```
public int BudgetTemplateId { get; set; }
```

Property Value

int

Month

```
[JsonConverter(typeof(SafeDefaultConverter<Month>))]
public Month Month { get; set; }
```

Property Value

Month

UserId

```
public int UserId { get; set; }
```

Property Value

int

Year

```
[JsonConverter(typeof(SafeDefaultConverter<int>))]
public int Year { get; set; }
```

Property Value

int

Class CreateBudgetRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Request model for creating the persistent budget entity (no month yet - see CreateBudgetMonthRequest)

```
[ExportTsClass]
public class CreateBudgetRequest
```

Inheritance

object ← CreateBudgetRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetName

```
[JsonConverter(typeof(SafeDefaultConverter<string>))]
public string BudgetName { get; set; }
```

Property Value

string

UserId

```
public int UserId { get; set; }
```

Property Value

int

Class CreateBudgetTemplateRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Request model to create a budget template. No envelopes here yet - use SetBudgetTemplateEnvelopesRequest to add/edit its entries afterward.

```
[ExportTsClass]
public class CreateBudgetTemplateRequest
```

Inheritance

object ← CreateBudgetTemplateRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetId

```
public int BudgetId { get; set; }
```

Property Value

int

TemplateName

```
public string TemplateName { get; set; }
```

Property Value

string

UserId

```
public int UserId { get; set; }
```

Property Value

int

Class CreateDebtAccountRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Request model for creating a debt account model

```
[ExportTsClass]
public class CreateDebtAccountRequest : CreateAccountRequest
```

Inheritance

object ← CreateAccountRequest ← CreateDebtAccountRequest

Inherited Members

CreateAccountRequest.BudgetId , CreateAccountRequest.UserId , CreateAccountRequest.AccountId , CreateAccountRequest.AccountName , CreateAccountRequest.Institution , CreateAccountRequest.Balance , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

AmountOfNextBill

```
public Optional<decimal?> AmountOfNextBill { get; set; }
```

Property Value

Optional<decimal?>

DateOfNextBill

```
public Optional<DateOnly?> DateOfNextBill { get; set; }
```

Property Value

Optional<DateOnly?>

DebtAccountNumber

```
public Optional<int?> DebtAccountNumber { get; set; }
```

Property Value

Optional<int?>

DebtAccountType

```
public DebtAccountType DebtAccountType { get; set; }
```

Property Value

DebtAccountType

DebtPaymentRegularity

```
public Optional<DebtPaymentRegularity?> DebtPaymentRegularity { get; set; }
```

Property Value

Optional<DebtPaymentRegularity?>

Class CreateDebtAccountResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Response model for creating a debt account

```
[ExportTsInterface]
public class CreateDebtAccountResponse : CreateAccountResponse
```

Inheritance

object ← BaseResponse ← CreateAccountResponse ← CreateDebtAccountResponse

Inherited Members

CreateAccountResponse.AccountId , BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

CreateDebtAccountResponse()

Default constructor for a debt account response

```
public CreateDebtAccountResponse()
```

CreateDebtAccountResponse(int, string)

Parameterized constructor for a debt account response

```
public CreateDebtAccountResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

`message` string

Properties

DebtAccountId

```
public int? DebtAccountId { get; set; }
```

Property Value

int?

Class CreateEnvelopeCategoryRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Request model for creating an envelope category

```
[ExportTsClass]
public class CreateEnvelopeCategoryRequest
```

Inheritance

object ← CreateEnvelopeCategoryRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetId

```
public int BudgetId { get; set; }
```

Property Value

int

EnvelopeCategoryName

```
public string EnvelopeCategoryName { get; set; }
```

Property Value

string

UserId

```
public int UserId { get; set; }
```

Property Value

int

Class CreateInvestmentAccountRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Request model to create an investment account

```
[ExportTsClass]
public class CreateInvestmentAccountRequest : CreateAccountRequest
```

Inheritance

object ← CreateAccountRequest ← CreateInvestmentAccountRequest

Inherited Members

CreateAccountRequest.BudgetId , CreateAccountRequest.UserId , CreateAccountRequest.AccountId , CreateAccountRequest.AccountName , CreateAccountRequest.Institution , CreateAccountRequest.Balance , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

InvestmentAccountNumber

```
public Optional<int?> InvestmentAccountNumber { get; set; }
```

Property Value

Optional<int?>

InvestmentAccountType

```
public InvestmentAccountType InvestmentAccountType { get; set; }
```

Property Value

InvestmentAccountType

IsTaxDeferred

```
public bool IsTaxDeferred { get; set; }
```

Property Value

bool

IsTaxExempt

```
public bool IsTaxExempt { get; set; }
```

Property Value

bool

Class CreateInvestmentAccountResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Response model for creating an investment account

```
[ExportTsInterface]
public class CreateInvestmentAccountResponse : CreateAccountResponse
```

Inheritance

object ← BaseResponse ← CreateAccountResponse ← CreateInvestmentAccountResponse

Inherited Members

CreateAccountResponse.AccountId , BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

CreateInvestmentAccountResponse()

Default constructor for an investment account response

```
public CreateInvestmentAccountResponse()
```

CreateInvestmentAccountResponse(int, string)

Parameterized constructor for an investment account response

```
public CreateInvestmentAccountResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

`message` string

Properties

InvestmentAccountId

```
public int? InvestmentAccountId { get; set; }
```

Property Value

int?

Class CreatePaycheckRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Request model to create a paycheck

```
[ExportTsClass]
public class CreatePaycheckRequest
```

Inheritance

object ← CreatePaycheckRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetId

```
public int BudgetId { get; set; }
```

Property Value

int

PaycheckName

```
public string PaycheckName { get; set; }
```

Property Value

string

PaycheckRegularity

```
public PaycheckRegularity PaycheckRegularity { get; set; }
```

Property Value

PaycheckRegularity

SecondaryDate

```
public Optional<DateOnly?> SecondaryDate { get; set; }
```

Property Value

Optional<DateOnly?>

StartingDate

```
public DateOnly StartingDate { get; set; }
```

Property Value

DateOnly

TotalAmount

```
public decimal TotalAmount { get; set; }
```

Property Value

decimal

UserId

```
public int UserId { get; set; }
```

Property Value

int

Class CreatePaymentTransactionRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Request model to create a new payment transaction

```
[ExportTsClass]
public class CreatePaymentTransactionRequest : CreateTransactionRequest
```

Inheritance

object ← CreateTransactionRequest ← CreatePaymentTransactionRequest

Inherited Members

CreateTransactionRequest.BudgetMonthId , CreateTransactionRequest.UserId , CreateTransactionRequest.TransactionId , CreateTransactionRequest.TransactionDate , CreateTransactionRequest.TotalAmount , CreateTransactionRequest.Notes , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

AccountId

```
public int AccountId { get; set; }
```

Property Value

int

CheckNumber

```
public Optional<int?> CheckNumber { get; set; }
```

Property Value

Optional<int?>

MerchantSourceName

```
public string MerchantSourceName { get; set; }
```

Property Value

string

PaymentTransactionType

```
public PaymentTransactionType PaymentTransactionType { get; set; }
```

Property Value

PaymentTransactionType

TransactionSplits

```
public List<CreateTransactionSplitRequest> TransactionSplits { get; set; }
```

Property Value

List<CreateTransactionSplitRequest>

Class CreatePaymentTransactionResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Response model for creating a payment transaction

```
[ExportTsInterface]
public class CreatePaymentTransactionResponse : BaseIdResponse
```

Inheritance

object ← BaseResponse ← BaseIdResponse ← CreatePaymentTransactionResponse

Inherited Members

BaseIdResponse.Id , BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

CreatePaymentTransactionResponse()

Default constructor for Create Payment Transaction Response

```
public CreatePaymentTransactionResponse()
```

CreatePaymentTransactionResponse(int, string)

Parameterized constructor for Create Payment Transaction Response

```
public CreatePaymentTransactionResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

`message` string

Properties

PaymentTransactionId

```
public int? PaymentTransactionId { get; set; }
```

Property Value

int?

TransactionSplitIds

```
public List<int> TransactionSplitIds { get; set; }
```

Property Value

List<int>

Class CreateRolloverEnvelopeRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Request class for creating a rollover envelope's persistent definition

```
[ExportTsClass]
public class CreateRolloverEnvelopeRequest
```

Inheritance

object ← CreateRolloverEnvelopeRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

EnvelopeCategoryId

```
public int EnvelopeCategoryId { get; set; }
```

Property Value

int

EnvelopeName

```
public string EnvelopeName { get; set; }
```

Property Value

string

TransferEnvelopeId

```
public int TransferEnvelopeId { get; set; }
```

Property Value

int

UserId

```
public int UserId { get; set; }
```

Property Value

int

Class CreateSinkingFundEnvelopeRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Request class for creating a sinking fund envelope

```
[ExportTsClass]
public class CreateSinkingFundEnvelopeRequest
```

Inheritance

object ← CreateSinkingFundEnvelopeRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

EnvelopeCategoryId

```
public int EnvelopeCategoryId { get; set; }
```

Property Value

int

EnvelopeName

```
public string EnvelopeName { get; set; }
```

Property Value

string

GoalAmount

```
public Optional<decimal?> GoalAmount { get; set; }
```

Property Value

Optional<decimal?>

GoalEndDate

```
public Optional<DateOnly?> GoalEndDate { get; set; }
```

Property Value

Optional<DateOnly?>

RemainingAmount

```
public decimal RemainingAmount { get; set; }
```

Property Value

decimal

UserId

```
public int UserId { get; set; }
```

Property Value

int

Class CreateTransactionRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Request model to create a new transaction

```
[ExportTsClass]
public abstract class CreateTransactionRequest
```

Inheritance

object ← CreateTransactionRequest

Derived

CreatePaymentTransactionRequest, CreateTransferTransactionRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetMonthId

```
public int BudgetMonthId { get; set; }
```

Property Value

int

Notes

```
public Optional<string?> Notes { get; set; }
```

Property Value

Optional<string>

TotalAmount

```
public decimal TotalAmount { get; set; }
```

Property Value

decimal

TransactionDate

```
public DateOnly TransactionDate { get; set; }
```

Property Value

DateOnly

TransactionId

```
public int TransactionId { get; set; }
```

Property Value

int

UserId

```
public int UserId { get; set; }
```

Property Value

int

Class CreateTransactionSplitRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Request model for creating a new transaction split

```
[ExportTsClass]
public class CreateTransactionSplitRequest
```

Inheritance

object ← CreateTransactionSplitRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Amount

```
public decimal Amount { get; set; }
```

Property Value

decimal

EnvelopeId

```
public int EnvelopeId { get; set; }
```

Property Value

int

IsSinkingFund

```
public bool IsSinkingFund { get; set; }
```

Property Value

bool

TransactionId

```
public int TransactionId { get; set; }
```

Property Value

int

Class CreateTransferTransactionRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Request model to create a new transfer transaction

```
[ExportTsClass]
public class CreateTransferTransactionRequest : CreateTransactionRequest
```

Inheritance

object ← CreateTransactionRequest ← CreateTransferTransactionRequest

Inherited Members

CreateTransactionRequest.BudgetMonthId , CreateTransactionRequest.UserId , CreateTransactionRequest.TransactionId , CreateTransactionRequest.TransactionDate , CreateTransactionRequest.TotalAmount , CreateTransactionRequest.Notes , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

TransferFromId

```
public int TransferFromId { get; set; }
```

Property Value

int

TransferFromIsSinkingFund

```
public bool? TransferFromIsSinkingFund { get; set; }
```

Property Value

bool?

TransferToId

```
public int TransferToId { get; set; }
```

Property Value

int

TransferToIsSinkingFund

```
public bool? TransferToIsSinkingFund { get; set; }
```

Property Value

bool?

TransferTransactionType

```
public TransferTransactionType TransferTransactionType { get; set; }
```

Property Value

TransferTransactionType

Class CreateTransferTransactionResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsInterface]
public class CreateTransferTransactionResponse : BaseIdResponse
```

Inheritance

object ← BaseResponse ← BaseIdResponse ← CreateTransferTransactionResponse

Inherited Members

BaseIdResponse.Id , BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

CreateTransferTransactionResponse()

Default constructor for Create Transfer Transaction Response

```
public CreateTransferTransactionResponse()
```

CreateTransferTransactionResponse(int, string)

Parameterized constructor for Create Transfer Transaction Response

```
public CreateTransferTransactionResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

`message` string

Properties

TransferTransactionId

```
public int? TransferTransactionId { get; set; }
```

Property Value

int?

Class DebtAccountEntityModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Model for a debt account

```
public class DebtAccountEntityModel : AccountEntityModel
```

Inheritance

object ← AccountEntityModel ← DebtAccountEntityModel

Inherited Members

AccountEntityModel.AccountId , AccountEntityModel.BudgetId , AccountEntityModel.AccountType , AccountEntityModel.AccountName , AccountEntityModel.Institution , AccountEntityModel.Balance , AccountEntityModel.CreatedAt , AccountEntityModel.LastUpdatedAt , AccountEntityModel.Transactions , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

AmountOfNextBill

```
public decimal? AmountOfNextBill { get; set; }
```

Property Value

decimal?

DateOfNextBill

```
public DateOnly? DateOfNextBill { get; set; }
```

Property Value

DateOnly?

DebtAccountId

```
public int DebtAccountId { get; set; }
```

Property Value

int

DebtAccountNumber

```
public string? DebtAccountNumber { get; set; }
```

Property Value

string

DebtAccountType

```
public DebtAccountType DebtAccountType { get; set; }
```

Property Value

DebtAccountType

DebtPaymentRegularity

```
public DebtPaymentRegularity? DebtPaymentRegularity { get; set; }
```

Property Value

DebtPaymentRegularity?

Class DebtAccountSummaryDTOModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsInterface]
public class DebtAccountSummaryDTOModel : SummaryAccountDTOModel
```

Inheritance

object ← SummaryAccountDTOModel ← DebtAccountSummaryDTOModel

Inherited Members

SummaryAccountDTOModel.AccountId , SummaryAccountDTOModel.BudgetId , SummaryAccountDTOModel.AccountType , SummaryAccountDTOModel.AccountName , SummaryAccountDTOModel.Balance , SummaryAccountDTOModel.Institution , SummaryAccountDTOModel.BankAccountType , SummaryAccountDTOModel.DebtAccountType , SummaryAccountDTOModel.InvestmentAccountType , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

DebtAccountId

```
public int DebtAccountId { get; set; }
```

Property Value

int

Class EditBudgetMonthRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Request model for updating a specific budget month's month/year (e.g. correcting a month created under the wrong month/year)

```
[ExportTsClass]
public class EditBudgetMonthRequest
```

Inheritance

object ← EditBudgetMonthRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetMonthId

```
public int BudgetMonthId { get; set; }
```

Property Value

int

Month

```
public Month? Month { get; set; }
```

Property Value

Month?

UserId

```
public int UserId { get; set; }
```

Property Value

int

Year

```
public int? Year { get; set; }
```

Property Value

int?

Class EditBudgetRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Request model for updating a budget's persistent fields (currently just the name)

```
[ExportTsClass]
public class EditBudgetRequest
```

Inheritance

object ← EditBudgetRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetId

```
public int BudgetId { get; set; }
```

Property Value

int

BudgetName

```
public string? BudgetName { get; set; }
```

Property Value

string

UserId

```
public int UserId { get; set; }
```

Property Value

int

Class EditBudgetTemplateRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Request model to rename a budget template

```
[ExportTsClass]
public class EditBudgetTemplateRequest
```

Inheritance

object ← EditBudgetTemplateRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetTemplateId

```
public int BudgetTemplateId { get; set; }
```

Property Value

int

TemplateName

```
public string? TemplateName { get; set; }
```

Property Value

string

UserId

```
public int UserId { get; set; }
```

Property Value

int

Class EditEnvelopeCategoryRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Request model for editing an envelope category

```
[ExportTsClass]
public class EditEnvelopeCategoryRequest
```

Inheritance

object ← EditEnvelopeCategoryRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

EnvelopeCategoryId

```
public int EnvelopeCategoryId { get; set; }
```

Property Value

int

EnvelopeCategoryName

```
public string? EnvelopeCategoryName { get; set; }
```

Property Value

string

UserId

```
public int UserId { get; set; }
```

Property Value

int

Class EditUserRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Request model for editing a user

```
[ExportTsClass]
public class EditUserRequest
```

Inheritance

object ← EditUserRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Email

```
public string? Email { get; set; }
```

Property Value

string

IsDarkMode

```
public bool? IsDarkMode { get; set; }
```

Property Value

bool?

PasswordHash

```
public string? PasswordHash { get; set; }
```

Property Value

string

UserId

```
public int UserId { get; set; }
```

Property Value

int

Username

```
public string? Username { get; set; }
```

Property Value

string

Class EnvelopeCategoryEntityModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Model for an envelope category

```
public class EnvelopeCategoryEntityModel
```

Inheritance

object ← EnvelopeCategoryEntityModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetId

```
public int BudgetId { get; set; }
```

Property Value

int

CreatedAt

```
public DateTime CreatedAt { get; set; }
```

Property Value

DateTime

EnvelopeCategoryId

```
public int EnvelopeCategoryId { get; set; }
```

Property Value

int

EnvelopeCategoryName

```
public string EnvelopeCategoryName { get; set; }
```

Property Value

string

Envelopes

```
public List<SummaryEnvelopeResponse> Envelopes { get; set; }
```

Property Value

List<SummaryEnvelopeResponse>

LastUpdatedAt

```
public DateTime LastUpdatedAt { get; set; }
```

Property Value

DateTime

Class EnvelopeFundingEntityModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Model for a single envelope funding row - one envelope's planned funding amount and priority order for a specific budget month. This is a plan, not a money movement - actual money still moves into envelopes via transactions. EnvelopeId is a display/API-level id - IsSinkingFund says whether it refers to a sinking_fund_envelope_id or the persistent rollover_envelope_id (not the month-specific rollover_envelope_month_id, which is an internal DAO detail).

```
public class EnvelopeFundingEntityModel
```

Inheritance

object ← EnvelopeFundingEntityModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetMonthId

```
public int BudgetMonthId { get; set; }
```

Property Value

int

CreatedAt

```
public DateTime CreatedAt { get; set; }
```

Property Value

DateTime

EnvelopeFundingId

```
public int EnvelopeFundingId { get; set; }
```

Property Value

int

EnvelopeId

```
public int EnvelopeId { get; set; }
```

Property Value

int

IsSinkingFund

```
public bool IsSinkingFund { get; set; }
```

Property Value

bool

LastUpdatedAt

```
public DateTime LastUpdatedAt { get; set; }
```

Property Value

DateTime

OrderIndex

```
public int OrderIndex { get; set; }
```

Property Value

int

PlannedAmount

```
public decimal PlannedAmount { get; set; }
```

Property Value

decimal

Class FindUserResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
public class FindUserResponse : BaseIdResponse
```

Inheritance

object ← BaseResponse ← BaseIdResponse ← FindUserResponse

Inherited Members

BaseIdResponse.Id , BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

FindUserResponse(int, string)

Parameterized constructor for status and message

```
public FindUserResponse(int status, string message)
```

Parameters

`status` int

`message` string

FindUserResponse(int, string, int)

Parameterized constructor for status, message, and user id

```
public FindUserResponse(int status, string message, int userId)
```

Parameters

`status` int

`message` string

`userId` int

FindUserResponse(int, string, int, string)

Parameterized constructor for user id, username or email, password hash, and message

```
public FindUserResponse(int status, string message, int userId, string passwordHash)
```

Parameters

`status` int

`message` string

`userId` int

`passwordHash` string

Properties

PasswordHash

```
public string PasswordHash { get; set; }
```

Property Value

string

Token

```
public string Token { get; set; }
```

Property Value

string

Class GetAccountBaseModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Base class for getting a specific account

```
[ExportTsInterface]
public class GetAccountBaseModel
```

Inheritance

object ← GetAccountBaseModel

Derived

GetBankAccountDTOModel, GetBankAccountEntityModel, GetDebtAccountDTOModel, GetDebtAccountEntityModel, GetInvestmentAccountDTOModel, GetInvestmentAccountEntityModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

AccountId

```
public int AccountId { get; set; }
```

Property Value

int

AccountName

```
public string AccountName { get; set; }
```

Property Value

string

AccountType

```
public AccountType AccountType { get; set; }
```

Property Value

AccountType

Balance

```
public decimal Balance { get; set; }
```

Property Value

decimal

BudgetId

```
public int BudgetId { get; set; }
```

Property Value

int

Institution

```
public string Institution { get; set; }
```

Property Value

string

UserId

```
public int UserId { get; set; }
```

Property Value

int

Class GetAllAccountsResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Response model for getting all accounts

```
[ExportTsInterface]
public class GetAllAccountsResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← GetAllAccountsResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetAllAccountsResponse()

Initializes a new instance of the GetAllAccountsResponse class

```
public GetAllAccountsResponse()
```

GetAllAccountsResponse(int, string)

Initializes a new instance of the GetAllAccountsResponse class with the specified HTTP status code and message

```
public GetAllAccountsResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

The HTTP status code that represents the result of the request

`message` string

A message that provides additional information about the response

Properties

Accounts

```
public List<SummaryAccountDTOModel> Accounts { get; set; }
```

Property Value

List <SummaryAccountDTOModel>

Class GetAllBudgetTemplatesResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsInterface]
public class GetAllBudgetTemplatesResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← GetAllBudgetTemplatesResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetAllBudgetTemplatesResponse()

Default constructor for the get all budget templates response

```
public GetAllBudgetTemplatesResponse()
```

GetAllBudgetTemplatesResponse(int, string)

Parameterized constructor with a status and a message

```
public GetAllBudgetTemplatesResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

`message` string

Properties

Templates

```
public List<SummaryBudgetTemplateResponse> Templates { get; set; }
```

Property Value

List <SummaryBudgetTemplateResponse>

Class GetAllEnvelopeCategoriesResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Response model for getting all envelope categories

```
[ExportTsInterface]
public class GetAllEnvelopeCategoriesResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← GetAllEnvelopeCategoriesResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetAllEnvelopeCategoriesResponse()

Default constructor for GetEnvelopeCategoryResponse

```
public GetAllEnvelopeCategoriesResponse()
```

GetAllEnvelopeCategoriesResponse(int, string)

Parameterized constructor for GetEnvelopeCategoryResponse

```
public GetAllEnvelopeCategoriesResponse(int status, string message)
```

Parameters

`status` int

`message` string

Properties

EnvelopeCategories

```
public List<SummaryEnvelopeCategoryResponse> EnvelopeCategories { get; set; }
```

Property Value

List <SummaryEnvelopeCategoryResponse>

Class GetAllPaychecksResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsInterface]
public class GetAllPaychecksResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← GetAllPaychecksResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetAllPaychecksResponse()

Default constructor for the get all paychecks response

```
public GetAllPaychecksResponse()
```

GetAllPaychecksResponse(int, string)

Parameterized constructor with a status and a message

```
public GetAllPaychecksResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

`message` string

Properties

Paychecks

```
public List<SummaryPaycheckResponse> Paychecks { get; set; }
```

Property Value

List <SummaryPaycheckResponse>

Class GetAllTransactionsDTOResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsInterface]
public class GetAllTransactionsDTOResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← GetAllTransactionsDTOResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetAllTransactionsDTOResponse()

```
public GetAllTransactionsDTOResponse()
```

GetAllTransactionsDTOResponse(int, string)

```
public GetAllTransactionsDTOResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

`message` string

Properties

Transactions

```
public List<SummaryTransactionDTOModel> Transactions { get; set; }
```

Property Value

List <SummaryTransactionDTOModel>

Class GetAllTransactionsEntityResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
public class GetAllTransactionsEntityResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← GetAllTransactionsEntityResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetAllTransactionsEntityResponse()

```
public GetAllTransactionsEntityResponse()
```

GetAllTransactionsEntityResponse(int, string)

```
public GetAllTransactionsEntityResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

`message` string

Properties

Transactions

```
public List<SummaryTransactionEntityModel> Transactions { get; set; }
```

Property Value

List <SummaryTransactionEntityModel>

Class GetBankAccountDTOModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

DTO model to get a specific bank account

```
[ExportTsInterface]
public class GetBankAccountDTOModel : GetAccountBaseModel
```

Inheritance

object ← GetAccountBaseModel ← GetBankAccountDTOModel

Inherited Members

GetAccountBaseModel.AccountId , GetAccountBaseModel.BudgetId , GetAccountBaseModel.UserId , GetAccountBaseModel.AccountType , GetAccountBaseModel.AccountName , GetAccountBaseModel.Institution , GetAccountBaseModel.Balance , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetBankAccountDTOModel()

Default constructor for the get bank account DTO

```
public GetBankAccountDTOModel()
```

Properties

BankAccountId

```
public int BankAccountId { get; set; }
```

Property Value

int

BankAccountType

```
public BankAccountType BankAccountType { get; set; }
```

Property Value

BankAccountType

Transactions

```
public List<SummaryTransactionDTOModel> Transactions { get; set; }
```

Property Value

List <SummaryTransactionDTOModel>

Class GetBankAccountDTOResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Response model for getting a specific bank account

```
[ExportTsInterface]
public class GetBankAccountDTOResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← GetBankAccountDTOResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetBankAccountDTOResponse(int, string)

```
public GetBankAccountDTOResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

`message` string

Properties

BankAccount

```
public GetBankAccountDTOModel BankAccount { get; set; }
```

Property Value

GetBankAccountDTOModel

Class GetBankAccountEntityModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Entity model to get a specific bank account

```
public class GetBankAccountEntityModel : GetAccountBaseModel
```

Inheritance

object ← GetAccountBaseModel ← GetBankAccountEntityModel

Inherited Members

GetAccountBaseModel.AccountId , GetAccountBaseModel.BudgetId , GetAccountBaseModel.UserId , GetAccountBaseModel.AccountType , GetAccountBaseModel.AccountName , GetAccountBaseModel.Institution , GetAccountBaseModel.Balance , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BankAccountId

```
public int BankAccountId { get; set; }
```

Property Value

int

BankAccountType

```
public BankAccountType BankAccountType { get; set; }
```

Property Value

BankAccountType

Transactions

```
public List<SummaryTransactionEntityModel> Transactions { get; set; }
```

Property Value

List <SummaryTransactionEntityModel>

Class GetBankAccountEntityResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Response model for getting a specific bank account

```
public class GetBankAccountEntityResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← GetBankAccountEntityResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetBankAccountEntityResponse()

Default constructor

```
public GetBankAccountEntityResponse()
```

GetBankAccountEntityResponse(int, string)

Parameterized constructor that takes a status and a message

```
public GetBankAccountEntityResponse(int status, string message)
```

Parameters

`status` int

`message` string

Properties

BankAccount

```
public GetBankAccountEntityModel BankAccount { get; set; }
```

Property Value

GetBankAccountEntityModel

Class GetBudgetMonthDTOModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

DTO model for sending a budget month to the API

```
[ExportTsInterface]
public class GetBudgetMonthDTOModel
```

Inheritance

object ← GetBudgetMonthDTOModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetBudgetMonthDTOModel()

Default constructor for a GetBudgetMonthDTO

```
public GetBudgetMonthDTOModel()
```

Properties

BudgetId

```
public int BudgetId { get; set; }
```

Property Value

int

BudgetMonthId

```
public int BudgetMonthId { get; set; }
```

Property Value

int

BudgetName

```
public string BudgetName { get; set; }
```

Property Value

string

BudgetTemplateId

```
public int? BudgetTemplateId { get; set; }
```

Property Value

int?

EnvelopeCategories

```
public List<SummaryEnvelopeCategoryResponse> EnvelopeCategories { get; set; }
```

Property Value

List<SummaryEnvelopeCategoryResponse>

Month

```
public Month Month { get; set; }
```

Property Value

Month

UserId

```
public int UserId { get; set; }
```

Property Value

int

Year

```
public int Year { get; set; }
```

Property Value

int

Class GetBudgetMonthResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Response model for getting a budget month

```
[ExportTsInterface]
public class GetBudgetMonthResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← GetBudgetMonthResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetBudgetMonthResponse()

```
public GetBudgetMonthResponse()
```

GetBudgetMonthResponse(int, string)

```
public GetBudgetMonthResponse(int status, string message)
```

Parameters

`status` int

`message` string

Properties

GetBudgetMonthDTO

```
[JsonPropertyName("budget")]
[TsMemberName("budget")]
public GetBudgetMonthDTOModel GetBudgetMonthDTO { get; set; }
```

Property Value

GetBudgetMonthDTOModel

Class GetBudgetRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Shared lookup key: a user id plus a year/month anchor point. Used both to fetch a single budget month (GetBudgetMonthAsync) and as the anchor for a rolling 12-month range (GetYearBudgetsAsync) - not itself a response, just the query key for both.

```
[ExportTsClass]
public class GetBudgetRequest
```

Inheritance

object ← GetBudgetRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetBudgetRequest(int, int, Month, int)

Parameterized constructor for a Get Budget DTO

```
public GetBudgetRequest(int userId, int budgetId, Month month, int year)
```

Parameters

`userId` int

`budgetId` int

`month` Month

`year` int

Properties

BudgetId

```
public int BudgetId { get; set; }
```

Property Value

int

Month

```
[JsonConverter(typeof(SafeDefaultConverter<Month>))]
public Month Month { get; set; }
```

Property Value

Month

UserId

```
public int UserId { get; set; }
```

Property Value

int

Year

```
[JsonConverter(typeof(SafeDefaultConverter<int>))]
public int Year { get; set; }
```

Property Value

int

Class GetBudgetTemplateDTOModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsInterface]
public class GetBudgetTemplateDTOModel
```

Inheritance

object ← GetBudgetTemplateDTOModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetId

```
public int BudgetId { get; set; }
```

Property Value

int

BudgetTemplateId

```
public int BudgetTemplateId { get; set; }
```

Property Value

int

Envelopes

```
public List<GetBudgetTemplateEnvelopeDTOModel> Envelopes { get; set; }
```

Property Value

List <GetBudgetTemplateEnvelopeDTOModel>

TemplateName

```
public string TemplateName { get; set; }
```

Property Value

string

Class GetBudgetTemplateEnvelopeDTOModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Model for a single envelope entry within a budget template. EnvelopeId is a display/API-level id - IsSinkingFund says whether it refers to a sinking_fund_envelope_id or the persistent rollover_envelope_id.

```
[ExportTsInterface]
public class GetBudgetTemplateEnvelopeDTOModel
```

Inheritance

object ← GetBudgetTemplateEnvelopeDTOModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetTemplateEnvelopeId

```
public int BudgetTemplateEnvelopeId { get; set; }
```

Property Value

int

BudgetTemplateId

```
public int BudgetTemplateId { get; set; }
```

Property Value

int

EnvelopeId

```
public int EnvelopeId { get; set; }
```

Property Value

int

IsSinkingFund

```
public bool IsSinkingFund { get; set; }
```

Property Value

bool

OrderIndex

```
public int OrderIndex { get; set; }
```

Property Value

int

PlannedAmount

```
public decimal PlannedAmount { get; set; }
```

Property Value

decimal

Class GetBudgetTemplateResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsInterface]
public class GetBudgetTemplateResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← GetBudgetTemplateResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetBudgetTemplateResponse()

```
public GetBudgetTemplateResponse()
```

GetBudgetTemplateResponse(int, string)

```
public GetBudgetTemplateResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

`message` string

Properties

Template

```
public GetBudgetTemplateDTOModel Template { get; set; }
```

Property Value

GetBudgetTemplateDTOModel

Class GetDebtAccountDTOModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

DTO model to get a specific debt account

```
[ExportTsInterface]
public class GetDebtAccountDTOModel : GetAccountBaseModel
```

Inheritance

object ← GetAccountBaseModel ← GetDebtAccountDTOModel

Inherited Members

GetAccountBaseModel.AccountId , GetAccountBaseModel.BudgetId , GetAccountBaseModel.UserId , GetAccountBaseModel.AccountType , GetAccountBaseModel.AccountName , GetAccountBaseModel.Institution , GetAccountBaseModel.Balance , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetDebtAccountDTOModel()

Default constructor for the get debt account DTO

```
public GetDebtAccountDTOModel()
```

Properties

AccountNumber

```
public int? AccountNumber { get; set; }
```

Property Value

int?

AmountOfNextBill

```
public decimal? AmountOfNextBill { get; set; }
```

Property Value

decimal?

DateOfNextBill

```
public DateOnly? DateOfNextBill { get; set; }
```

Property Value

DateOnly?

DebtAccountId

```
public int DebtAccountId { get; set; }
```

Property Value

int

DebtAccountType

```
public DebtAccountType DebtAccountType { get; set; }
```

Property Value

DebtAccountType

DebtPaymentRegularity

```
public DebtPaymentRegularity DebtPaymentRegularity { get; set; }
```

Property Value

DebtPaymentRegularity

Transactions

```
public List<SummaryTransactionDTOModel> Transactions { get; set; }
```

Property Value

List <SummaryTransactionDTOModel>

Class GetDebtAccountDTOResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Response model for getting a specific debt account

```
[ExportTsInterface]
public class GetDebtAccountDTOResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← GetDebtAccountDTOResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetDebtAccountDTOResponse(int, string)

```
public GetDebtAccountDTOResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

`message` string

Properties

DebtAccount

```
public GetDebtAccountDTOModel DebtAccount { get; set; }
```

Property Value

GetDebtAccountDTOModel

Class GetDebtAccountEntityModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Entity model to get a specific debt account

```
public class GetDebtAccountEntityModel : GetAccountBaseModel
```

Inheritance

object ← GetAccountBaseModel ← GetDebtAccountEntityModel

Inherited Members

GetAccountBaseModel.AccountId , GetAccountBaseModel.BudgetId , GetAccountBaseModel.UserId , GetAccountBaseModel.AccountType , GetAccountBaseModel.AccountName , GetAccountBaseModel.Institution , GetAccountBaseModel.Balance , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

AccountNumber

```
public int? AccountNumber { get; set; }
```

Property Value

int?

AmountOfNextBill

```
public decimal? AmountOfNextBill { get; set; }
```

Property Value

decimal?

DateOfNextBill

```
public DateOnly? DateOfNextBill { get; set; }
```

Property Value

DateOnly?

DebtAccountId

```
public int DebtAccountId { get; set; }
```

Property Value

int

DebtAccountType

```
public DebtAccountType DebtAccountType { get; set; }
```

Property Value

DebtAccountType

DebtPaymentRegularity

```
public DebtPaymentRegularity DebtPaymentRegularity { get; set; }
```

Property Value

DebtPaymentRegularity

Transactions

```
public List<SummaryTransactionEntityModel> Transactions { get; set; }
```

Property Value

List <SummaryTransactionEntityModel>

Class GetDebtAccountEntityResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Response model for getting a specific debt account

```
public class GetDebtAccountEntityResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← GetDebtAccountEntityResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetDebtAccountEntityResponse()

Default constructor

```
public GetDebtAccountEntityResponse()
```

GetDebtAccountEntityResponse(int, string)

Parameterized constructor that takes a status and a message

```
public GetDebtAccountEntityResponse(int status, string message)
```

Parameters

`status` int

`message` string

Properties

DebtAccount

```
public GetDebtAccountEntityModel DebtAccount { get; set; }
```

Property Value

GetDebtAccountEntityModel

Class GetEnvelopeCategoryDTOModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsInterface]
public class GetEnvelopeCategoryDTOModel
```

Inheritance

object ← GetEnvelopeCategoryDTOModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetId

```
public int BudgetId { get; set; }
```

Property Value

int

EnvelopeCategoryName

```
public string EnvelopeCategoryName { get; set; }
```

Property Value

string

Class GetEnvelopeCategoryResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsInterface]
public class GetEnvelopeCategoryResponse : BaseIdResponse
```

Inheritance

object ← BaseResponse ← BaseIdResponse ← GetEnvelopeCategoryResponse

Inherited Members

BaseIdResponse.Id , BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetEnvelopeCategoryResponse()

```
public GetEnvelopeCategoryResponse()
```

GetEnvelopeCategoryResponse(int, string)

```
public GetEnvelopeCategoryResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

`message` string

Properties

EnvelopeCategory

```
public GetEnvelopeCategoryDTOModel EnvelopeCategory { get; set; }
```

Property Value

GetEnvelopeCategoryDTOModel

Class GetEnvelopeFundingDTOModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

DTO model for a single envelope funding row

```
[ExportTsInterface]
public class GetEnvelopeFundingDTOModel
```

Inheritance

object ← GetEnvelopeFundingDTOModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetMonthId

```
public int BudgetMonthId { get; set; }
```

Property Value

int

EnvelopeFundingId

```
public int EnvelopeFundingId { get; set; }
```

Property Value

int

EnvelopeId

```
public int EnvelopeId { get; set; }
```

Property Value

int

IsSinkingFund

```
public bool IsSinkingFund { get; set; }
```

Property Value

bool

OrderIndex

```
public int OrderIndex { get; set; }
```

Property Value

int

PlannedAmount

```
public decimal PlannedAmount { get; set; }
```

Property Value

decimal

Class GetEnvelopeFundingQueueResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Response model for a budget month's funding queue, in priority order

```
[ExportTsInterface]
public class GetEnvelopeFundingQueueResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← GetEnvelopeFundingQueueResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetEnvelopeFundingQueueResponse()

Default constructor for the get envelope funding queue response

```
public GetEnvelopeFundingQueueResponse()
```

GetEnvelopeFundingQueueResponse(int, string)

Parameterized constructor with a status and a message

```
public GetEnvelopeFundingQueueResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

`message` string

Properties

FundingQueue

```
public List<GetEnvelopeFundingDTOModel> FundingQueue { get; set; }
```

Property Value

List <GetEnvelopeFundingDTOModel>

Class GetInvestmentAccountDTOModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

DTO model to get a specific investment account

```
[ExportTsInterface]
public class GetInvestmentAccountDTOModel : GetAccountBaseModel
```

Inheritance

object ← GetAccountBaseModel ← GetInvestmentAccountDTOModel

Inherited Members

GetAccountBaseModel.AccountId , GetAccountBaseModel.BudgetId , GetAccountBaseModel.UserId , GetAccountBaseModel.AccountType , GetAccountBaseModel.AccountName , GetAccountBaseModel.Institution , GetAccountBaseModel.Balance , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetInvestmentAccountDTOModel()

Default constructor for the get investment account DTO

```
public GetInvestmentAccountDTOModel()
```

Properties

AccountNumber

```
public int? AccountNumber { get; set; }
```

Property Value

int?

InvestmentAccountId

```
public int InvestmentAccountId { get; set; }
```

Property Value

int

InvestmentAccountType

```
public InvestmentAccountType InvestmentAccountType { get; set; }
```

Property Value

InvestmentAccountType

IsTaxDeferred

```
public bool IsTaxDeferred { get; set; }
```

Property Value

bool

IsTaxExempt

```
public bool IsTaxExempt { get; set; }
```

Property Value

bool

Transactions

```
public List<SummaryTransactionDTOModel> Transactions { get; set; }
```

Property Value

List <SummaryTransactionDTOModel>

Class GetInvestmentAccountDTOResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Response model for getting a specific investment account

```
[ExportTsInterface]
public class GetInvestmentAccountDTOResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← GetInvestmentAccountDTOResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetInvestmentAccountDTOResponse(int, string)

```
public GetInvestmentAccountDTOResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

`message` string

Properties

InvestmentAccount

```
public GetInvestmentAccountDTOModel InvestmentAccount { get; set; }
```

Property Value

GetInvestmentAccountDTOModel

Class GetInvestmentAccountEntityModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Entity model to get a specific investment account

```
public class GetInvestmentAccountEntityModel : GetAccountBaseModel
```

Inheritance

object ← GetAccountBaseModel ← GetInvestmentAccountEntityModel

Inherited Members

GetAccountBaseModel.AccountId , GetAccountBaseModel.BudgetId , GetAccountBaseModel.UserId , GetAccountBaseModel.AccountType , GetAccountBaseModel.AccountName , GetAccountBaseModel.Institution , GetAccountBaseModel.Balance , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

AccountNumber

```
public int? AccountNumber { get; set; }
```

Property Value

int?

InvestmentAccountId

```
public int InvestmentAccountId { get; set; }
```

Property Value

int

InvestmentAccountType

```
public InvestmentAccountType InvestmentAccountType { get; set; }
```

Property Value

InvestmentAccountType

IsTaxDeferred

```
public bool IsTaxDeferred { get; set; }
```

Property Value

bool

IsTaxExempt

```
public bool IsTaxExempt { get; set; }
```

Property Value

bool

Transactions

```
public List<SummaryTransactionEntityModel> Transactions { get; set; }
```

Property Value

List<SummaryTransactionEntityModel>

Class GetInvestmentAccountEntityResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Response model for getting a specific investment account

```
public class GetInvestmentAccountEntityResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← GetInvestmentAccountEntityResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetInvestmentAccountEntityResponse()

Default constructor

```
public GetInvestmentAccountEntityResponse()
```

GetInvestmentAccountEntityResponse(int, string)

Parameterized constructor that takes a status and a message

```
public GetInvestmentAccountEntityResponse(int status, string message)
```

Parameters

`status` int

`message` string

Properties

InvestmentAccount

```
public GetInvestmentAccountEntityModel InvestmentAccount { get; set; }
```

Property Value

GetInvestmentAccountEntityModel

Class GetPaycheckDTOModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsInterface]
public class GetPaycheckDTOModel
```

Inheritance

object ← GetPaycheckDTOModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetId

```
public int BudgetId { get; set; }
```

Property Value

int

PaycheckId

```
public int PaycheckId { get; set; }
```

Property Value

int

PaycheckName

```
public string PaycheckName { get; set; }
```

Property Value

string

PaycheckRegularity

```
public PaycheckRegularity PaycheckRegularity { get; set; }
```

Property Value

PaycheckRegularity

SecondaryDate

```
public DateOnly? SecondaryDate { get; set; }
```

Property Value

DateOnly?

StartingDate

```
public DateOnly StartingDate { get; set; }
```

Property Value

DateOnly

TotalAmount

```
public decimal TotalAmount { get; set; }
```

Property Value

decimal

Class GetPaycheckResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsInterface]
public class GetPaycheckResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← GetPaycheckResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetPaycheckResponse()

```
public GetPaycheckResponse()
```

GetPaycheckResponse(int, string)

```
public GetPaycheckResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

`message` string

Properties

Paycheck

```
public GetPaycheckDTOModel Paycheck { get; set; }
```

Property Value

GetPaycheckDTOModel

Class GetPaymentTransactionDTOModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsInterface]
public class GetPaymentTransactionDTOModel : GetTransactionDTOModel
```

Inheritance

object ← GetTransactionDTOModel ← GetPaymentTransactionDTOModel

Inherited Members

GetTransactionDTOModel.TransactionId , GetTransactionDTOModel.BudgetMonthId , GetTransactionDTOModel.TransactionDate , GetTransactionDTOModel.TotalAmount , GetTransactionDTOModel.IsReconciled , GetTransactionDTOModel.Notes , GetTransactionDTOModel.TransactionType , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

AccountId

```
public int AccountId { get; set; }
```

Property Value

int

CheckNumber

```
public int? CheckNumber { get; set; }
```

Property Value

int?

MerchantSourceName

```
public string MerchantSourceName { get; set; }
```

Property Value

string

PaymentTransactionId

```
public int PaymentTransactionId { get; set; }
```

Property Value

int

PaymentTransactionType

```
public PaymentTransactionType PaymentTransactionType { get; set; }
```

Property Value

PaymentTransactionType

TransactionSplits

```
public List<GetTransactionSplitDTOModel> TransactionSplits { get; set; }
```

Property Value

List<GetTransactionSplitDTOModel>

Class GetPaymentTransactionResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsInterface]
public class GetPaymentTransactionResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← GetPaymentTransactionResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetPaymentTransactionResponse()

```
public GetPaymentTransactionResponse()
```

GetPaymentTransactionResponse(int, string)

```
public GetPaymentTransactionResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

`message` string

Properties

PaymentTransaction

```
public GetPaymentTransactionDTOModel PaymentTransaction { get; set; }
```

Property Value

GetPaymentTransactionDTOModel

Class GetRolloverEnvelopeBaseModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Base fields returned for a rollover envelope in the context of one budget month. RemainingAmount/RolloverEnvelopeMonthId are null if not yet activated for that month. Planned funding for the month lives on envelope_funding, not here.

```
[ExportTsInterface]
public class GetRolloverEnvelopeBaseModel
```

Inheritance

object ← GetRolloverEnvelopeBaseModel

Derived

GetRolloverEnvelopeDTOModel, GetRolloverEnvelopeEntityModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

EnvelopeCategoryId

```
public int EnvelopeCategoryId { get; set; }
```

Property Value

int

EnvelopeName

```
public string EnvelopeName { get; set; }
```

Property Value

string

RemainingAmount

```
public decimal? RemainingAmount { get; set; }
```

Property Value

decimal?

RolloverEnvelopeId

```
public int RolloverEnvelopeId { get; set; }
```

Property Value

int

RolloverEnvelopeMonthId

```
public int? RolloverEnvelopeMonthId { get; set; }
```

Property Value

int?

TransferEnvelopeId

```
public int TransferEnvelopeId { get; set; }
```

Property Value

int

Class GetRolloverEnvelopeDTOModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

DTO model to get a specific rollover envelope for a given month

```
[ExportTsInterface]
public class GetRolloverEnvelopeDTOModel : GetRolloverEnvelopeBaseModel
```

Inheritance

object ← GetRolloverEnvelopeBaseModel ← GetRolloverEnvelopeDTOModel

Inherited Members

GetRolloverEnvelopeBaseModel.RolloverEnvelopeId , GetRolloverEnvelopeBaseModel.EnvelopeCategoryId , GetRolloverEnvelopeBaseModel.EnvelopeName , GetRolloverEnvelopeBaseModel.TransferEnvelopeId , GetRolloverEnvelopeBaseModel.RolloverEnvelopeMonthId , GetRolloverEnvelopeBaseModel.RemainingAmount , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Transactions

```
public List<SummaryTransactionDTOModel> Transactions { get; set; }
```

Property Value

List <SummaryTransactionDTOModel>

Class GetRolloverEnvelopeDTOResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Response model for getting a specific rollover envelope

```
[ExportTsInterface]
public class GetRolloverEnvelopeDTOResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← GetRolloverEnvelopeDTOResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetRolloverEnvelopeDTOResponse()

```
public GetRolloverEnvelopeDTOResponse()
```

GetRolloverEnvelopeDTOResponse(int, string)

```
public GetRolloverEnvelopeDTOResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

`message` string

Properties

RolloverEnvelopeDTO

```
public GetRolloverEnvelopeDTOModel RolloverEnvelopeDTO { get; set; }
```

Property Value

GetRolloverEnvelopeDTOModel

Class GetRolloverEnvelopeEntityModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Entity model for getting a specific rollover envelope

```
public class GetRolloverEnvelopeEntityModel : GetRolloverEnvelopeBaseModel
```

Inheritance

object ← GetRolloverEnvelopeBaseModel ← GetRolloverEnvelopeEntityModel

Inherited Members

GetRolloverEnvelopeBaseModel.RolloverEnvelopeId , GetRolloverEnvelopeBaseModel.EnvelopeCategoryId , GetRolloverEnvelopeBaseModel.EnvelopeName , GetRolloverEnvelopeBaseModel.TransferEnvelopeId , GetRolloverEnvelopeBaseModel.RolloverEnvelopeMonthId , GetRolloverEnvelopeBaseModel.RemainingAmount , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Transactions

```
public List<SummaryTransactionEntityModel> Transactions { get; set; }
```

Property Value

List<SummaryTransactionEntityModel>

Class GetRolloverEnvelopeEntityResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Entity response model for getting a specific rollover envelope

```
public class GetRolloverEnvelopeEntityResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← GetRolloverEnvelopeEntityResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetRolloverEnvelopeEntityResponse()

Default constructor

```
public GetRolloverEnvelopeEntityResponse()
```

GetRolloverEnvelopeEntityResponse(int, string)

Parameterized constructor that takes a status and a message

```
public GetRolloverEnvelopeEntityResponse(int status, string message)
```

Parameters

`status` int

`message` string

Properties

RolloverEnvelope

```
public GetRolloverEnvelopeEntityModel RolloverEnvelope { get; set; }
```

Property Value

GetRolloverEnvelopeEntityModel

Class GetRolloverEnvelopeMonthRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Request used to look up a rollover envelope in the context of one specific budget month. Built server-side from the route/JWT, same as BaseIdRequest - not sent by the client directly.

```
public class GetRolloverEnvelopeMonthRequest
```

Inheritance

object ← GetRolloverEnvelopeMonthRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetRolloverEnvelopeMonthRequest()

```
public GetRolloverEnvelopeMonthRequest()
```

GetRolloverEnvelopeMonthRequest(int, int, int)

```
public GetRolloverEnvelopeMonthRequest(int userId, int rolloverEnvelopeId,
int budgetMonthId)
```

Parameters

`userId` int

`rolloverEnvelopeId` int

`budgetMonthId` int

Properties

BudgetMonthId

```
public int BudgetMonthId { get; set; }
```

Property Value

int

RolloverEnvelopeId

```
public int RolloverEnvelopeId { get; set; }
```

Property Value

int

UserId

```
public int UserId { get; set; }
```

Property Value

int

Class GetSinkingFundDependentsResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Response model for the rollover envelopes that use a sinking fund as their transfer target

```
[ExportTsInterface]
public class GetSinkingFundDependentsResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← GetSinkingFundDependentsResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetSinkingFundDependentsResponse()

```
public GetSinkingFundDependentsResponse()
```

GetSinkingFundDependentsResponse(int, string)

```
public GetSinkingFundDependentsResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

`message` string

Properties

RolloverEnvelopes

```
public List<RolloverEnvelopeDependentModel> RolloverEnvelopes { get; set; }
```

Property Value

List <RolloverEnvelopeDependentModel>

Class GetSinkingFundEnvelopeBaseModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Base fields returned for a sinking fund envelope

```
[ExportTsInterface]
public class GetSinkingFundEnvelopeBaseModel
```

Inheritance

object ← GetSinkingFundEnvelopeBaseModel

Derived

GetSinkingFundEnvelopeDTOModel, GetSinkingFundEnvelopeEntityModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

EnvelopeCategoryId

```
public int EnvelopeCategoryId { get; set; }
```

Property Value

int

EnvelopeName

```
public string EnvelopeName { get; set; }
```

Property Value

string

GoalAmount

```
public decimal? GoalAmount { get; set; }
```

Property Value

decimal?

GoalEndDate

```
public DateOnly? GoalEndDate { get; set; }
```

Property Value

DateOnly?

RemainingAmount

```
public decimal RemainingAmount { get; set; }
```

Property Value

decimal

SinkingFundEnvelopeId

```
public int SinkingFundEnvelopeId { get; set; }
```

Property Value

int

Class GetSinkingFundEnvelopeDTOModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

DTO model to get a specific sinking fund envelope

```
[ExportTsInterface]
public class GetSinkingFundEnvelopeDTOModel : GetSinkingFundEnvelopeBaseModel
```

Inheritance

object ← GetSinkingFundEnvelopeBaseModel ← GetSinkingFundEnvelopeDTOModel

Inherited Members

GetSinkingFundEnvelopeBaseModel.SinkingFundEnvelopeId , GetSinkingFundEnvelopeBaseModel.EnvelopeCategoryId , GetSinkingFundEnvelopeBaseModel.EnvelopeName , GetSinkingFundEnvelopeBaseModel.RemainingAmount , GetSinkingFundEnvelopeBaseModel.GoalAmount , GetSinkingFundEnvelopeBaseModel.GoalEndDate , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Transactions

```
public List<SummaryTransactionDTOModel> Transactions { get; set; }
```

Property Value

List <SummaryTransactionDTOModel>

Class GetSinkingFundEnvelopeDTOResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Response model for getting a specific sinking fund envelope

```
[ExportTsInterface]
public class GetSinkingFundEnvelopeDTOResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← GetSinkingFundEnvelopeDTOResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetSinkingFundEnvelopeDTOResponse()

```
public GetSinkingFundEnvelopeDTOResponse()
```

GetSinkingFundEnvelopeDTOResponse(int, string)

```
public GetSinkingFundEnvelopeDTOResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

`message` string

Properties

SinkingFundEnvelopeDTO

```
public GetSinkingFundEnvelopeDTOModel SinkingFundEnvelopeDTO { get; set; }
```

Property Value

GetSinkingFundEnvelopeDTOModel

Class GetSinkingFundEnvelopeEntityModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Entity model for getting a specific sinking fund envelope

```
public class GetSinkingFundEnvelopeEntityModel : GetSinkingFundEnvelopeBaseModel
```

Inheritance

object ← GetSinkingFundEnvelopeBaseModel ← GetSinkingFundEnvelopeEntityModel

Inherited Members

GetSinkingFundEnvelopeBaseModel.SinkingFundEnvelopeId ,
GetSinkingFundEnvelopeBaseModel.EnvelopeCategoryId ,
GetSinkingFundEnvelopeBaseModel.EnvelopeName ,
GetSinkingFundEnvelopeBaseModel.RemainingAmount ,
GetSinkingFundEnvelopeBaseModel.GoalAmount , GetSinkingFundEnvelopeBaseModel.GoalEndDate ,
object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() ,
object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Transactions

```
public List<SummaryTransactionEntityModel> Transactions { get; set; }
```

Property Value

List<SummaryTransactionEntityModel>

Class GetSinkingFundEnvelopeEntityResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Entity response model for getting a specific sinking fund envelope

```
public class GetSinkingFundEnvelopeEntityResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← GetSinkingFundEnvelopeEntityResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetSinkingFundEnvelopeEntityResponse()

Default constructor

```
public GetSinkingFundEnvelopeEntityResponse()
```

GetSinkingFundEnvelopeEntityResponse(int, string)

Parameterized constructor that takes a status and a message

```
public GetSinkingFundEnvelopeEntityResponse(int status, string message)
```

Parameters

`status` int

`message` string

Properties

SinkingFundEnvelope

```
public GetSinkingFundEnvelopeEntityModel SinkingFundEnvelope { get; set; }
```

Property Value

GetSinkingFundEnvelopeEntityModel

Class GetTransactionDTOModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsInterface]
public class GetTransactionDTOModel
```

Inheritance

object ← GetTransactionDTOModel

Derived

GetPaymentTransactionDTOModel, GetTransferTransactionDTOModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetMonthId

```
public int BudgetMonthId { get; set; }
```

Property Value

int

IsReconciled

```
public bool IsReconciled { get; set; }
```

Property Value

bool

Notes

```
public string? Notes { get; set; }
```

Property Value

string

TotalAmount

```
public decimal TotalAmount { get; set; }
```

Property Value

decimal

TransactionDate

```
public DateOnly TransactionDate { get; set; }
```

Property Value

DateOnly

TransactionId

```
public int TransactionId { get; set; }
```

Property Value

int

TransactionType

```
public TransactionType TransactionType { get; set; }
```

Property Value

TransactionType

Class GetTransactionEntityModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
public class GetTransactionEntityModel
```

Inheritance

object ← GetTransactionEntityModel

Derived

GetTransferTransactionEntityModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetMonthId

```
public int BudgetMonthId { get; set; }
```

Property Value

int

IsReconciled

```
public bool IsReconciled { get; set; }
```

Property Value

bool

Notes

```
public string? Notes { get; set; }
```

Property Value

string

TotalAmount

```
public decimal TotalAmount { get; set; }
```

Property Value

decimal

TransactionDate

```
public DateOnly TransactionDate { get; set; }
```

Property Value

DateOnly

TransactionId

```
public int TransactionId { get; set; }
```

Property Value

int

TransactionType

```
public TransactionType TransactionType { get; set; }
```

Property Value

TransactionType

UserId

```
public int UserId { get; set; }
```

Property Value

int

Class GetTransactionSplitDTOModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsInterface]
public class GetTransactionSplitDTOModel
```

Inheritance

object ← GetTransactionSplitDTOModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Amount

```
public decimal Amount { get; set; }
```

Property Value

decimal

EnvelopeId

```
public int EnvelopeId { get; set; }
```

Property Value

int

IsSinkingFund

```
public bool IsSinkingFund { get; set; }
```

Property Value

bool

TransactionId

```
public int TransactionId { get; set; }
```

Property Value

int

TransactionSplitId

```
public int TransactionSplitId { get; set; }
```

Property Value

int

Class GetTransferTransactionDTOModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsInterface]
public class GetTransferTransactionDTOModel : GetTransactionDTOModel
```

Inheritance

object ← GetTransactionDTOModel ← GetTransferTransactionDTOModel

Inherited Members

GetTransactionDTOModel.TransactionId , GetTransactionDTOModel.BudgetMonthId , GetTransactionDTOModel.TransactionDate , GetTransactionDTOModel.TotalAmount , GetTransactionDTOModel.IsReconciled , GetTransactionDTOModel.Notes , GetTransactionDTOModel.TransactionType , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

TransferFromId

```
public int? TransferFromId { get; set; }
```

Property Value

int?

TransferFromIsSinkingFund

```
public bool? TransferFromIsSinkingFund { get; set; }
```

Property Value

bool?

TransferToId

```
public int? TransferToId { get; set; }
```

Property Value

int?

TransferToIsSinkingFund

```
public bool? TransferToIsSinkingFund { get; set; }
```

Property Value

bool?

TransferTransactionId

```
public int TransferTransactionId { get; set; }
```

Property Value

int

TransferTransactionType

```
public TransferTransactionType TransferTransactionType { get; set; }
```

Property Value

TransferTransactionType

Class GetTransferTransactionDTOResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsInterface]
public class GetTransferTransactionDTOResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← GetTransferTransactionDTOResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetTransferTransactionDTOResponse()

```
public GetTransferTransactionDTOResponse()
```

GetTransferTransactionDTOResponse(int, string)

```
public GetTransferTransactionDTOResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

`message` string

Properties

TransferTransaction

```
public GetTransferTransactionDTOModel TransferTransaction { get; set; }
```

Property Value

GetTransferTransactionDTOModel

Class GetTransferTransactionEntityModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Entity model for a specific transfer transaction. TransferFromEnvelopeId/ToEnvelopeId and their IsSinkingFund flags are only populated (via a follow-up transaction_split lookup) when TransferTransactionType is Envelope - they aren't columns on transfer_transaction itself.

```
public class GetTransferTransactionEntityModel : GetTransactionEntityModel
```

Inheritance

object ← GetTransactionEntityModel ← GetTransferTransactionEntityModel

Inherited Members

GetTransactionEntityModel.TransactionId , GetTransactionEntityModel.BudgetMonthId , GetTransactionEntityModel.UserId , GetTransactionEntityModel.TransactionDate , GetTransactionEntityModel.TotalAmount , GetTransactionEntityModel.IsReconciled , GetTransactionEntityModel.Notes , GetTransactionEntityModel.TransactionType , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

TransferFromAccountId

```
public int? TransferFromAccountId { get; set; }
```

Property Value

int?

TransferFromEnvelopeId

```
public int? TransferFromEnvelopeId { get; set; }
```

Property Value

int?

TransferFromIsSinkingFund

```
public bool? TransferFromIsSinkingFund { get; set; }
```

Property Value

bool?

TransferToAccountId

```
public int? TransferToAccountId { get; set; }
```

Property Value

int?

TransferToEnvelopeId

```
public int? TransferToEnvelopeId { get; set; }
```

Property Value

int?

TransferToIsSinkingFund

```
public bool? TransferToIsSinkingFund { get; set; }
```

Property Value

bool?

TransferTransactionId

```
public int TransferTransactionId { get; set; }
```

Property Value

int

TransferTransactionType

```
public TransferTransactionType TransferTransactionType { get; set; }
```

Property Value

TransferTransactionType

Class GetTransferTransactionEntityResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
public class GetTransferTransactionEntityResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← GetTransferTransactionEntityResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetTransferTransactionEntityResponse()

```
public GetTransferTransactionEntityResponse()
```

GetTransferTransactionEntityResponse(int, string)

```
public GetTransferTransactionEntityResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

`message` string

Properties

TransferTransaction

```
public GetTransferTransactionEntityModel TransferTransaction { get; set; }
```

Property Value

GetTransferTransactionEntityModel

Class GetUserBudgetsResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Response model for every budget a user has access to

```
[ExportTsInterface]
public class GetUserBudgetsResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← GetUserBudgetsResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetUserBudgetsResponse()

```
public GetUserBudgetsResponse()
```

GetUserBudgetsResponse(int, string)

```
public GetUserBudgetsResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

`message` string

Properties

Budgets

```
public List<SummaryBudgetResponse> Budgets { get; set; }
```

Property Value

List <SummaryBudgetResponse>

Class GetUserDTOModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

DTO model for getting a user

```
[ExportTsInterface]
public class GetUserDTOModel
```

Inheritance

object ← GetUserDTOModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Email

```
public string Email { get; set; }
```

Property Value

string

IsDarkMode

```
public bool IsDarkMode { get; set; }
```

Property Value

bool

UserId

```
public int UserId { get; set; }
```

Property Value

int

Username

```
public string Username { get; set; }
```

Property Value

string

Class GetUserResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsInterface]
public class GetUserResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← GetUserResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetUserResponse(int, string)

Parameterized constructor that takes a status and message

```
public GetUserResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

`message` string

GetUserResponse(int, string, GetUserDTOModel)

Parameterized constructor that takes a status, message, and GetUserDTO object

```
public GetUserResponse(int httpStatus, string message, GetUserDTOModel getUserDTO)
```

Parameters

`httpStatus` int

`message` string

`getUserDTO` GetUserDTOModel

Properties

GetUserDTO

```
public GetUserDTOModel GetUserDTO { get; set; }
```

Property Value

GetUserDTOModel

Class GetYearBudgetDTOModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsClass]
public class GetYearBudgetDTOModel
```

Inheritance

object ← GetYearBudgetDTOModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetId

```
public int BudgetId { get; set; }
```

Property Value

int

BudgetMonthId

```
public int BudgetMonthId { get; set; }
```

Property Value

int

BudgetName

```
public string BudgetName { get; set; }
```

Property Value

string

BudgetTemplateId

```
public int? BudgetTemplateId { get; set; }
```

Property Value

int?

Month

```
public Month Month { get; set; }
```

Property Value

Month

UserId

```
public int UserId { get; set; }
```

Property Value

int

Year

```
public int Year { get; set; }
```

Property Value

int

Class GetYearBudgetsResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsInterface]
public class GetYearBudgetsResponse : BaseIdResponse
```

Inheritance

object ← BaseResponse ← BaseIdResponse ← GetYearBudgetsResponse

Inherited Members

BaseIdResponse.Id , BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GetYearBudgetsResponse()

```
public GetYearBudgetsResponse()
```

GetYearBudgetsResponse(int, string)

```
public GetYearBudgetsResponse(int status, string message)
```

Parameters

`status` int

`message` string

Properties

Budgets

```
public List<GetYearBudgetDTOModel> Budgets { get; set; }
```

Property Value

List <GetYearBudgetDTOModel>

Interface IOptional

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsClass]
public interface IOptional
```

Properties

HasValue

```
bool HasValue { get; }
```

Property Value

bool

Methods

GetValue()

```
object? GetValue()
```

Returns

object

Class InvestmentAccountEntityModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Model for an investment account

```
public class InvestmentAccountEntityModel : AccountEntityModel
```

Inheritance

object ← AccountEntityModel ← InvestmentAccountEntityModel

Inherited Members

AccountEntityModel.AccountId , AccountEntityModel.BudgetId , AccountEntityModel.AccountType , AccountEntityModel.AccountName , AccountEntityModel.Institution , AccountEntityModel.Balance , AccountEntityModel.CreatedAt , AccountEntityModel.LastUpdatedAt , AccountEntityModel.Transactions , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

InvestmentAccountId

```
public int InvestmentAccountId { get; set; }
```

Property Value

int

InvestmentAccountNumber

```
public string? InvestmentAccountNumber { get; set; }
```

Property Value

string

InvestmentAccountType

```
public InvestmentAccountType InvestmentAccountType { get; set; }
```

Property Value

InvestmentAccountType

IsTaxDeferred

```
public bool IsTaxDeferred { get; set; }
```

Property Value

bool

IsTaxExempt

```
public bool IsTaxExempt { get; set; }
```

Property Value

bool

Class InvestmentAccountSummaryDTOModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsInterface]
public class InvestmentAccountSummaryDTOModel : SummaryAccountDTOModel
```

Inheritance

object ← SummaryAccountDTOModel ← InvestmentAccountSummaryDTOModel

Inherited Members

SummaryAccountDTOModel.AccountId , SummaryAccountDTOModel.BudgetId , SummaryAccountDTOModel.AccountType , SummaryAccountDTOModel.AccountName , SummaryAccountDTOModel.Balance , SummaryAccountDTOModel.Institution , SummaryAccountDTOModel.BankAccountType , SummaryAccountDTOModel.DebtAccountType , SummaryAccountDTOModel.InvestmentAccountType , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

InvestmentAccountId

```
public int InvestmentAccountId { get; set; }
```

Property Value

int

Class LoginRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Request model for a user

```
[ExportTsClass]
public class LoginRequest
```

Inheritance

object ← LoginRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Password

```
public string Password { get; set; }
```

Property Value

string

UsernameOrEmail

```
public string UsernameOrEmail { get; set; }
```

Property Value

string

Class LoginResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Response model for the login process

```
[ExportTsInterface]
public class LoginResponse : BaseIdResponse
```

Inheritance

object ← BaseResponse ← BaseIdResponse ← LoginResponse

Inherited Members

BaseIdResponse.Id , BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

LoginResponse()

```
public LoginResponse()
```

LoginResponse(int, string)

Parameterized constructor for status and message

```
public LoginResponse(int status, string message)
```

Parameters

`status` int

`message` string

LoginResponse(int, string, int)

Parameterized constructor for status, message, and user id

```
public LoginResponse(int status, string message, int userId)
```

Parameters

`status` int

`message` string

`userId` int

Properties

Token

```
public string Token { get; set; }
```

Property Value

string

Struct Optional<T>

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Optional class for creating and updating nulls

```
[ExportTsClass]
public readonly struct Optional<T> : IOptional
```

Type Parameters

T

Implements
IOptional

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Constructors

Optional(bool)

Parameterized constructor to bring in a HasValue value

```
public Optional(bool hasValue)
```

Parameters

hasValue bool

Optional(bool, T?)

Parameterized constructor to bring in a HasValue and Value values

```
public Optional(bool hasValue, T? value)
```

Parameters

`hasValue` bool

`value` T

Optional(T?)

Parameterized constructor to set Value

```
public Optional(T? value)
```

Parameters

`value` T

Properties

HasValue

Checks if the value was sent

```
public bool HasValue { get; }
```

Property Value

bool

Value

The value that was sent

```
[TsType("T | null", null, null, false)]
```

```
public T? Value { get; }
```

Property Value

T

Methods

GetValue()

```
public object? GetValue()
```

Returns

object

Operators

implicit operator Optional<T>(T?)

```
public static implicit operator Optional<T>(T? value)
```

Parameters

value T

Returns

Optional<T>

Class PaycheckEntityModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Model for a paycheck. Forecast-only - a paycheck no longer funds envelopes directly (see envelope_funding for that) and isn't tied to any specific month. It exists purely to represent expected income on a recurring schedule.

```
public class PaycheckEntityModel
```

Inheritance

object ← PaycheckEntityModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetId

```
public int BudgetId { get; set; }
```

Property Value

int

CreatedAt

```
public DateTime CreatedAt { get; set; }
```

Property Value

DateTime

LastUpdatedAt

```
public DateTime LastUpdatedAt { get; set; }
```

Property Value

DateTime

PaycheckId

```
public int PaycheckId { get; set; }
```

Property Value

int

PaycheckName

```
public string PaycheckName { get; set; }
```

Property Value

string

PaycheckRegularity

```
public PaycheckRegularity PaycheckRegularity { get; set; }
```

Property Value

PaycheckRegularity

SecondaryDate

```
public DateOnly? SecondaryDate { get; set; }
```

Property Value

DateOnly?

StartingDate

```
public DateOnly StartingDate { get; set; }
```

Property Value

DateOnly

TotalAmount

```
public decimal TotalAmount { get; set; }
```

Property Value

decimal

Class PaymentTransactionEntityModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Model for a payment transaction

```
public class PaymentTransactionEntityModel : TransactionEntityModel
```

Inheritance

object ← TransactionEntityModel ← PaymentTransactionEntityModel

Inherited Members

TransactionEntityModel.TransactionId , TransactionEntityModel.BudgetMonthId , TransactionEntityModel.TransactionDate , TransactionEntityModel.TotalAmount , TransactionEntityModel.IsReconciled , TransactionEntityModel.Notes , TransactionEntityModel.TransactionType , TransactionEntityModel.CreatedAt , TransactionEntityModel.LastUpdatedAt , TransactionEntityModel.DeletedAt , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

AccountId

```
public int AccountId { get; set; }
```

Property Value

int

CheckNumber

```
public int? CheckNumber { get; set; }
```

Property Value

int?

MerchantSourceName

```
public string MerchantSourceName { get; set; }
```

Property Value

string

PaymentTransactionId

```
public int PaymentTransactionId { get; set; }
```

Property Value

int

PaymentTransactionType

```
public PaymentTransactionType PaymentTransactionType { get; set; }
```

Property Value

PaymentTransactionType

TransactionSplits

```
public List<TransactionSplitEntityModel> TransactionSplits { get; set; }
```

Property Value

List <TransactionSplitEntityModel>

Class PaymentTransactionSnapshotModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Internal helper: the current account_id/total_amount/budget_month_id for a payment transaction, read before an update is applied so the old account-balance effect can be reversed and the new one applied cleanly.

```
public class PaymentTransactionSnapshotModel
```

Inheritance

object ← PaymentTransactionSnapshotModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

AccountId

```
public int AccountId { get; set; }
```

Property Value

int

BudgetMonthId

```
public int BudgetMonthId { get; set; }
```

Property Value

int

IsDeleted

```
public bool IsDeleted { get; set; }
```

Property Value

bool

TotalAmount

```
public decimal TotalAmount { get; set; }
```

Property Value

decimal

Class RegisterRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsClass]
public class RegisterRequest
```

Inheritance

object ← RegisterRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Email

```
public string Email { get; set; }
```

Property Value

string

PasswordHash

```
public string PasswordHash { get; set; }
```

Property Value

string

Username

```
public string Username { get; set; }
```

Property Value

string

Class RegisterResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Response model for registration

```
[ExportTsInterface]
public class RegisterResponse : BaseIdResponse
```

Inheritance

object ← BaseResponse ← BaseIdResponse ← RegisterResponse

Inherited Members

BaseIdResponse.Id , BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

RegisterResponse(int, string)

Parameterized constructor for the register response model

```
public RegisterResponse(int status, string message)
```

Parameters

`status` int

`message` string

RegisterResponse(int, string, int)

Parameterized constructor for the register response model

```
public RegisterResponse(int status, string message, int userId)
```

Parameters

`status` int

`message` string

`userId` int

Properties

Token

```
public string? Token { get; set; }
```

Property Value

string

Class RolloverEnvelopeDependentModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

A rollover envelope that sweeps into a given sinking fund envelope

```
[ExportTsInterface]
public class RolloverEnvelopeDependentModel
```

Inheritance

object ← RolloverEnvelopeDependentModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

EnvelopeName

```
public string EnvelopeName { get; set; }
```

Property Value

string

RolloverEnvelopeId

```
public int RolloverEnvelopeId { get; set; }
```

Property Value

int

Class RolloverEnvelopeEntityModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Model for a rollover envelope's persistent definition. No dollar amounts live here - a rollover envelope isn't guaranteed to be active every month, so amounts live on the per-month instance (see RolloverEnvelopeMonthEntityModel).

```
public class RolloverEnvelopeEntityModel
```

Inheritance

object ← RolloverEnvelopeEntityModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

CreatedAt

```
public DateTime CreatedAt { get; set; }
```

Property Value

DateTime

EnvelopeCategoryId

```
public int EnvelopeCategoryId { get; set; }
```

Property Value

int

EnvelopeName

```
public string EnvelopeName { get; set; }
```

Property Value

string

LastUpdatedAt

```
public DateTime LastUpdatedAt { get; set; }
```

Property Value

DateTime

RolloverEnvelopeId

```
public int RolloverEnvelopeId { get; set; }
```

Property Value

int

TransferEnvelopeId

```
public int TransferEnvelopeId { get; set; }
```

Property Value

int

Class RolloverEnvelopeMonthEntityModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Model for a single month's instance of a rollover envelope. Absence of a row for a given (RolloverEnvelopeId, BudgetMonthId) pair means the envelope was inactive that month.

```
public class RolloverEnvelopeMonthEntityModel
```

Inheritance

object ← RolloverEnvelopeMonthEntityModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetMonthId

```
public int BudgetMonthId { get; set; }
```

Property Value

int

CreatedAt

```
public DateTime CreatedAt { get; set; }
```

Property Value

DateTime

LastUpdatedAt

```
public DateTime LastUpdatedAt { get; set; }
```

Property Value

DateTime

RemainingAmount

```
public decimal RemainingAmount { get; set; }
```

Property Value

decimal

RolloverEnvelopeId

```
public int RolloverEnvelopeId { get; set; }
```

Property Value

int

RolloverEnvelopeMonthId

```
public int RolloverEnvelopeMonthId { get; set; }
```

Property Value

int

Class SetBudgetTemplateEnvelopeItemRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Request model for a single item within a template's envelope list being set. BudgetTemplateEnvelopeId is null for a brand new entry, or the existing entry's id to update it in place.

```
[ExportTsClass]
public class SetBudgetTemplateEnvelopeItemRequest
```

Inheritance

object ← SetBudgetTemplateEnvelopeItemRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetTemplateEnvelopeId

```
public int? BudgetTemplateEnvelopeId { get; set; }
```

Property Value

int?

EnvelopeId

```
public int EnvelopeId { get; set; }
```

Property Value

int

IsSinkingFund

```
public bool IsSinkingFund { get; set; }
```

Property Value

bool

PlannedAmount

```
public decimal PlannedAmount { get; set; }
```

Property Value

decimal

Class SetBudgetTemplateEnvelopesRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Request model to set (create/reorder/update/remove all at once) a template's envelope entries. The list's order is the funding priority for whenever this template is applied.

```
[ExportTsClass]
public class SetBudgetTemplateEnvelopesRequest
```

Inheritance

object ← SetBudgetTemplateEnvelopesRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetTemplateId

```
public int BudgetTemplateId { get; set; }
```

Property Value

int

Envelopes

```
public List<SetBudgetTemplateEnvelopeItemRequest> Envelopes { get; set; }
```

Property Value

List<SetBudgetTemplateEnvelopeItemRequest>

UserId

```
public int UserId { get; set; }
```

Property Value

int

Class SetEnvelopeFundingItemRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Request model for a single item within a funding queue being set. EnvelopeFundingId is null for a brand new row, or the existing row's id to update it in place.

```
[ExportTsClass]
public class SetEnvelopeFundingItemRequest
```

Inheritance

object ← SetEnvelopeFundingItemRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

EnvelopeFundingId

```
public int? EnvelopeFundingId { get; set; }
```

Property Value

int?

EnvelopeId

```
public int EnvelopeId { get; set; }
```

Property Value

int

IsSinkingFund

```
public bool IsSinkingFund { get; set; }
```

Property Value

bool

PlannedAmount

```
public decimal PlannedAmount { get; set; }
```

Property Value

decimal

Class SetEnvelopeFundingQueueRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Request model to set (create/reorder/update/remove all at once) the funding queue for a budget month. The list's order is the funding priority - item 0 is funded first.

```
[ExportTsClass]
public class SetEnvelopeFundingQueueRequest
```

Inheritance

object ← SetEnvelopeFundingQueueRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetMonthId

```
public int BudgetMonthId { get; set; }
```

Property Value

int

FundingQueue

```
public List<SetEnvelopeFundingItemRequest> FundingQueue { get; set; }
```

Property Value

List<SetEnvelopeFundingItemRequest>

UserId

```
public int UserId { get; set; }
```

Property Value

int

Class SinkingFundEnvelopeEntityModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Model for a sinking fund envelope. Sinking fund envelopes are persistent - one row holds a live remaining_amount that carries forward month to month, unlike rollover envelopes.

```
public class SinkingFundEnvelopeEntityModel
```

Inheritance

object ← SinkingFundEnvelopeEntityModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

CreatedAt

```
public DateTime CreatedAt { get; set; }
```

Property Value

DateTime

EnvelopeCategoryId

```
public int EnvelopeCategoryId { get; set; }
```

Property Value

int

EnvelopeName

```
public string EnvelopeName { get; set; }
```

Property Value

string

GoalAmount

```
public decimal? GoalAmount { get; set; }
```

Property Value

decimal?

GoalEndDate

```
public DateOnly? GoalEndDate { get; set; }
```

Property Value

DateOnly?

LastUpdatedAt

```
public DateTime LastUpdatedAt { get; set; }
```

Property Value

DateTime

RemainingAmount

```
public decimal RemainingAmount { get; set; }
```

Property Value

decimal

SinkingFundEnvelopeId

```
public int SinkingFundEnvelopeId { get; set; }
```

Property Value

int

Transactions

```
public List<TransactionEntityModel> Transactions { get; set; }
```

Property Value

List <TransactionEntityModel>

Class SummaryAccountDTOModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsInterface]
public class SummaryAccountDTOModel
```

Inheritance

object ← SummaryAccountDTOModel

Derived

BankAccountSummaryDTOModel, DebtAccountSummaryDTOModel, InvestmentAccountSummaryDTOModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

AccountId

```
public int AccountId { get; set; }
```

Property Value

int

AccountName

```
public string AccountName { get; set; }
```

Property Value

string

AccountType

```
public AccountType AccountType { get; set; }
```

Property Value

AccountType

Balance

```
public decimal Balance { get; set; }
```

Property Value

decimal

BankAccountType

```
public BankAccountType? BankAccountType { get; set; }
```

Property Value

BankAccountType?

BudgetId

```
public int BudgetId { get; set; }
```

Property Value

int

DebtAccountType

```
public DebtAccountType? DebtAccountType { get; set; }
```

Property Value

DebtAccountType?

Institution

```
public string Institution { get; set; }
```

Property Value

string

InvestmentAccountType

```
public InvestmentAccountType? InvestmentAccountType { get; set; }
```

Property Value

InvestmentAccountType?

Class SummaryBudgetResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Response model for a single budget in a user's budget list - just enough to let them pick which one to open, before any month/id is known.

```
[ExportTsInterface]
public class SummaryBudgetResponse
```

Inheritance

object ← SummaryBudgetResponse

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetId

```
public int BudgetId { get; set; }
```

Property Value

int

BudgetName

```
public string BudgetName { get; set; }
```

Property Value

string

Class SummaryBudgetTemplateResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Response model for a single template in a budget's template list

```
[ExportTsInterface]
public class SummaryBudgetTemplateResponse
```

Inheritance

object ← SummaryBudgetTemplateResponse

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetTemplateId

```
public int BudgetTemplateId { get; set; }
```

Property Value

int

TemplateName

```
public string TemplateName { get; set; }
```

Property Value

string

Class SummaryEnvelopeCategoryResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Response model for an envelope category for the Get Budget method

```
[ExportTsInterface]
public class SummaryEnvelopeCategoryResponse
```

Inheritance

object ← SummaryEnvelopeCategoryResponse

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetId

```
public int BudgetId { get; set; }
```

Property Value

int

EnvelopeCategoryId

```
public int EnvelopeCategoryId { get; set; }
```

Property Value

int

EnvelopeCategoryName

```
public string EnvelopeCategoryName { get; set; }
```

Property Value

string

Envelopes

```
public List<SummaryEnvelopeResponse> Envelopes { get; set; }
```

Property Value

List<SummaryEnvelopeResponse>

Class SummaryEnvelopeResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Response model for a single envelope for the Get Budget method. Sinking fund and rollover envelopes come from two different tables with two different id spaces, so IsSinkingFund tells the frontend which table EnvelopeId belongs to (same pattern SummaryAccountDTOModel uses to distinguish bank/debt/investment accounts). RemainingAmount is nullable because a rollover envelope may not be activated for the month being viewed.

```
[ExportTsInterface]
public class SummaryEnvelopeResponse
```

Inheritance

object ← SummaryEnvelopeResponse

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

EnvelopeCategoryId

```
public int EnvelopeCategoryId { get; set; }
```

Property Value

int

EnvelopeId

```
public int EnvelopeId { get; set; }
```

Property Value

int

EnvelopeName

```
public string EnvelopeName { get; set; }
```

Property Value

string

IsSinkingFund

```
public bool IsSinkingFund { get; set; }
```

Property Value

bool

RemainingAmount

```
public decimal? RemainingAmount { get; set; }
```

Property Value

decimal?

Class SummaryPaycheckResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsInterface]
public class SummaryPaycheckResponse
```

Inheritance

object ← SummaryPaycheckResponse

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

PaycheckId

```
public int PaycheckId { get; set; }
```

Property Value

int

PaycheckRegularity

```
public PaycheckRegularity PaycheckRegularity { get; set; }
```

Property Value

PaycheckRegularity

SecondaryDate

```
public DateOnly? SecondaryDate { get; set; }
```

Property Value

DateOnly?

StartingDate

```
public DateOnly StartingDate { get; set; }
```

Property Value

DateOnly

TotalAmount

```
public decimal TotalAmount { get; set; }
```

Property Value

decimal

Class SummaryTransactionDTOModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsInterface]
public class SummaryTransactionDTOModel
```

Inheritance

object ← SummaryTransactionDTOModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Amount

```
public decimal Amount { get; set; }
```

Property Value

decimal

Date

```
public DateOnly Date { get; set; }
```

Property Value

DateOnly

Envelopes

```
public string Envelopes { get; set; }
```

Property Value

string

Location

```
public string Location { get; set; }
```

Property Value

string

TransactionId

```
public int TransactionId { get; set; }
```

Property Value

int

Class SummaryTransactionEntityModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
public class SummaryTransactionEntityModel
```

Inheritance

object ← SummaryTransactionEntityModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Date

```
public DateOnly Date { get; set; }
```

Property Value

DateOnly

EnvelopeNames

```
public string? EnvelopeNames { get; set; }
```

Property Value

string

MerchantSourceName

```
public string? MerchantSourceName { get; set; }
```

Property Value

string

TotalAmount

```
public decimal TotalAmount { get; set; }
```

Property Value

decimal

TransactionId

```
public int TransactionId { get; set; }
```

Property Value

int

TransactionType

```
public TransactionType TransactionType { get; set; }
```

Property Value

TransactionType

TransferFromAccount

```
public string? TransferFromAccount { get; set; }
```

Property Value

string

TransferFromEnvelope

```
public string? TransferFromEnvelope { get; set; }
```

Property Value

string

TransferToAccount

```
public string? TransferToAccount { get; set; }
```

Property Value

string

TransferToEnvelope

```
public string? TransferToEnvelope { get; set; }
```

Property Value

string

TransferTransactionType

```
public TransferTransactionType? TransferTransactionType { get; set; }
```

Property Value

TransferTransactionType?

Class TransactionEffectSnapshotModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Internal helper: what a transaction currently does to account balances, plus whether it is soft-deleted. Used by the delete/restore methods to reverse or re-apply its effect.

```
public class TransactionEffectSnapshotModel
```

Inheritance

object ← TransactionEffectSnapshotModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

IsDeleted

```
public bool IsDeleted { get; set; }
```

Property Value

bool

PaymentAccountId

```
public int? PaymentAccountId { get; set; }
```

Property Value

int?

TotalAmount

```
public decimal TotalAmount { get; set; }
```

Property Value

decimal

TransferFromAccountId

```
public int? TransferFromAccountId { get; set; }
```

Property Value

int?

TransferToAccountId

```
public int? TransferToAccountId { get; set; }
```

Property Value

int?

Class TransactionEntityModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Model for an abstract transaction

```
public abstract class TransactionEntityModel
```

Inheritance

object ← TransactionEntityModel

Derived

PaymentTransactionEntityModel, TransferTransactionEntityModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetMonthId

```
public int BudgetMonthId { get; set; }
```

Property Value

int

CreatedAt

```
public DateTime CreatedAt { get; set; }
```

Property Value

DateTime

DeletedAt

```
public DateTime? DeletedAt { get; set; }
```

Property Value

DateTime?

IsReconciled

```
public bool IsReconciled { get; set; }
```

Property Value

bool

LastUpdatedAt

```
public DateTime LastUpdatedAt { get; set; }
```

Property Value

DateTime

Notes

```
public string? Notes { get; set; }
```

Property Value

string

TotalAmount

```
public decimal TotalAmount { get; set; }
```

Property Value

decimal

TransactionDate

```
public DateOnly TransactionDate { get; set; }
```

Property Value

DateOnly

TransactionId

```
public int TransactionId { get; set; }
```

Property Value

int

TransactionType

```
public TransactionType TransactionType { get; set; }
```

Property Value

TransactionType

Class TransactionSplitEntityModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Model for a transaction split model. EnvelopeId is a display/API-level id - IsSinkingFund says whether it refers to a sinking_fund_envelope_id or the persistent rollover_envelope_id (not the month-specific rollover_envelope_month_id, which is an internal DAO detail).

```
public class TransactionSplitEntityModel
```

Inheritance

object ← TransactionSplitEntityModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Amount

```
public decimal Amount { get; set; }
```

Property Value

decimal

CreatedAt

```
public DateTime CreatedAt { get; set; }
```

Property Value

DateTime

EnvelopeId

```
public int EnvelopeId { get; set; }
```

Property Value

int

IsSinkingFund

```
public bool IsSinkingFund { get; set; }
```

Property Value

bool

LastUpdatedAt

```
public DateTime LastUpdatedAt { get; set; }
```

Property Value

DateTime

TransactionId

```
public int TransactionId { get; set; }
```

Property Value

int

TransactionSplitId

```
public int TransactionSplitId { get; set; }
```

Property Value

int

Class TransactionSplitSnapshotModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Internal helper: the raw current state of one transaction_split row, used to reverse its old effect on an envelope's remaining amount before an update applies the new one.

```
public class TransactionSplitSnapshotModel
```

Inheritance

object ← TransactionSplitSnapshotModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Amount

```
public decimal Amount { get; set; }
```

Property Value

decimal

RolloverEnvelopeMonthId

```
public int? RolloverEnvelopeMonthId { get; set; }
```

Property Value

int?

SinkingFundEnvelopeId

```
public int? SinkingFundEnvelopeId { get; set; }
```

Property Value

int?

TransactionSplitId

```
public int TransactionSplitId { get; set; }
```

Property Value

int

Class TransactionSplitTransferSideModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Internal helper for reading one side of an envelope-to-envelope transfer back out of transaction_split (used by TransferTransactionDAO/TransactionDAO, not exposed to the API).

```
public class TransactionSplitTransferSideModel
```

Inheritance

object ← TransactionSplitTransferSideModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Amount

```
public decimal Amount { get; set; }
```

Property Value

decimal

EnvelopeId

```
public int EnvelopeId { get; set; }
```

Property Value

int

IsSinkingFund

```
public bool IsSinkingFund { get; set; }
```

Property Value

bool

Class TransferTransactionEntityModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Model for a transfer transaction. v3's transfer_transaction table only carries account columns - an envelope-to-envelope transfer is a transfer_transaction row of type Envelope with null account columns, paired with two opposite-signed transaction_split rows that carry the actual envelope references.

```
public class TransferTransactionEntityModel : TransactionEntityModel
```

Inheritance

object ← TransactionEntityModel ← TransferTransactionEntityModel

Inherited Members

TransactionEntityModel.TransactionId , TransactionEntityModel.BudgetMonthId , TransactionEntityModel.TransactionDate , TransactionEntityModel.TotalAmount , TransactionEntityModel.IsReconciled , TransactionEntityModel.Notes , TransactionEntityModel.TransactionType , TransactionEntityModel.CreatedAt , TransactionEntityModel.LastUpdatedAt , TransactionEntityModel.DeletedAt , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

TransferFromAccountId

```
public int? TransferFromAccountId { get; set; }
```

Property Value

int?

TransferToAccountId

```
public int? TransferToAccountId { get; set; }
```

Property Value

int?

TransferTransactionId

```
public int TransferTransactionId { get; set; }
```

Property Value

int

TransferTransactionType

```
public TransferTransactionType TransferTransactionType { get; set; }
```

Property Value

TransferTransactionType

Class TransferTransactionSnapshotModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Internal helper: the current type/accounts/total_amount/budget_month_id for a transfer transaction, read before an update is applied so the old effect (account balances or envelope remaining amounts, depending on type) can be reversed and the new one applied.

```
public class TransferTransactionSnapshotModel
```

Inheritance

object ← TransferTransactionSnapshotModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BudgetMonthId

```
public int BudgetMonthId { get; set; }
```

Property Value

int

IsDeleted

```
public bool IsDeleted { get; set; }
```

Property Value

bool

TotalAmount

```
public decimal TotalAmount { get; set; }
```

Property Value

decimal

TransactionType

```
public TransactionType TransactionType { get; set; }
```

Property Value

TransactionType

TransferFromAccountId

```
public int? TransferFromAccountId { get; set; }
```

Property Value

int?

TransferToAccountId

```
public int? TransferToAccountId { get; set; }
```

Property Value

int?

TransferTransactionType

```
public TransferTransactionType TransferTransactionType { get; set; }
```

Property Value

TransferTransactionType

Class UpdateAccountRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsClass]
public class UpdateAccountRequest
```

Inheritance

object ← UpdateAccountRequest

Derived

UpdateBankAccountRequest, UpdateDebtAccountRequest, UpdateInvestmentAccountRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

AccountId

```
[JsonConverter(typeof(SafeDefaultConverter<int>))]
public int AccountId { get; set; }
```

Property Value

int

AccountName

```
public string? AccountName { get; set; }
```

Property Value

string

Balance

```
public decimal? Balance { get; set; }
```

Property Value

decimal?

Institution

```
public string? Institution { get; set; }
```

Property Value

string

UserId

```
public int UserId { get; set; }
```

Property Value

int

Class UpdateBankAccountRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsClass]
public class UpdateBankAccountRequest : UpdateAccountRequest
```

Inheritance

object ← UpdateAccountRequest ← UpdateBankAccountRequest

Inherited Members

UpdateAccountRequest.UserId , UpdateAccountRequest.AccountId , UpdateAccountRequest.AccountName , UpdateAccountRequest.Institution , UpdateAccountRequest.Balance , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BankAccountId

```
[JsonConverter(typeof(SafeDefaultConverter<int>))]
public int BankAccountId { get; set; }
```

Property Value

int

BankAccountType

```
public BankAccountType? BankAccountType { get; set; }
```

Property Value

BankAccountType?

Class UpdateDebtAccountRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsClass]
public class UpdateDebtAccountRequest : UpdateAccountRequest
```

Inheritance

object ← UpdateAccountRequest ← UpdateDebtAccountRequest

Inherited Members

UpdateAccountRequest.UserId , UpdateAccountRequest.AccountId , UpdateAccountRequest.AccountName , UpdateAccountRequest.Institution , UpdateAccountRequest.Balance , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

AccountNumber

```
public Optional<int?> AccountNumber { get; set; }
```

Property Value

Optional<int?>

AmountOfNextBill

```
public Optional<decimal?> AmountOfNextBill { get; set; }
```

Property Value

Optional<decimal?>

DateOfNextBill

```
public Optional<DateOnly?> DateOfNextBill { get; set; }
```

Property Value

Optional<DateOnly?>

DebtAccountId

```
public int DebtAccountId { get; set; }
```

Property Value

int

DebtAccountType

```
public DebtAccountType? DebtAccountType { get; set; }
```

Property Value

DebtAccountType?

DebtPaymentRegularity

```
public Optional<DebtPaymentRegularity?> DebtPaymentRegularity { get; set; }
```

Property Value

Optional<DebtPaymentRegularity?>

Class UpdateInvestmentAccountRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsClass]
public class UpdateInvestmentAccountRequest : UpdateAccountRequest
```

Inheritance

object ← UpdateAccountRequest ← UpdateInvestmentAccountRequest

Inherited Members

UpdateAccountRequest.UserId , UpdateAccountRequest.AccountId , UpdateAccountRequest.AccountName , UpdateAccountRequest.Institution , UpdateAccountRequest.Balance , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

AccountNumber

```
public Optional<int?> AccountNumber { get; set; }
```

Property Value

Optional<int?>

InvestmentAccountId

```
public int InvestmentAccountId { get; set; }
```

Property Value

int

InvestmentAccountType

```
public InvestmentAccountType? InvestmentAccountType { get; set; }
```

Property Value

InvestmentAccountType?

IsTaxDeferred

```
public bool? IsTaxDeferred { get; set; }
```

Property Value

bool?

IsTextExempt

```
public bool? IsTextExempt { get; set; }
```

Property Value

bool?

Class UpdatePaycheckRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsClass]
public class UpdatePaycheckRequest
```

Inheritance

object ← UpdatePaycheckRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

PaycheckId

```
public int PaycheckId { get; set; }
```

Property Value

int

PaycheckRegularity

```
public PaycheckRegularity? PaycheckRegularity { get; set; }
```

Property Value

PaycheckRegularity?

SecondaryDate

```
public Optional<DateOnly?> SecondaryDate { get; set; }
```

Property Value

Optional<DateOnly?>

StartingDate

```
public DateOnly? StartingDate { get; set; }
```

Property Value

DateOnly?

TotalAmount

```
public decimal? TotalAmount { get; set; }
```

Property Value

decimal?

UserId

```
public int UserId { get; set; }
```

Property Value

int

Class UpdatePaymentTransactionRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsClass]
public class UpdatePaymentTransactionRequest : UpdateTransactionRequest
```

Inheritance

object ← UpdateTransactionRequest ← UpdatePaymentTransactionRequest

Inherited Members

UpdateTransactionRequest.UserId , UpdateTransactionRequest.TransactionId , UpdateTransactionRequest.TransactionDate , UpdateTransactionRequest.TotalAmount , UpdateTransactionRequest.Notes , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

AccountId

```
public int? AccountId { get; set; }
```

Property Value

int?

CheckNumber

```
public Optional<int?> CheckNumber { get; set; }
```

Property Value

Optional<int?>

MerchantSourceName

```
public string? MerchantSourceName { get; set; }
```

Property Value

string

PaymentTransactionId

```
public int PaymentTransactionId { get; set; }
```

Property Value

int

PaymentTransactionType

```
public PaymentTransactionType? PaymentTransactionType { get; set; }
```

Property Value

PaymentTransactionType?

TransactionSplits

```
public List<UpdateTransactionSplitRequest> TransactionSplits { get; set; }
```

Property Value

List <UpdateTransactionSplitRequest>

Class UpdatePaymentTransactionResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsInterface]
public class UpdatePaymentTransactionResponse : BaseIdResponse
```

Inheritance

object ← BaseResponse ← BaseIdResponse ← UpdatePaymentTransactionResponse

Inherited Members

BaseIdResponse.Id , BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

UpdatePaymentTransactionResponse()

```
public UpdatePaymentTransactionResponse()
```

UpdatePaymentTransactionResponse(int, string)

```
public UpdatePaymentTransactionResponse(int httpStatus, string message)
```

Parameters

`httpStatus` int

`message` string

UpdatePaymentTransactionResponse(int, string, int)

```
public UpdatePaymentTransactionResponse(int httpStatus, string message, int id)
```

Parameters

`httpStatus` int

`message` string

`id` int

Properties

TransactionSplitIds

```
public List<int> TransactionSplitIds { get; set; }
```

Property Value

List<int>

Class UpdateRolloverEnvelopeRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Request class for updating a rollover envelope's persistent definition

```
[ExportTsClass]
public class UpdateRolloverEnvelopeRequest
```

Inheritance

object ← UpdateRolloverEnvelopeRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

EnvelopeCategoryId

```
public int? EnvelopeCategoryId { get; set; }
```

Property Value

int?

EnvelopeName

```
public string? EnvelopeName { get; set; }
```

Property Value

string

RolloverEnvelopeId

```
public int RolloverEnvelopeId { get; set; }
```

Property Value

int

TransferEnvelopeId

```
public int? TransferEnvelopeId { get; set; }
```

Property Value

int?

UserId

```
public int UserId { get; set; }
```

Property Value

int

Class UpdateSinkingFundEnvelopeRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Request class for updating a sinking fund envelope

```
[ExportTsClass]
public class UpdateSinkingFundEnvelopeRequest
```

Inheritance

object ← UpdateSinkingFundEnvelopeRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

EnvelopeCategoryId

```
public int? EnvelopeCategoryId { get; set; }
```

Property Value

int?

EnvelopeName

```
public string? EnvelopeName { get; set; }
```

Property Value

string

GoalAmount

```
public Optional<decimal?> GoalAmount { get; set; }
```

Property Value

Optional<decimal?>

GoalEndDate

```
public Optional<DateOnly?> GoalEndDate { get; set; }
```

Property Value

Optional<DateOnly?>

SinkingFundEnvelopeId

```
public int SinkingFundEnvelopeId { get; set; }
```

Property Value

int

UserId

```
public int UserId { get; set; }
```

Property Value

int

Class UpdateTransactionRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsClass]
public class UpdateTransactionRequest
```

Inheritance

object ← UpdateTransactionRequest

Derived

UpdatePaymentTransactionRequest, UpdateTransferTransactionRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Notes

```
public Optional<string?> Notes { get; set; }
```

Property Value

Optional<string>

TotalAmount

```
public decimal? TotalAmount { get; set; }
```

Property Value

decimal?

TransactionDate

```
public DateOnly? TransactionDate { get; set; }
```

Property Value

DateOnly?

TransactionId

```
public int TransactionId { get; set; }
```

Property Value

int

UserId

```
public int UserId { get; set; }
```

Property Value

int

Class UpdateTransactionSplitRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsClass]
public class UpdateTransactionSplitRequest
```

Inheritance

object ← UpdateTransactionSplitRequest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Amount

```
public decimal? Amount { get; set; }
```

Property Value

decimal?

EnvelopeId

```
public int? EnvelopeId { get; set; }
```

Property Value

int?

IsSinkingFund

```
public bool? IsSinkingFund { get; set; }
```

Property Value

bool?

TransactionId

```
public int? TransactionId { get; set; }
```

Property Value

int?

TransactionSplitId

```
public int? TransactionSplitId { get; set; }
```

Property Value

int?

Class UpdateTransferTransactionRequest

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

```
[ExportTsClass]
public class UpdateTransferTransactionRequest : UpdateTransactionRequest
```

Inheritance

object ← UpdateTransactionRequest ← UpdateTransferTransactionRequest

Inherited Members

UpdateTransactionRequest.UserId , UpdateTransactionRequest.TransactionId , UpdateTransactionRequest.TransactionDate , UpdateTransactionRequest.TotalAmount , UpdateTransactionRequest.Notes , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

TransferFromId

```
public int? TransferFromId { get; set; }
```

Property Value

int?

TransferFromIsSinkingFund

```
public bool? TransferFromIsSinkingFund { get; set; }
```

Property Value

bool?

TransferToId

```
public int? TransferToId { get; set; }
```

Property Value

int?

TransferToIsSinkingFund

```
public bool? TransferToIsSinkingFund { get; set; }
```

Property Value

bool?

TransferTransactionId

```
public int TransferTransactionId { get; set; }
```

Property Value

int

TransferTransactionType

```
public TransferTransactionType? TransferTransactionType { get; set; }
```

Property Value

TransferTransactionType?

Class UserDTOModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Model for a user

```
[ExportTsInterface]
public class UserDTOModel
```

Inheritance

object ← UserDTOModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Budget

```
public BudgetEntityModel? Budget { get; set; }
```

Property Value

BudgetEntityModel

Email

```
public string Email { get; set; }
```

Property Value

string

IsDarkMode

```
public bool IsDarkMode { get; set; }
```

Property Value

bool

UserId

```
public int UserId { get; set; }
```

Property Value

int

Username

```
public string Username { get; set; }
```

Property Value

string

Class UserDTOResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Response model for a DTO user

```
[ExportTsInterface]
public class UserDTOResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← UserDTOResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Token

```
public string? Token { get; set; }
```

Property Value

string

User

```
public UserDTOModel? User { get; set; }
```

Property Value

UserDTOModel

Class UserEntityModel

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Model for a user

```
public class UserEntityModel
```

Inheritance

object ← UserEntityModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Budget

```
public BudgetEntityModel? Budget { get; set; }
```

Property Value

BudgetEntityModel

CreatedAt

```
public DateTime CreatedAt { get; set; }
```

Property Value

DateTime

Email

```
public string Email { get; set; }
```

Property Value

string

IsDarkMode

```
public bool IsDarkMode { get; set; }
```

Property Value

bool

LastUpdatedAt

```
public DateTime LastUpdatedAt { get; set; }
```

Property Value

DateTime

PasswordHash

```
public string PasswordHash { get; set; }
```

Property Value

string

UserId

```
public int UserId { get; set; }
```

Property Value

int

Username

```
public string Username { get; set; }
```

Property Value

string

Class UserEntityResponse

Namespace: MakesCentsBackend.Models

Assembly: MakesCentsBackend.dll

Response model for an entity user

```
[ExportTsInterface]
public class UserEntityResponse : BaseResponse
```

Inheritance

object ← BaseResponse ← UserEntityResponse

Inherited Members

BaseResponse.HttpStatus , BaseResponse.Message , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

UserEntityResponse(UserEntityModel, int, string)

Parameterized constructor for the register response model

```
public UserEntityResponse(UserEntityModel user, int status, string message)
```

Parameters

`user` UserEntityModel

`status` int

`message` string

Properties

Token

```
public string? Token { get; set; }
```

Property Value

string

User

```
public UserEntityModel User { get; set; }
```

Property Value

UserEntityModel

Namespace MakesCentsBackend.Models.Converters

Classes

OptionalDBConverter<T>

OptionalJsonConverterFactory

OptionalJsonConverter<T>

SafeDefaultConverter<T>

A generic JSON converter that safely deserializes a property. Returns the default value if the input cannot be converted to the target type.

Class OptionalDBConverter<T>

Namespace: MakesCentsBackend.Models.Converters

Assembly: MakesCentsBackend.dll

```
public class OptionalDBConverter<T> : SqlMapper.TypeHandler<Optional<T>>,
SqlMapper.ITypeHandler
```

Type Parameters

T

Inheritance

object ← SqlMapper.TypeHandler<Optional<T>> ← OptionalDBConverter<T>

Implements

SqlMapper.ITypeHandler

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

Parse(object)

Parse a database value back to a typed value

```
public override Optional<T> Parse(object value)
```

Parameters

value object

The value from the database

Returns

Optional<T>

The typed value

SetValue(IDbDataParameter, Optional<T>)

Assign the value of a parameter before a command executes

```
public override void SetValue(IDbDataParameter parameter, Optional<T> value)
```

Parameters

`parameter` IDbDataParameter

The parameter to configure

`value` Optional<T>

Parameter value

Class OptionalJsonConverterFactory

Namespace: MakesCentsBackend.Models.Converters

Assembly: MakesCentsBackend.dll

```
public class OptionalJsonConverterFactory : JsonConverterFactory
```

Inheritance

object ← JsonConverter ← JsonConverterFactory ← OptionalJsonConverterFactory

Inherited Members

JsonConverterFactory.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

CanConvert(Type)

```
public override bool CanConvert(Type typeToConvert)
```

Parameters

`typeToConvert` Type

Returns

bool

CreateConverter(Type, JsonSerializerOptions)

```
public override JsonConverter? CreateConverter(Type typeToConvert,
JsonSerializerOptions options)
```

Parameters

`typeToConvert` Type

`options` JsonSerializerOptions

Returns

JsonConverter

Class OptionalJsonConverter<T>

Namespace: MakesCentsBackend.Models.Converters

Assembly: MakesCentsBackend.dll

```
public class OptionalJsonConverter<T> : JsonConverter<Optional<T>>
```

Type Parameters

T

Inheritance

object ← JsonConverter ← JsonConverter<Optional<T>> ← OptionalJsonConverter<T>

Inherited Members

JsonConverter<Optional<T>>.CanConvert(Type) ,
JsonConverter<Optional<T>>.ReadAsPropertyName(ref Utf8JsonReader, Type, JsonSerializerOptions) ,
JsonConverter<Optional<T>>.WriteAsPropertyName(Utf8JsonWriter, Optional<T>, JsonSerializerOptions) ,
JsonConverter<Optional<T>>.HandleNull , JsonConverter<Optional<T>>.Type ,
object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() ,
object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

Read(ref Utf8JsonReader, Type, JsonSerializerOptions)

```
public override Optional<T> Read(ref Utf8JsonReader reader, Type typeToConvert,
JsonSerializerOptions options)
```

Parameters

`reader` Utf8JsonReader

`typeToConvert` Type

`options` JsonSerializerOptions

Returns

Optional<T>

Write(Utf8JsonWriter, Optional<T>, JsonSerializerOptions)

```
public override void Write(Utf8JsonWriter writer, Optional<T> value,
JsonSerializerOptions options)
```

Parameters

`writer` Utf8JsonWriter

`value` Optional<T>

`options` JsonSerializerOptions

Class SafeDefaultConverter<T>

Namespace: MakesCentsBackend.Models.Converters

Assembly: MakesCentsBackend.dll

A generic JSON converter that safely deserializes a property. Returns the default value if the input cannot be converted to the target type.

```
public class SafeDefaultConverter<T> : JsonConverter<T>
```

Type Parameters

T

Inheritance

object ← JsonConverter ← JsonConverter<T> ← SafeDefaultConverter<T>

Inherited Members

JsonConverter<T>.CanConvert(Type) ,
JsonConverter<T>.ReadAsPropertyName(ref Utf8JsonReader, Type, JsonSerializerOptions) ,
JsonConverter<T>.WriteAsPropertyName(Utf8JsonWriter, T, JsonSerializerOptions) ,
JsonConverter<T>.HandleNull , JsonConverter<T>.Type , object.Equals(object) ,
object.Equals(object, object) , object.GetHashCode() , object.GetType() ,
object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

Read(ref Utf8JsonReader, Type, JsonSerializerOptions)

Reads JSON and converts it to the target type. Returns fallback for invalid input.

```
public override T Read(ref Utf8JsonReader reader, Type typeToConvert,
JsonSerializerOptions options)
```

Parameters

reader Utf8JsonReader

`typeToConvert` Type

`options` JsonSerializerOptions

Returns

T

Write(Utf8JsonWriter, T, JsonSerializerOptions)

Writes the property value to JSON.

```
public override void Write(Utf8JsonWriter writer, T value, JsonSerializerOptions options)
```

Parameters

`writer` Utf8JsonWriter

`value` T

`options` JsonSerializerOptions

Namespace MakesCentsBackend.Models.Entities

Classes

PlannedExpenseEntity

Model for a planned expense

Class PlannedExpenseEntity

Namespace: MakesCentsBackend.Models.Entities

Assembly: MakesCentsBackend.dll

Model for a planned expense

```
public class PlannedExpenseEntity
```

Inheritance

object ← PlannedExpenseEntity

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Amount

```
public decimal Amount { get; set; }
```

Property Value

decimal

BudgetId

```
public int BudgetId { get; set; }
```

Property Value

int

CreatedAt

```
public DateTime CreatedAt { get; set; }
```

Property Value

DateTime

DayOfMonth

```
public int? DayOfMonth { get; set; }
```

Property Value

int?

EnvelopeId

```
public int EnvelopeId { get; set; }
```

Property Value

int

LastUpdatedAt

```
public DateTime LastUpdatedAt { get; set; }
```

Property Value

DateTime

PlannedExpenseId

```
public int PlannedExpenseId { get; set; }
```

Property Value

int

PlannedExpenseOccurrence

```
public PlannedExpenseOccurrence? PlannedExpenseOccurrence { get; set; }
```

Property Value

PlannedExpenseOccurrence?

PlannedExpenseRegularity

```
public PlannedExpenseRegularity PlannedExpenseRegularity { get; set; }
```

Property Value

PlannedExpenseRegularity

Weekday

```
public Weekday? Weekday { get; set; }
```

Property Value

Weekday?

Namespace MakesCentsBackend.Models.Enums

Enums

AccountType
Enum for Account Types for the Account Model

BankAccountType
Enum for Bank Account Types for the Bank Account Model

DebtAccountType
Enum for Debt Account Types for the Debt Account Model

DebtPaymentRegularity
Enum for Debt Payment Regularities for the Debt Account Model

InvestmentAccountType
Enum for Investment Account Types for Investment Account Model

Month
Enum for the Months for the Budget Model

PaycheckRegularity
Enum for Paycheck Regularities for the Paycheck Model

PaymentTransactionType
Enum for Payment Transaction Types for the Payment Transaction Model

PlannedExpenseOccurrence
Enum for Planned Expense Occurrences for the Planned Expense Model

PlannedExpenseRegularity
Enum for Planned Expense Regularities for the Planned Expense Model

TransactionType
Enum for Transaction Types for the Transaction Model

TransferTransactionType
Enum for Transfer Transaction Types for the Transfer Transaction Model

Weekday
Enum for Weekdays for the Planned Expense Model

Enum AccountType

Namespace: MakesCentsBackend.Models.Enums

Assembly: MakesCentsBackend.dll

Enum for Account Types for the Account Model

```
public enum AccountType
```

Fields

`Bank = 2`

`Debt = 3`

`Investment = 4`

`Unknown = 1`

Enum BankAccountType

Namespace: MakesCentsBackend.Models.Enums

Assembly: MakesCentsBackend.dll

Enum for Bank Account Types for the Bank Account Model

```
public enum BankAccountType
```

Fields

`Checking = 3`

`Savings = 2`

`Unknown = 1`

Enum DebtAccountType

Namespace: MakesCentsBackend.Models.Enums

Assembly: MakesCentsBackend.dll

Enum for Debt Account Types for the Debt Account Model

```
public enum DebtAccountType
```

Fields

CarLoan = 4

CreditCard = 2

LineOfCreditLoan = 3

Mortgage = 5

Unknown = 1

Enum DebtPaymentRegularity

Namespace: MakesCentsBackend.Models.Enums

Assembly: MakesCentsBackend.dll

Enum for Debt Payment Regularities for the Debt Account Model

```
public enum DebtPaymentRegularity
```

Fields

`Annually = 8`

`BiMonthlyEveryTwoMonths = 5`

`BiWeeklyEveryTwoWeeks = 3`

`Monthly = 4`

`Quarterly = 6`

`TwiceAnnually = 7`

`Unknown = 1`

`Weekly = 2`

Enum InvestmentAccountType

Namespace: MakesCentsBackend.Models.Enums

Assembly: MakesCentsBackend.dll

Enum for Investment Account Types for Investment Account Model

```
public enum InvestmentAccountType
```

Fields

Brokerage = 4

IRA = 2

Other = 5

Retirement401K403B = 3

Unknown = 1

Enum Month

Namespace: MakesCentsBackend.Models.Enums

Assembly: MakesCentsBackend.dll

Enum for the Months for the Budget Model

```
public enum Month
```

Fields

April = 5

August = 9

December = 13

February = 3

January = 2

July = 8

June = 7

March = 4

May = 6

November = 12

October = 11

September = 10

Unknown = 1

Enum PaycheckRegularity

Namespace: MakesCentsBackend.Models.Enums

Assembly: MakesCentsBackend.dll

Enum for Paycheck Regularities for the Paycheck Model

```
public enum PaycheckRegularity
```

Fields

BiMonthlyTwiceAMonth = 4

BiWeeklyEveryTwoWeeks = 3

Monthly = 5

Unknown = 1

Weekly = 2

Enum PaymentTransactionType

Namespace: MakesCentsBackend.Models.Enums

Assembly: MakesCentsBackend.dll

Enum for Payment Transaction Types for the Payment Transaction Model

```
public enum PaymentTransactionType
```

Fields

ATM = 2

Check = 3

CreditCard = 5

DebitCard = 4

Deposit = 6

LoanDeposit = 9

Paycheck = 7

Refund = 8

Unknown = 1

Enum PlannedExpenseOccurrence

Namespace: MakesCentsBackend.Models.Enums

Assembly: MakesCentsBackend.dll

Enum for Planned Expense Occurrences for the Planned Expense Model

```
public enum PlannedExpenseOccurrence
```

Fields

`First = 2`

`Fourth = 5`

`Last = 6`

`Second = 3`

`Third = 4`

`Unknown = 1`

Enum PlannedExpenseRegularity

Namespace: MakesCentsBackend.Models.Enums

Assembly: MakesCentsBackend.dll

Enum for Planned Expense Regularities for the Planned Expense Model

```
public enum PlannedExpenseRegularity
```

Fields

DayOfMonth = 2

LastDayOfMonth = 3

Unknown = 1

WeekdayOccurrence = 4

Enum TransactionType

Namespace: MakesCentsBackend.Models.Enums

Assembly: MakesCentsBackend.dll

Enum for Transaction Types for the Transaction Model

```
public enum TransactionType
```

Fields

`Payment = 2`

`Transfer = 3`

`Unknown = 1`

Enum TransferTransactionType

Namespace: MakesCentsBackend.Models.Enums

Assembly: MakesCentsBackend.dll

Enum for Transfer Transaction Types for the Transfer Transaction Model

```
public enum TransferTransactionType
```

Fields

`Account = 2`

`Envelope = 3`

`Unknown = 1`

Enum Weekday

Namespace: MakesCentsBackend.Models.Enums

Assembly: MakesCentsBackend.dll

Enum for Weekdays for the Planned Expense Model

```
public enum Weekday
```

Fields

`Friday = 6`

`Monday = 2`

`Saturday = 7`

`Sunday = 8`

`Thursday = 5`

`Tuesday = 3`

`Unknown = 1`

`Wednesday = 4`

Namespace MakesCentsBackend.Services.BusinessLogicLayer

Classes

AccountLogic

BankAccountLogic

BudgetLogic

BudgetTemplateLogic

DebtAccountLogic

EnvelopeCategoryLogic

EnvelopeFundingLogic

InvestmentAccountLogic

PaycheckLogic

PaymentTransactionLogic

RolloverEnvelopeLogic

SinkingFundEnvelopeLogic

TransactionLogic

TransferTransactionLogic

UserLogic

Class for all business rules for users

Class AccountLogic

Namespace: MakesCentsBackend.Services.BusinessLogicLayer

Assembly: MakesCentsBackend.dll

```
public class AccountLogic
```

Inheritance

object ← AccountLogic

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

AccountLogic(AccountDAO)

Parameterized constructor to bring in DI variables

```
public AccountLogic(AccountDAO accountDAO)
```

Parameters

`accountDAO` AccountDAO

Methods

DeleteAccountAsync(BaseIdRequest)

Logic method to delete an account

```
public Task<BaseResponse> DeleteAccountAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<BaseResponse>

GetAllAccountsAsync(BaseIdRequest)

```
public Task<GetAllAccountsResponse> GetAllAccountsAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<GetAllAccountsResponse>

Class BankAccountLogic

Namespace: MakesCentsBackend.Services.BusinessLogicLayer

Assembly: MakesCentsBackend.dll

```
public class BankAccountLogic
```

Inheritance

object ← BankAccountLogic

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

BankAccountLogic(BankAccountDAO, IMapper)

Parameterized constructor to bring in DI variables

```
public BankAccountLogic(BankAccountDAO bankAccountDAO, IMapper mapper)
```

Parameters

`bankAccountDAO` BankAccountDAO

`mapper` IMapper

Methods

CreateBankAccountAsync(CreateBankAccountRequest)

Logic method to create a new bank account

```
public Task<CreateBankAccountResponse> CreateBankAccountAsync(CreateBankAccountRequest
bankAccount)
```

Parameters

`bankAccount` CreateBankAccountRequest

Returns

Task<CreateBankAccountResponse>

GetBankAccountAsync(BaseIdRequest)

```
public Task<GetBankAccountDTOResponse> GetBankAccountAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<GetBankAccountDTOResponse>

UpdateBankAccountAsync(UpdateBankAccountRequest)

```
public Task<BaseIdResponse> UpdateBankAccountAsync(UpdateBankAccountRequest bankAccount)
```

Parameters

`bankAccount` UpdateBankAccountRequest

Returns

Task<BaseIdResponse>

Class BudgetLogic

Namespace: MakesCentsBackend.Services.BusinessLogicLayer

Assembly: MakesCentsBackend.dll

```
public class BudgetLogic
```

Inheritance

object ← BudgetLogic

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

BudgetLogic(BudgetDAO)

Parameterized constructor to bring in DI variables

```
public BudgetLogic(BudgetDAO budgetDAO)
```

Parameters

`budgetDAO` BudgetDAO

Methods

CreateBudgetAsync(CreateBudgetRequest)

Logic method to create a new persistent budget (no month yet - see CreateBudgetMonthAsync)

```
public Task<BaseIdResponse> CreateBudgetAsync(CreateBudgetRequest budget)
```

Parameters

`budget` CreateBudgetRequest

Returns

Task<BaseIdResponse>

CreateBudgetMonthAsync(CreateBudgetMonthRequest)

Logic method to create a new month under an existing persistent budget

```
public Task<BaseIdResponse> CreateBudgetMonthAsync(CreateBudgetMonthRequest budgetMonth)
```

Parameters

`budgetMonth` CreateBudgetMonthRequest

Returns

Task<BaseIdResponse>

DeleteBudgetAsync(BaseIdRequest)

Logic method to delete a persistent budget entirely (cascades every month, account, envelope category, paycheck, and planned expense it has)

```
public Task<BaseResponse> DeleteBudgetAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<BaseResponse>

DeleteBudgetMonthAsync(BaseIdRequest)

Logic method to delete a single month's budget (that month's transactions go with it - accounts, envelope categories, and paychecks are unaffected since they're persistent)

```
public Task<BaseResponse> DeleteBudgetMonthAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<BaseResponse>

GetBudgetMonthAsync(BaseIdRequest)

Logic method to get a budget month based on its budget_month_id

```
public Task<GetBudgetMonthResponse> GetBudgetMonthAsync(BaseIdRequest budget)
```

Parameters

`budget` BaseIdRequest

Returns

Task<GetBudgetMonthResponse>

GetBudgetMonthAsync(GetBudgetRequest)

Logic method to get a budget month based on a year/month/user lookup

```
public Task<GetBudgetMonthResponse> GetBudgetMonthAsync(GetBudgetRequest budget)
```

Parameters

`budget` GetBudgetRequest

Returns

Task<GetBudgetMonthResponse>

GetUserBudgetsAsync(int)

Logic method to get every budget a user has access to (id and name only) - meant for use right after login, before any month or budget id is known yet.

```
public Task<GetUserBudgetsResponse> GetUserBudgetsAsync(int userId)
```

Parameters

`userId` int

Returns

Task<GetUserBudgetsResponse>

GetYearBudgetsAsync(GetBudgetRequest)

Logic method to get up to a year's worth of a user's past budget months

```
public Task<GetYearBudgetsResponse> GetYearBudgetsAsync(GetBudgetRequest request)
```

Parameters

`request` GetBudgetRequest

Returns

Task<GetYearBudgetsResponse>

UpdateBudgetAsync(EditBudgetRequest)

Logic method to update a budget's persistent fields (currently just the name)

```
public Task<BaseIdResponse> UpdateBudgetAsync(EditBudgetRequest budget)
```

Parameters

`budget` EditBudgetRequest

Returns

Task<BaseIdResponse>

UpdateBudgetMonthAsync(EditBudgetMonthRequest)

Logic method to update a specific budget month's month/year (e.g. fixing a mistake)

```
public Task<BaseIdResponse> UpdateBudgetMonthAsync(EditBudgetMonthRequest budgetMonth)
```

Parameters

`budgetMonth` EditBudgetMonthRequest

Returns

Task<BaseIdResponse>

Class BudgetTemplateLogic

Namespace: MakesCentsBackend.Services.BusinessLogicLayer

Assembly: MakesCentsBackend.dll

```
public class BudgetTemplateLogic
```

Inheritance

object ← BudgetTemplateLogic

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

BudgetTemplateLogic(BudgetTemplateDAO)

Parameterized constructor to bring in DI variables

```
public BudgetTemplateLogic(BudgetTemplateDAO budgetTemplateDAO)
```

Parameters

`budgetTemplateDAO` BudgetTemplateDAO

Methods

ApplyBudgetTemplateAsync(ApplyBudgetTemplateRequest)

Logic method to apply a template to a budget month, fully replacing that month's funding queue with the template's entries

```
public Task<GetEnvelopeFundingQueueResponse>
ApplyBudgetTemplateAsync(ApplyBudgetTemplateRequest apply)
```

Parameters

`apply` ApplyBudgetTemplateRequest

Returns

Task<GetEnvelopeFundingQueueResponse>

CreateBudgetTemplateAsync(CreateBudgetTemplateRequest)

Logic method to create a budget template

```
public Task<BaseIdResponse> CreateBudgetTemplateAsync(CreateBudgetTemplateRequest template)
```

Parameters

`template` CreateBudgetTemplateRequest

Returns

Task<BaseIdResponse>

DeleteBudgetTemplateAsync(BaseIdRequest)

Logic method to delete a budget template

```
public Task<BaseResponse> DeleteBudgetTemplateAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<BaseResponse>

GetAllBudgetTemplatesAsync(BaseIdRequest)

Logic method to get every template for a budget

```
public Task<GetAllBudgetTemplatesResponse> GetAllBudgetTemplatesAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task <GetAllBudgetTemplatesResponse>

GetBudgetTemplateAsync(BaseIdRequest)

Logic method to get a single budget template, including its envelope entries

```
public Task<GetBudgetTemplateResponse> GetBudgetTemplateAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task <GetBudgetTemplateResponse>

SetBudgetTemplateEnvelopesAsync(SetBudgetTemplateEnvelopesRequest)

Logic method to set a template's envelope entries

```
public Task<GetBudgetTemplateResponse>
SetBudgetTemplateEnvelopesAsync(SetBudgetTemplateEnvelopesRequest templateEnvelopes)
```

Parameters

`templateEnvelopes` SetBudgetTemplateEnvelopesRequest

Returns

Task<GetBudgetTemplateResponse>

UpdateBudgetTemplateAsync(EditBudgetTemplateRequest)

Logic method to rename a budget template

```
public Task<BaseIdResponse> UpdateBudgetTemplateAsync(EditBudgetTemplateRequest template)
```

Parameters

`template` EditBudgetTemplateRequest

Returns

Task<BaseIdResponse>

Class DebtAccountLogic

Namespace: MakesCentsBackend.Services.BusinessLogicLayer

Assembly: MakesCentsBackend.dll

```
public class DebtAccountLogic
```

Inheritance

object ← DebtAccountLogic

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

DebtAccountLogic(DebtAccountDAO, IMapper)

Parameterized constructor to bring in DI variables

```
public DebtAccountLogic(DebtAccountDAO debtAccountDAO, IMapper mapper)
```

Parameters

`debtAccountDAO` DebtAccountDAO

`mapper` IMapper

Methods

CreateDebtAccountAsync(CreateDebtAccountRequest)

Logic method to create a new debt account

```
public Task<CreateDebtAccountResponse> CreateDebtAccountAsync(CreateDebtAccountRequest
debtAccount)
```

Parameters

`debtAccount` CreateDebtAccountRequest

Returns

Task<CreateDebtAccountResponse>

GetDebtAccountAsync(BaseIdRequest)

```
public Task<GetDebtAccountDTOResponse> GetDebtAccountAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<GetDebtAccountDTOResponse>

UpdateDebtAccountAsync(UpdateDebtAccountRequest)

```
public Task<BaseIdResponse> UpdateDebtAccountAsync(UpdateDebtAccountRequest debtAccount)
```

Parameters

`debtAccount` UpdateDebtAccountRequest

Returns

Task<BaseIdResponse>

Class EnvelopeCategoryLogic

Namespace: MakesCentsBackend.Services.BusinessLogicLayer

Assembly: MakesCentsBackend.dll

```
public class EnvelopeCategoryLogic
```

Inheritance

object ← EnvelopeCategoryLogic

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

EnvelopeCategoryLogic(EnvelopeCategoryDAO)

Parameterized constructor to bring in DI variables

```
public EnvelopeCategoryLogic(EnvelopeCategoryDAO envelopeCategoryDAO)
```

Parameters

`envelopeCategoryDAO` EnvelopeCategoryDAO

Methods

CreateEnvelopeCategoryAsync(CreateEnvelopeCategoryRequest)

Logic method to create an envelope category. Still keyed to a persistent BudgetId - categories aren't month-scoped.

```
public Task<BaseIdResponse> CreateEnvelopeCategoryAsync(CreateEnvelopeCategoryRequest
envelopeCategory)
```

Parameters

`envelopeCategory` CreateEnvelopeCategoryRequest

Returns

Task<BaseIdResponse>

DeleteEnvelopeCategoryAsync(BaseIdRequest)

Logic method to delete an envelope category

```
public Task<BaseResponse> DeleteEnvelopeCategoryAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<BaseResponse>

GetAllEnvelopeCategoriesAsync(BaseIdRequest)

Logic method to get all envelope categories (with envelopes) for a budget month. NOTE: request.EntityId is now a budget_month_id, not a budget_id - the DAO derives the persistent budget_id via a join, and needs the month to resolve rollover envelope amounts for the right month.

```
public Task<GetAllEnvelopeCategoriesResponse> GetAllEnvelopeCategoriesAsync(BaseIdRequest
request)
```

Parameters

`request` BaseIdRequest

Returns

Task<GetAllEnvelopeCategoriesResponse>

GetEnvelopeCategoryAsync(BaseIdRequest)

Logic method to get a single envelope category

```
public Task<GetEnvelopeCategoryResponse> GetEnvelopeCategoryAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task <GetEnvelopeCategoryResponse>

UpdateEnvelopeCategoryAsync(EditEnvelopeCategoryRequest)

Logic method to update an envelope category based on provided fields

```
public Task<BaseIdResponse> UpdateEnvelopeCategoryAsync(EditEnvelopeCategoryRequest
envelopeCategory)
```

Parameters

`envelopeCategory` EditEnvelopeCategoryRequest

Returns

Task <BaseIdResponse>

Class EnvelopeFundingLogic

Namespace: MakesCentsBackend.Services.BusinessLogicLayer

Assembly: MakesCentsBackend.dll

```
public class EnvelopeFundingLogic
```

Inheritance

object ← EnvelopeFundingLogic

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

EnvelopeFundingLogic(EnvelopeFundingDAO)

Parameterized constructor to bring in DI variables

```
public EnvelopeFundingLogic(EnvelopeFundingDAO envelopeFundingDAO)
```

Parameters

`envelopeFundingDAO` EnvelopeFundingDAO

Methods

DeleteEnvelopeFundingAsync(BaseIdRequest)

Logic method to delete a single envelope funding row

```
public Task<BaseResponse> DeleteEnvelopeFundingAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task <BaseResponse>

GetEnvelopeFundingQueueAsync(BaseIdRequest)

Logic method to get the envelope funding queue for a budget month

```
public Task<GetEnvelopeFundingQueueResponse> GetEnvelopeFundingQueueAsync(BaseIdRequest
request)
```

Parameters

`request` BaseIdRequest

Returns

Task <GetEnvelopeFundingQueueResponse>

SetEnvelopeFundingQueueAsync(SetEnvelopeFundingQueueRequest)

Logic method to set the envelope funding queue for a budget month

```
public Task<GetEnvelopeFundingQueueResponse>
SetEnvelopeFundingQueueAsync(SetEnvelopeFundingQueueRequest fundingQueue)
```

Parameters

`fundingQueue` SetEnvelopeFundingQueueRequest

Returns

Task <GetEnvelopeFundingQueueResponse>

Class InvestmentAccountLogic

Namespace: MakesCentsBackend.Services.BusinessLogicLayer

Assembly: MakesCentsBackend.dll

```
public class InvestmentAccountLogic
```

Inheritance

object ← InvestmentAccountLogic

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

InvestmentAccountLogic(InvestmentAccountDAO, IMapper)

Parameterized constructor to bring in DI variables

```
public InvestmentAccountLogic(InvestmentAccountDAO investmentAccountDAO, IMapper mapper)
```

Parameters

`investmentAccountDAO` InvestmentAccountDAO

`mapper` IMapper

Methods

CreateInvestmentAccountAsync(CreateInvestmentAccountRequest)

Logic method to create a new investment account

```
public Task<CreateInvestmentAccountResponse>
CreateInvestmentAccountAsync(CreateInvestmentAccountRequest investmentAccount)
```

Parameters

`investmentAccount` CreateInvestmentAccountRequest

Returns

Task <CreateInvestmentAccountResponse>

GetInvestmentAccountAsync(BaseIdRequest)

```
public Task<GetInvestmentAccountDTOResponse> GetInvestmentAccountAsync(BaseIdRequest
request)
```

Parameters

`request` BaseIdRequest

Returns

Task <GetInvestmentAccountDTOResponse>

UpdateInvestmentAccountAsync(UpdateInvestmentAccountRequest)

```
public Task<BaseIdResponse> UpdateInvestmentAccountAsync(UpdateInvestmentAccountRequest
investmentAccount)
```

Parameters

`investmentAccount` UpdateInvestmentAccountRequest

Returns

Task <BaseIdResponse>

Class PaycheckLogic

Namespace: MakesCentsBackend.Services.BusinessLogicLayer

Assembly: MakesCentsBackend.dll

```
public class PaycheckLogic
```

Inheritance

object ← PaycheckLogic

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

PaycheckLogic(PaycheckDAO)

Parameterized constructor to bring in DI variables

```
public PaycheckLogic(PaycheckDAO paycheckDAO)
```

Parameters

`paycheckDAO` PaycheckDAO

Methods

CreatePaycheckAsync(CreatePaycheckRequest)

Logic method to create a paycheck

```
public Task<BaseIdResponse> CreatePaycheckAsync(CreatePaycheckRequest paycheck)
```

Parameters

`paycheck` CreatePaycheckRequest

Returns

Task <BaseIdResponse>

DeletePaycheckAsync(BaseIdRequest)

Logic method to delete a paycheck

```
public Task<BaseResponse> DeletePaycheckAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task <BaseResponse>

GetAllPaychecksAsync(BaseIdRequest)

Logic method to get every paycheck for a budget

```
public Task<GetAllPaychecksResponse> GetAllPaychecksAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task <GetAllPaychecksResponse>

GetPaycheckAsync(BaseIdRequest)

Logic method to get a single paycheck

```
public Task<GetPaycheckResponse> GetPaycheckAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<GetPaycheckResponse>

UpdatePaycheckAsync(UpdatePaycheckRequest)

Logic method to update a paycheck

```
public Task<BaseIdResponse> UpdatePaycheckAsync(UpdatePaycheckRequest paycheck)
```

Parameters

`paycheck` UpdatePaycheckRequest

Returns

Task<BaseIdResponse>

Class PaymentTransactionLogic

Namespace: MakesCentsBackend.Services.BusinessLogicLayer

Assembly: MakesCentsBackend.dll

```
public class PaymentTransactionLogic
```

Inheritance

object ← PaymentTransactionLogic

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

PaymentTransactionLogic(PaymentTransactionDAO)

Parameterized constructor to bring in DI variables

```
public PaymentTransactionLogic(PaymentTransactionDAO paymentTransactionDAO)
```

Parameters

`paymentTransactionDAO` PaymentTransactionDAO

Methods

CreatePaymentTransactionAsync(CreatePaymentTransactionRequest)

Logic method to create a new payment transaction

```
public Task<CreatePaymentTransactionResponse>
CreatePaymentTransactionAsync(CreatePaymentTransactionRequest paymentTransaction)
```

Parameters

`paymentTransaction` CreatePaymentTransactionRequest

Returns

Task<CreatePaymentTransactionResponse>

GetPaymentTransactionAsync(BaseIdRequest)

Logic method to get a payment transaction and its splits

```
public Task<GetPaymentTransactionResponse> GetPaymentTransactionAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<GetPaymentTransactionResponse>

UpdatePaymentTransactionAsync(UpdatePaymentTransactionRequest)

Logic method to update a payment transaction and its splits

```
public Task<UpdatePaymentTransactionResponse>
UpdatePaymentTransactionAsync(UpdatePaymentTransactionRequest paymentTransaction)
```

Parameters

`paymentTransaction` UpdatePaymentTransactionRequest

Returns

Task<UpdatePaymentTransactionResponse>

Class RolloverEnvelopeLogic

Namespace: MakesCentsBackend.Services.BusinessLogicLayer

Assembly: MakesCentsBackend.dll

```
public class RolloverEnvelopeLogic
```

Inheritance

object ← RolloverEnvelopeLogic

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

RolloverEnvelopeLogic(RolloverEnvelopeDAO, IMapper)

Parameterized constructor to bring in DI variables

```
public RolloverEnvelopeLogic(RolloverEnvelopeDAO rolloverEnvelopeDAO, IMapper mapper)
```

Parameters

`rolloverEnvelopeDAO` RolloverEnvelopeDAO

`mapper` IMapper

Methods

CreateRolloverEnvelopeAsync(CreateRolloverEnvelopeRequest)

Logic method to create a new rollover envelope's persistent definition

```
public Task<BaseIdResponse> CreateRolloverEnvelopeAsync(CreateRolloverEnvelopeRequest
envelope)
```

Parameters

`envelope` CreateRolloverEnvelopeRequest

Returns

Task<BaseIdResponse>

DeleteRolloverEnvelopeAsync(BaseIdRequest)

Logic method to delete a rollover envelope's persistent definition (cascades every month it was ever activated for)

```
public Task<BaseResponse> DeleteRolloverEnvelopeAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<BaseResponse>

GetRolloverEnvelopeForMonthAsync(GetRolloverEnvelopeMonthRequest)

Logic method to get a rollover envelope for a specific budget month, including its transaction history for that month

```
public Task<GetRolloverEnvelopeDTOResponse>
GetRolloverEnvelopeForMonthAsync(GetRolloverEnvelopeMonthRequest request)
```

Parameters

`request` GetRolloverEnvelopeMonthRequest

Returns

Task<GetRolloverEnvelopeDTOResponse>

UpdateRolloverEnvelopeAsync(UpdateRolloverEnvelopeRequest)

Logic method to update a rollover envelope's persistent definition fields

```
public Task<BaseIdResponse> UpdateRolloverEnvelopeAsync(UpdateRolloverEnvelopeRequest envelope)
```

Parameters

`envelope` UpdateRolloverEnvelopeRequest

Returns

Task<BaseIdResponse>

Class SinkingFundEnvelopeLogic

Namespace: MakesCentsBackend.Services.BusinessLogicLayer

Assembly: MakesCentsBackend.dll

```
public class SinkingFundEnvelopeLogic
```

Inheritance

object ← SinkingFundEnvelopeLogic

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

SinkingFundEnvelopeLogic(SinkingFundEnvelopeDAO, IMapper)

Parameterized constructor to bring in DI variables

```
public SinkingFundEnvelopeLogic(SinkingFundEnvelopeDAO sinkingFundEnvelopeDAO,
IMapper mapper)
```

Parameters

`sinkingFundEnvelopeDAO` SinkingFundEnvelopeDAO

`mapper` IMapper

Methods

CreateSinkingFundEnvelopeAsync(CreateSinkingFundEnvelopeRequest)

Logic method to create a new sinking fund envelope

```
public Task<BaseIdResponse> CreateSinkingFundEnvelopeAsync(CreateSinkingFundEnvelopeRequest
envelope)
```

Parameters

`envelope` CreateSinkingFundEnvelopeRequest

Returns

Task<BaseIdResponse>

DeleteSinkingFundEnvelopeAsync(BaseIdRequest)

Logic method to delete a sinking fund envelope

```
public Task<BaseResponse> DeleteSinkingFundEnvelopeAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<BaseResponse>

GetRolloverDependentsAsync(BaseIdRequest)

Logic method to get the rollover envelopes that use a sinking fund envelope as their transfer target

```
public Task<GetSinkingFundDependentsResponse> GetRolloverDependentsAsync(BaseIdRequest
request)
```

Parameters

`request` BaseIdRequest

Returns

Task<GetSinkingFundDependentsResponse>

GetSinkingFundEnvelopeAsync(BaseIdRequest)

Logic method to get a specific sinking fund envelope, including its transaction history

```
public Task<GetSinkingFundEnvelopeDTOResponse> GetSinkingFundEnvelopeAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<GetSinkingFundEnvelopeDTOResponse>

UpdateSinkingFundEnvelopeAsync(UpdateSinkingFundEnvelopeRequest)

Logic method to update a sinking fund envelope based on provided fields

```
public Task<BaseIdResponse> UpdateSinkingFundEnvelopeAsync(UpdateSinkingFundEnvelopeRequest envelope)
```

Parameters

`envelope` UpdateSinkingFundEnvelopeRequest

Returns

Task<BaseIdResponse>

Class TransactionLogic

Namespace: MakesCentsBackend.Services.BusinessLogicLayer

Assembly: MakesCentsBackend.dll

```
public class TransactionLogic
```

Inheritance

object ← TransactionLogic

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

TransactionLogic(TransactionDAO, IMapper)

Parameterized constructor to bring in DI variables

```
public TransactionLogic(TransactionDAO transactionDAO, IMapper mapper)
```

Parameters

`transactionDAO` TransactionDAO

`mapper` IMapper

Methods

GetAllTransactionsAsync(BaseIdRequest)

Logic method to get every transaction for a budget month. NOTE: request.EntityId is a budget_month_id, not a budget_id - transactions are scoped to one specific month.

```
public Task<GetAllTransactionsDTOResponse> GetAllTransactionsAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<GetAllTransactionsDTOResponse>

HardDeleteTransactionAsync(BaseIdRequest)

Logic method to hard delete a transaction

```
public Task<BaseResponse> HardDeleteTransactionAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<BaseResponse>

RestoreTransactionAsync(BaseIdRequest)

Logic method to restore a transaction

```
public Task<BaseResponse> RestoreTransactionAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<BaseResponse>

SoftDeleteTransactionAsync(BaseIdRequest)

Logic method to soft delete a transaction

```
public Task<BaseResponse> SoftDeleteTransactionAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task <BaseResponse>

Class TransferTransactionLogic

Namespace: MakesCentsBackend.Services.BusinessLogicLayer

Assembly: MakesCentsBackend.dll

```
public class TransferTransactionLogic
```

Inheritance

object ← TransferTransactionLogic

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

TransferTransactionLogic(TransferTransactionDAO, IMapper)

Parameterized constructor to bring in DI variables

```
public TransferTransactionLogic(TransferTransactionDAO transferTransactionDAO,
IMapper mapper)
```

Parameters

`transferTransactionDAO` TransferTransactionDAO

`mapper` IMapper

Methods

CreateTransferTransactionAsync(CreateTransferTransactionRequest)

Logic method to create a new transfer transaction (account-to-account or envelope-to-envelope)

```
public Task<CreateTransferTransactionResponse>
CreateTransferTransactionAsync(CreateTransferTransactionRequest transferTransaction)
```

Parameters

`transferTransaction` CreateTransferTransactionRequest

Returns

Task <CreateTransferTransactionResponse>

GetTransferTransactionAsync(BaseIdRequest)

Logic method to get a transfer transaction

```
public Task<GetTransferTransactionDTOResponse> GetTransferTransactionAsync(BaseIdRequest
request)
```

Parameters

`request` BaseIdRequest

Returns

Task <GetTransferTransactionDTOResponse>

UpdateTransferTransactionAsync(UpdateTransferTransactionRequest)

Logic method to update a transfer transaction

```
public Task<BaseIdResponse> UpdateTransferTransactionAsync(UpdateTransferTransactionRequest
transferTransaction)
```

Parameters

`transferTransaction` UpdateTransferTransactionRequest

Returns

Task <BaseIdResponse>

Class UserLogic

Namespace: MakesCentsBackend.Services.BusinessLogicLayer

Assembly: MakesCentsBackend.dll

Class for all business rules for users

```
public class UserLogic
```

Inheritance

object ← UserLogic

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

UserLogic(UserDAO, JwtService, IMapper)

Parameterized constructor for UserLogic to pull in necessary dependencies

```
public UserLogic(UserDAO userDAO, JwtService jwtService, IMapper mapper)
```

Parameters

`userDAO` UserDAO

`jwtService` JwtService

`mapper` IMapper

Methods

DeleteUserAsync(int)

Async method to delete a user based on a userId

```
public Task<BaseResponse> DeleteUserAsync(int userId)
```

Parameters

`userId` int

Returns

Task<BaseResponse>

GetUserAsync(int)

Get a user based on a user id

```
public Task<GetUserResponse> GetUserAsync(int userId)
```

Parameters

`userId` int

Returns

Task<GetUserResponse>

LoginUserAsync(LoginRequest)

Authenticate a user based on a username or email and a password

```
public Task<LoginResponse> LoginUserAsync(LoginRequest user)
```

Parameters

`user` LoginRequest

Returns

Task<LoginResponse>

RegisterUserAsync(RegisterRequest)

Create a user from a userModel

```
public Task<RegisterResponse> RegisterUserAsync(RegisterRequest user)
```

Parameters

`user` RegisterRequest

The new user to create

Returns

Task<RegisterResponse>

UpdateUserAsync(EditUserRequest)

Async method to update a user

```
public Task<BaseIdResponse> UpdateUserAsync(EditUserRequest user)
```

Parameters

`user` EditUserRequest

The new user to create

Returns

Task<BaseIdResponse>

Namespace MakesCentsBackend.Services.DataAccessLayer

Classes

AccountDAO

AuthorizationService

BankAccountDAO
Data access object for bank accounts

BudgetDAO

BudgetTemplateDAO
Data access object for budget templates - named, reusable presets of envelope funding amounts that can be applied to a budget month to bulk-create its funding queue.

DebtAccountDAO

EnvelopeCategoryDAO
Data access object for envelope categories

EnvelopeFundingDAO
Data access object for the envelope funding queue - a budget month's plan for how much to put into each envelope and in what priority order. This is a plan only; it doesn't move money on its own the way a transaction does.

InvestmentAccountDAO

PaycheckDAO
Data access object for paychecks. Paychecks are forecast-only now - they don't fund envelopes directly (see EnvelopeFundingDAO for that) and aren't tied to any specific month, so this is plain single-table CRUD.

PaymentTransactionDAO

RolloverEnvelopeDAO
Data access object for rollover envelopes. Rollover envelopes are split across two tables:

- rollover_envelope: the persistent definition (name, category, default transfer target). No dollar amounts live here since a rollover envelope isn't guaranteed to be active every month.
- rollover_envelope_month: the amounts for one specific month. Absence of a row here means the envelope wasn't activated that month. Most "get/update amount" operations need a

budget_month_id in addition to the envelope id.

SinkingFundEnvelopeDAO

Data access object for sinking fund envelopes. Sinking fund envelopes are persistent - one row per envelope, with a live remaining_amount that carries forward month to month (no monthly instance table, unlike rollover envelopes).

TransactionDAO

TransferTransactionDAO

UserDAO

Data Access Object for Users

Class AccountDAO

Namespace: MakesCentsBackend.Services.DataAccessLayer

Assembly: MakesCentsBackend.dll

```
public class AccountDAO
```

Inheritance

object ← AccountDAO

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

AccountDAO(MySqlConnection, AuthorizationService)

Parameterized constructor to bring in DI variables

```
public AccountDAO(MySqlConnection connection, AuthorizationService authService)
```

Parameters

`connection` MySqlConnection

`authService` AuthorizationService

Methods

DeleteAccountAsync(BaseIdRequest)

```
public Task<BaseResponse> DeleteAccountAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<BaseResponse>

GetAllAccountsAsync(BaseIdRequest)

```
public Task<GetAllAccountsResponse> GetAllAccountsAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<GetAllAccountsResponse>

UpdateAccountAsync(UpdateAccountRequest, MySqlTransaction)

```
public Task<BaseIdResponse> UpdateAccountAsync(UpdateAccountRequest account,
MySqlTransaction dbTransaction)
```

Parameters

`account` UpdateAccountRequest

`dbTransaction` MySqlTransaction

Returns

Task<BaseIdResponse>

UpdateAccountBalanceAsync(int?, int?, decimal?, MySqlTransaction, MySqlConnection)

DAO method to update the balance of an account by the amount

```
public Task<bool> UpdateAccountBalanceAsync(int? userId, int? accountId, decimal? amount,
MySqlTransaction dbTransaction, MySqlConnection connection)
```

Parameters

`userId` int?

`accountId` int?

`amount` decimal?

`dbTransaction` MySqlTransaction

`connection` MySqlConnection

Returns

Task<bool>

Class AuthorizationService

Namespace: MakesCentsBackend.Services.DataAccessLayer

Assembly: MakesCentsBackend.dll

```
public class AuthorizationService
```

Inheritance

object ← AuthorizationService

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

AuthorizationService(MySqlConnection)

Parameterized constructor to bring in DI variables

```
public AuthorizationService(MySqlConnection connection)
```

Parameters

`connection` MySqlConnection

Methods

VerifyUserOwnsAccountAsync(int?, int?, MySqlTransaction?)

Auth method to check if a user owns the account specified

```
public Task<bool> VerifyUserOwnsAccountAsync(int? accountId, int? userId, MySqlTransaction?
transaction = null)
```

Parameters

`accountId` int?

`userId` int?

`transaction` MySqlTransaction

Returns

Task<bool>

VerifyUserOwnsBudgetAsync(int?, int?, MySqlTransaction?)

Auth method to check if a user owns the budget specified

```
public Task<bool> VerifyUserOwnsBudgetAsync(int? budgetId, int? userId, MySqlTransaction?
transaction = null)
```

Parameters

`budgetId` int?

`userId` int?

`transaction` MySqlTransaction

Returns

Task<bool>

VerifyUserOwnsBudgetMonthAsync(int?, int?, MySqlTransaction?)

Auth method to check if a user owns the specific month's budget instance specified

```
public Task<bool> VerifyUserOwnsBudgetMonthAsync(int? budgetMonthId, int? userId,
MySqlTransaction? transaction = null)
```

Parameters

`budgetMonthId` int?

`userId` int?

`transaction` MySqlTransaction

Returns

Task<bool>

VerifyUserOwnsBudgetTemplateAsync(int?, int?, MySqlTransaction?)

Auth method to check if a user owns the budget template specified

```
public Task<bool> VerifyUserOwnsBudgetTemplateAsync(int? budgetTemplateId, int? userId,
MySqlTransaction? transaction = null)
```

Parameters

`budgetTemplateId` int?

`userId` int?

`transaction` MySqlTransaction

Returns

Task<bool>

VerifyUserOwnsEnvelopeCategoryAsync(int?, int?, MySqlTransaction?)

Auth method to check if a user owns the envelope category specified

```
public Task<bool> VerifyUserOwnsEnvelopeCategoryAsync(int? envelopeCategoryId, int? userId,
MySqlTransaction? transaction = null)
```

Parameters

`envelopeCategoryId` int?

`userId` int?

`transaction` MySqlTransaction

Returns

Task<bool>

VerifyUserOwnsEnvelopeFundingAsync(int?, int?, MySqlTransaction?)

Auth method to check if a user owns the envelope funding row specified

```
public Task<bool> VerifyUserOwnsEnvelopeFundingAsync(int? envelopeFundingId, int? userId,
MySqlTransaction? transaction = null)
```

Parameters

`envelopeFundingId` int?

`userId` int?

`transaction` MySqlTransaction

Returns

Task<bool>

VerifyUserOwnsPaycheckAsync(int?, int?, MySqlTransaction?)

Auth method to check if a user owns the paycheck specified

```
public Task<bool> VerifyUserOwnsPaycheckAsync(int? paycheckId, int? userId,
MySqlTransaction? transaction = null)
```

Parameters

`paycheckId` int?

`userId` int?

`transaction` MySqlTransaction

Returns

Task<bool>

VerifyUserOwnsRolloverEnvelopeAsync(int?, int?, MySqlTransaction?)

Auth method to check if a user owns the persistent rollover envelope definition specified (used by planned_expense, which points at the definition rather than a specific month's instance)

```
public Task<bool> VerifyUserOwnsRolloverEnvelopeAsync(int? rolloverEnvelopeId, int? userId,
MySqlTransaction? transaction = null)
```

Parameters

`rolloverEnvelopeId` int?

`userId` int?

`transaction` MySqlTransaction

Returns

Task<bool>

VerifyUserOwnsRolloverEnvelopeMonthAsync(int?, int?, MySqlTransaction?)

Auth method to check if a user owns the specific month's rollover envelope instance specified (used by transaction_split and paycheck_split, which point at the monthly instance)

```
public Task<bool> VerifyUserOwnsRolloverEnvelopeMonthAsync(int? rolloverEnvelopeMonthId,
int? userId, MySqlTransaction? transaction = null)
```

Parameters

`rolloverEnvelopeMonthId` int?

`userId` int?

`transaction` MySqlTransaction

Returns

Task<bool>

VerifyUserOwnsSinkingFundEnvelopeAsync(int?, int?, MySqlTransaction?)

Auth method to check if a user owns the sinking fund envelope specified

```
public Task<bool> VerifyUserOwnsSinkingFundEnvelopeAsync(int? sinkingFundEnvelopeId, int?
userId, MySqlTransaction? transaction = null)
```

Parameters

`sinkingFundEnvelopeId` int?

`userId` int?

`transaction` MySqlTransaction

Returns

Task<bool>

VerifyUserOwnsTransactionAsync(int?, int?, MySqlTransaction?)

Auth method to check if a user owns the transaction specified

```
public Task<bool> VerifyUserOwnsTransactionAsync(int? transactionId, int? userId,
MySqlTransaction? transaction = null)
```

Parameters

`transactionId` int?

`userId` int?

`transaction` MySqlTransaction

Returns

Task<bool>

Class BankAccountDAO

Namespace: MakesCentsBackend.Services.DataAccessLayer

Assembly: MakesCentsBackend.dll

Data access object for bank accounts

```
public class BankAccountDAO
```

Inheritance

object ← BankAccountDAO

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Parameterized constructor to bring in DI variables

Constructors

BankAccountDAO(MySqlConnection, AuthorizationService, AccountDAO)

Parameterized constructor to bring in DI variables

```
public BankAccountDAO(MySqlConnection connection, AuthorizationService authService,
AccountDAO accountDAO)
```

Parameters

`connection` MySqlConnection

`authService` AuthorizationService

`accountDAO` AccountDAO

Methods

CreateBankAccountAsync(CreateBankAccountRequest)

DAO method to create a new bank account

```
public Task<CreateBankAccountResponse> CreateBankAccountAsync(CreateBankAccountRequest
bankAccount)
```

Parameters

`bankAccount` CreateBankAccountRequest

Returns

Task<CreateBankAccountResponse>

GetBankAccountAsync(BaseIdRequest)

DAO method to get a bank account, including its transaction history

```
public Task<GetBankAccountEntityResponse> GetBankAccountAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<GetBankAccountEntityResponse>

UpdateBankAccountAsync(UpdateBankAccountRequest)

DAO method to update a bank account

```
public Task<BaseIdResponse> UpdateBankAccountAsync(UpdateBankAccountRequest bankAccount)
```

Parameters

`bankAccount` UpdateBankAccountRequest

Returns

Task<BaseIdResponse>

Class BudgetDAO

Namespace: MakesCentsBackend.Services.DataAccessLayer

Assembly: MakesCentsBackend.dll

```
public class BudgetDAO
```

Inheritance

object ← BudgetDAO

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

BudgetDAO(MySqlConnection, AuthorizationService, BudgetTemplateDAO)

Parameterized constructor to bring in DI variables

```
public BudgetDAO(MySqlConnection connection, AuthorizationService authService,
BudgetTemplateDAO budgetTemplateDAO)
```

Parameters

`connection` MySqlConnection

`authService` AuthorizationService

`budgetTemplateDAO` BudgetTemplateDAO

Methods

CreateBudgetAsync(CreateBudgetRequest)

DAO method to create the persistent budget entity and link the creating user to it. Does not create a month - call CreateBudgetMonthAsync separately for that.

```
public Task<BaseIdResponse> CreateBudgetAsync(CreateBudgetRequest budget)
```

Parameters

`budget` CreateBudgetRequest

Returns

Task<BaseIdResponse>

CreateBudgetMonthAsync(CreateBudgetMonthRequest)

DAO method to create a new month under an existing persistent budget

```
public Task<BaseIdResponse> CreateBudgetMonthAsync(CreateBudgetMonthRequest budgetMonth)
```

Parameters

`budgetMonth` CreateBudgetMonthRequest

Returns

Task<BaseIdResponse>

DeleteBudgetAsync(BaseIdRequest)

DAO method to delete the persistent budget entity. Cascades to remove every month, every account, every envelope category (and its envelopes), every paycheck, and every planned expense that belongs to it.

```
public Task<BaseResponse> DeleteBudgetAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<BaseResponse>

DeleteBudgetMonthAsync(BaseIdRequest)

DAO method to delete a single month's budget. Only removes that month's budget_month row (and its transactions, via cascade) - accounts, envelope categories, and paychecks belong to the persistent budget and are shared across all of its months, so they survive the deletion of any single month.

```
public Task<BaseResponse> DeleteBudgetMonthAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<BaseResponse>

GetBudgetMonthAsync(BaseIdRequest)

Get a budget month - with envelope categories and envelopes - based on its budget_month_id

```
public Task<GetBudgetMonthResponse> GetBudgetMonthAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<GetBudgetMonthResponse>

GetBudgetMonthAsync(GetBudgetRequest)

Get a budget month - with envelope categories and envelopes - based on the userId, month, and year

```
public Task<GetBudgetMonthResponse> GetBudgetMonthAsync(GetBudgetRequest budget)
```

Parameters

`budget` GetBudgetRequest

Returns

Task<GetBudgetMonthResponse>

GetUserBudgetsAsync(int)

DAO method to get every budget a user has access to - just the id and name, since this is meant to be usable before any month or budget id is known (e.g. right after login).

```
public Task<GetUserBudgetsResponse> GetUserBudgetsAsync(int userId)
```

Parameters

`userId` int

Returns

Task<GetUserBudgetsResponse>

GetYearBudgetsAsync(GetBudgetRequest)

Get up to a year's worth of this user's past budget months, anchored at the given month/year

```
public Task<GetYearBudgetsResponse> GetYearBudgetsAsync(GetBudgetRequest request)
```

Parameters

`request` GetBudgetRequest

Returns

Task<GetYearBudgetsResponse>

UpdateBudgetAsync(EditBudgetRequest)

DAO method to update a budget's persistent fields (currently just the name)

```
public Task<BaseIdResponse> UpdateBudgetAsync(EditBudgetRequest budget)
```

Parameters

`budget` EditBudgetRequest

Returns

Task<BaseIdResponse>

UpdateBudgetMonthAsync(EditBudgetMonthRequest)

DAO method to update a specific budget month's month/year (e.g. correcting a month that was created under the wrong month/year)

```
public Task<BaseIdResponse> UpdateBudgetMonthAsync(EditBudgetMonthRequest budgetMonth)
```

Parameters

`budgetMonth` EditBudgetMonthRequest

Returns

Task<BaseIdResponse>

Class BudgetTemplateDAO

Namespace: MakesCentsBackend.Services.DataAccessLayer

Assembly: MakesCentsBackend.dll

Data access object for budget templates - named, reusable presets of envelope funding amounts that can be applied to a budget month to bulk-create its funding queue.

```
public class BudgetTemplateDAO
```

Inheritance

object ← BudgetTemplateDAO

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

BudgetTemplateDAO(MySqlConnection, AuthorizationService, RolloverEnvelopeDAO, EnvelopeFundingDAO)

Parameterized constructor to bring in DI variables

```
public BudgetTemplateDAO(MySqlConnection connection, AuthorizationService authService,
RolloverEnvelopeDAO rolloverEnvelopeDAO, EnvelopeFundingDAO envelopeFundingDAO)
```

Parameters

`connection` MySqlConnection

`authService` AuthorizationService

`rolloverEnvelopeDAO` RolloverEnvelopeDAO

`envelopeFundingDAO` EnvelopeFundingDAO

Methods

ApplyBudgetTemplateAsync(ApplyBudgetTemplateRequest)

DAO method to apply a template to a budget month as a standalone call. Thin wrapper around ApplyBudgetTemplateWithinTransactionAsync that owns its own transaction - see that method for what actually happens.

```
public Task<GetEnvelopeFundingQueueResponse>
ApplyBudgetTemplateAsync(ApplyBudgetTemplateRequest apply)
```

Parameters

`apply` ApplyBudgetTemplateRequest

Returns

Task<GetEnvelopeFundingQueueResponse>

ApplyBudgetTemplateWithinTransactionAsync(ApplyBudgetTemplateRequest, MySqlTransaction)

Core logic for applying a template to a budget month, runs inside a transaction the CALLER owns (doesn't commit/rollback itself) - so it can be reused by BudgetDAO to make "create a budget month" and "apply its template" one atomic operation. This ALWAYS fully replaces that month's existing funding queue - it does not merge with or preserve whatever was there before. Records which template was applied on budget_month.budget_template_id.

```
public Task<BaseResponse>
ApplyBudgetTemplateWithinTransactionAsync(ApplyBudgetTemplateRequest apply,
MySqlTransaction dbTransaction)
```

Parameters

`apply` ApplyBudgetTemplateRequest

`dbTransaction` MySqlTransaction

Returns

Task<BaseResponse>

CreateBudgetTemplateAsync(CreateBudgetTemplateRequest)

DAO method to create a budget template. No envelope entries yet - use SetBudgetTemplateEnvelopesAsync to add them afterward.

```
public Task<BaseIdResponse> CreateBudgetTemplateAsync(CreateBudgetTemplateRequest template)
```

Parameters

`template` CreateBudgetTemplateRequest

Returns

Task<BaseIdResponse>

DeleteBudgetTemplateAsync(BaseIdRequest)

DAO method to delete a budget template. Cascades to remove its envelope entries. Any budget_month that had this template applied keeps its funding queue, but its budget_template_id is set back to null (the record of "which template" is lost, the funding queue itself is not).

```
public Task<BaseResponse> DeleteBudgetTemplateAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<BaseResponse>

GetAllBudgetTemplatesAsync(BaseIdRequest)

DAO method to get every template for a budget

```
public Task<GetAllBudgetTemplatesResponse> GetAllBudgetTemplatesAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<GetAllBudgetTemplatesResponse>

GetBudgetTemplateAsync(BaseIdRequest)

DAO method to get a single budget template, including its envelope entries

```
public Task<GetBudgetTemplateResponse> GetBudgetTemplateAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<GetBudgetTemplateResponse>

SetBudgetTemplateEnvelopesAsync(SetBudgetTemplateEnvelopesRequest)

DAO method to set (create/reorder/update/remove all at once) a template's envelope entries. Existing rows not present in the incoming list are deleted. Rows with a BudgetTemplateEnvelopeId are updated in place Rows without one are inserted new. order_index is reassigned by each item's position in the incoming list - kept rows are first bumped into a high temporary range before being written back to their final low value, since unique_template_order is checked immediately (not deferred) and a straight reorder could otherwise collide with another row's still-current value. Also enforces that the same envelope can't appear twice in one template (unique_template_envelope_sinking / unique_template_envelope_rollover).

```
public Task<GetBudgetTemplateResponse>
SetBudgetTemplateEnvelopesAsync(SetBudgetTemplateEnvelopesRequest templateEnvelopes)
```

Parameters

`templateEnvelopes` SetBudgetTemplateEnvelopesRequest

Returns

Task<GetBudgetTemplateResponse>

UpdateBudgetTemplateAsync(EditBudgetTemplateRequest)

DAO method to rename a budget template

```
public Task<BaseIdResponse> UpdateBudgetTemplateAsync(EditBudgetTemplateRequest template)
```

Parameters

`template` EditBudgetTemplateRequest

Returns

Task<BaseIdResponse>

Class DebtAccountDAO

Namespace: MakesCentsBackend.Services.DataAccessLayer

Assembly: MakesCentsBackend.dll

```
public class DebtAccountDAO
```

Inheritance

object ← DebtAccountDAO

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

DebtAccountDAO(MySqlConnection, AuthorizationService, AccountDAO)

Parameterized constructor to bring in DI variables

```
public DebtAccountDAO(MySqlConnection connection, AuthorizationService authService,
AccountDAO accountDAO)
```

Parameters

`connection` MySqlConnection

`authService` AuthorizationService

`accountDAO` AccountDAO

Methods

CreateDebtAccountAsync(CreateDebtAccountRequest)

DAO method to create a new debt account

```
public Task<CreateDebtAccountResponse> CreateDebtAccountAsync(CreateDebtAccountRequest
debtAccount)
```

Parameters

`debtAccount` CreateDebtAccountRequest

Returns

Task<CreateDebtAccountResponse>

GetDebtAccountAsync(BaseIdRequest)

DAO method to get a debt account, including its transaction history

```
public Task<GetDebtAccountEntityResponse> GetDebtAccountAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<GetDebtAccountEntityResponse>

UpdateDebtAccountAsync(UpdateDebtAccountRequest)

DAO method to update a debt account

```
public Task<BaseIdResponse> UpdateDebtAccountAsync(UpdateDebtAccountRequest debtAccount)
```

Parameters

`debtAccount` UpdateDebtAccountRequest

Returns

Task <BaseIdResponse>

Class EnvelopeCategoryDAO

Namespace: MakesCentsBackend.Services.DataAccessLayer

Assembly: MakesCentsBackend.dll

Data access object for envelope categories

```
public class EnvelopeCategoryDAO
```

Inheritance

object ← EnvelopeCategoryDAO

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

EnvelopeCategoryDAO(MySqlConnection, AuthorizationService)

Parameterized constructor to bring in DI variables

```
public EnvelopeCategoryDAO(MySqlConnection connection, AuthorizationService authService)
```

Parameters

`connection` MySqlConnection

`authService` AuthorizationService

Methods

CreateEnvelopeCategoryAsync(CreateEnvelopeCategoryRequest)

DAO method to create an envelope category

```
public Task<BaseIdResponse> CreateEnvelopeCategoryAsync(CreateEnvelopeCategoryRequest
envelopeCategory)
```

Parameters

`envelopeCategory` CreateEnvelopeCategoryRequest

Returns

Task<BaseIdResponse>

DeleteEnvelopeCategoryAsync(BaseIdRequest)

DAO method to delete an envelope category. Cascades to remove every sinking fund and rollover envelope in it as well.

```
public Task<BaseResponse> DeleteEnvelopeCategoryAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<BaseResponse>

GetAllEnvelopeCategoriesAsync(BaseIdRequest)

DAO method to get all envelope categories (with their envelopes) for a given budget month. Sinking fund envelopes are persistent (no month context needed); rollover envelopes are joined to this month's instance. The budget_id is derived via a join to budget_month rather than taken as a separate parameter, since a budget_month_id already implies it.

```
public Task<GetAllEnvelopeCategoriesResponse> GetAllEnvelopeCategoriesAsync(BaseIdRequest
request)
```

Parameters

`request` BaseIdRequest

Returns

Task<GetAllEnvelopeCategoriesResponse>

GetEnvelopeCategoryAsync(BaseIdRequest)

```
public Task<GetEnvelopeCategoryResponse> GetEnvelopeCategoryAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<GetEnvelopeCategoryResponse>

UpdateEnvelopeCategoryAsync(EditEnvelopeCategoryRequest)

DAO method to update an envelope category based on the given fields

```
public Task<BaseIdResponse> UpdateEnvelopeCategoryAsync(EditEnvelopeCategoryRequest
envelopeCategory)
```

Parameters

`envelopeCategory` EditEnvelopeCategoryRequest

Returns

Task<BaseIdResponse>

Class EnvelopeFundingDAO

Namespace: MakesCentsBackend.Services.DataAccessLayer

Assembly: MakesCentsBackend.dll

Data access object for the envelope funding queue - a budget month's plan for how much to put into each envelope and in what priority order. This is a plan only; it doesn't move money on its own the way a transaction does.

```
public class EnvelopeFundingDAO
```

Inheritance

object ← EnvelopeFundingDAO

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

EnvelopeFundingDAO(MySqlConnection, AuthorizationService, RolloverEnvelopeDAO)

Parameterized constructor to bring in DI variables

```
public EnvelopeFundingDAO(MySqlConnection connection, AuthorizationService authService,
RolloverEnvelopeDAO rolloverEnvelopeDAO)
```

Parameters

`connection` MySqlConnection

`authService` AuthorizationService

`rolloverEnvelopeDAO` RolloverEnvelopeDAO

Methods

DeleteEnvelopeFundingAsync(BaseIdRequest)

DAO method to delete a single envelope funding row, without touching the rest of the queue

```
public Task<BaseResponse> DeleteEnvelopeFundingAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task <BaseResponse>

GetEnvelopeFundingQueueAsync(BaseIdRequest)

DAO method to get the funding queue for a budget month, in priority order

```
public Task<GetEnvelopeFundingQueueResponse> GetEnvelopeFundingQueueAsync(BaseIdRequest
request)
```

Parameters

`request` BaseIdRequest

Returns

Task <GetEnvelopeFundingQueueResponse>

SetEnvelopeFundingQueueAsync(SetEnvelopeFundingQueueRequest)

DAO method to set (create/reorder/update/remove all at once) the funding queue for a budget month. Existing rows not present in the incoming list are deleted. Rows with an EnvelopeFundingId are updated in place. Rows without one are inserted new. order_index is reassigned by each item's position in the incoming list - kept rows are first bumped into a high temporary range before being written back to their final low value, since unique_envelope_funding_order is checked immediately (not deferred) and a straight reorder could otherwise collide with another row's still-current value. Also enforces that the

same envelope can't have two funding rows in the same month (unique_envelope_funding_sinking / unique_envelope_funding_rollover).

```
public Task<GetEnvelopeFundingQueueResponse>
SetEnvelopeFundingQueueAsync(SetEnvelopeFundingQueueRequest fundingQueue)
```

Parameters

`fundingQueue` SetEnvelopeFundingQueueRequest

Returns

Task <GetEnvelopeFundingQueueResponse>

Class InvestmentAccountDAO

Namespace: MakesCentsBackend.Services.DataAccessLayer

Assembly: MakesCentsBackend.dll

```
public class InvestmentAccountDAO
```

Inheritance

object ← InvestmentAccountDAO

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

InvestmentAccountDAO(MySqlConnection, AuthorizationService, AccountDAO)

Parameterized constructor to bring in DI variables

```
public InvestmentAccountDAO(MySqlConnection connection, AuthorizationService authService,
AccountDAO accountDAO)
```

Parameters

`connection` MySqlConnection

`authService` AuthorizationService

`accountDAO` AccountDAO

Methods

CreateInvestmentAccountAsync(CreateInvestmentAccountRequest)

DAO method to create an investment account

```
public Task<CreateInvestmentAccountResponse>
CreateInvestmentAccountAsync(CreateInvestmentAccountRequest investmentAccount)
```

Parameters

`investmentAccount` CreateInvestmentAccountRequest

Returns

Task <CreateInvestmentAccountResponse>

GetInvestmentAccountAsync(BaseIdRequest)

DAO method to get an investment account, including its transaction history

```
public Task<GetInvestmentAccountEntityResponse> GetInvestmentAccountAsync(BaseIdRequest
request)
```

Parameters

`request` BaseIdRequest

Returns

Task <GetInvestmentAccountEntityResponse>

UpdateInvestmentAccountAsync(UpdateInvestmentAccountRequest)

DAO method to update an investment account

```
public Task<BaseIdResponse> UpdateInvestmentAccountAsync(UpdateInvestmentAccountRequest
investmentAccount)
```

Parameters

`investmentAccount` UpdateInvestmentAccountRequest

Returns

Task <BaseIdResponse>

Class PaycheckDAO

Namespace: MakesCentsBackend.Services.DataAccessLayer

Assembly: MakesCentsBackend.dll

Data access object for paychecks. Paychecks are forecast-only now - they don't fund envelopes directly (see EnvelopeFundingDAO for that) and aren't tied to any specific month, so this is plain single-table CRUD.

```
public class PaycheckDAO
```

Inheritance

object ← PaycheckDAO

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

PaycheckDAO(MySqlConnection, AuthorizationService)

Parameterized constructor to bring in DI variables

```
public PaycheckDAO(MySqlConnection connection, AuthorizationService authService)
```

Parameters

`connection` MySqlConnection

`authService` AuthorizationService

Methods

CreatePaycheckAsync(CreatePaycheckRequest)

DAO method to create a paycheck

```
public Task<BaseIdResponse> CreatePaycheckAsync(CreatePaycheckRequest paycheck)
```

Parameters

`paycheck` CreatePaycheckRequest

Returns

Task<BaseIdResponse>

DeletePaycheckAsync(BaseIdRequest)

DAO method to delete a paycheck

```
public Task<BaseResponse> DeletePaycheckAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<BaseResponse>

GetAllPaychecksAsync(BaseIdRequest)

DAO method to get every paycheck for a budget

```
public Task<GetAllPaychecksResponse> GetAllPaychecksAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<GetAllPaychecksResponse>

GetPaycheckAsync(BaseIdRequest)

DAO method to get a single paycheck

```
public Task<GetPaycheckResponse> GetPaycheckAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<GetPaycheckResponse>

UpdatePaycheckAsync(UpdatePaycheckRequest)

DAO method to update a paycheck based on given fields

```
public Task<BaseIdResponse> UpdatePaycheckAsync(UpdatePaycheckRequest paycheck)
```

Parameters

`paycheck` UpdatePaycheckRequest

Returns

Task<BaseIdResponse>

Class PaymentTransactionDAO

Namespace: MakesCentsBackend.Services.DataAccessLayer

Assembly: MakesCentsBackend.dll

```
public class PaymentTransactionDAO
```

Inheritance

object ← PaymentTransactionDAO

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

PaymentTransactionDAO(MySqlConnection, AuthorizationService, AccountDAO, SinkingFundEnvelopeDAO, RolloverEnvelopeDAO, TransactionDAO)

Parameterized constructor to bring in DI variables

```
public PaymentTransactionDAO(MySqlConnection connection, AuthorizationService authService,
AccountDAO accountDAO, SinkingFundEnvelopeDAO sinkingFundEnvelopeDAO, RolloverEnvelopeDAO
rolloverEnvelopeDAO, TransactionDAO transactionDAO)
```

Parameters

`connection` MySqlConnection

`authService` AuthorizationService

`accountDAO` AccountDAO

`sinkingFundEnvelopeDAO` SinkingFundEnvelopeDAO

`rolloverEnvelopeDAO` RolloverEnvelopeDAO

`transactionDAO` TransactionDAO

Methods

CreatePaymentTransactionAsync(CreatePaymentTransactionRequest)

DAO method to add a new payment transaction

```
public Task<CreatePaymentTransactionResponse>
CreatePaymentTransactionAsync(CreatePaymentTransactionRequest paymentTransaction)
```

Parameters

`paymentTransaction` CreatePaymentTransactionRequest

Returns

Task<CreatePaymentTransactionResponse>

GetPaymentTransactionAsync(BaseIdRequest)

DAO method to get a payment transaction and its splits

```
public Task<GetPaymentTransactionResponse> GetPaymentTransactionAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<GetPaymentTransactionResponse>

UpdatePaymentTransactionAsync(UpdatePaymentTransactionRequest)

DAO method to update a payment transaction and its splits. Reverses every old effect (the old account balance change, and each old split's effect on its envelope's remaining amount) before applying the new ones, so edited amounts and re-assigned envelopes/accounts don't leave balances double-counted or stale.

```
public Task<UpdatePaymentTransactionResponse>
UpdatePaymentTransactionAsync(UpdatePaymentTransactionRequest paymentTransaction)
```

Parameters

`paymentTransaction` UpdatePaymentTransactionRequest

Returns

Task<UpdatePaymentTransactionResponse>

Class RolloverEnvelopeDAO

Namespace: MakesCentsBackend.Services.DataAccessLayer

Assembly: MakesCentsBackend.dll

Data access object for rollover envelopes. Rollover envelopes are split across two tables:

- rollover_envelope: the persistent definition (name, category, default transfer target). No dollar amounts live here since a rollover envelope isn't guaranteed to be active every month.
- rollover_envelope_month: the amounts for one specific month. Absence of a row here means the envelope wasn't activated that month. Most "get/update amount" operations need a budget_month_id in addition to the envelope id.

```
public class RolloverEnvelopeDAO
```

Inheritance

object ← RolloverEnvelopeDAO

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

RolloverEnvelopeDAO(MySqlConnection, AuthorizationService)

Parameterized constructor to bring in DI variables

```
public RolloverEnvelopeDAO(MySqlConnection connection, AuthorizationService authService)
```

Parameters

`connection` MySqlConnection

`authService` AuthorizationService

Methods

CreateRolloverEnvelopeAsync(CreateRolloverEnvelopeRequest)

DAO method to create the persistent definition for a rollover envelope

```
public Task<BaseIdResponse> CreateRolloverEnvelopeAsync(CreateRolloverEnvelopeRequest
envelope)
```

Parameters

`envelope` CreateRolloverEnvelopeRequest

Returns

Task<BaseIdResponse>

DeleteRolloverEnvelopeAsync(BaseIdRequest)

DAO method to delete a rollover envelope's persistent definition. Cascades to remove every month's instance of it as well (ON DELETE CASCADE).

```
public Task<BaseResponse> DeleteRolloverEnvelopeAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<BaseResponse>

GetOrActivateForMonthAsync(int, int, MySqlTransaction?)

DAO method to get (or create, if not yet activated) this month's instance of a rollover envelope. Called when a budget month is first being set up, or the first time a transaction split needs to point at this envelope for a given month. No longer takes a planned amount - that now lives on envelope_funding, not here - so a newly activated month just starts at its table default remaining_amount of 0.

```
public Task<int?> GetOrActivateForMonthAsync(int rolloverEnvelopeId, int budgetMonthId,
MySqlTransaction? dbTransaction = null)
```

Parameters

`rolloverEnvelopeId` int

`budgetMonthId` int

`dbTransaction` MySqlTransaction

Returns

Task<int?>

The rollover_envelope_month_id for this envelope + month, or null on failure

GetRolloverEnvelopeForMonthAsync(GetRolloverEnvelopeMonthRequest)

DAO method to get a rollover envelope for a specific month, including its transaction history. If the envelope hasn't been activated for that month, RemainingAmount comes back null. Planned funding lives on envelope_funding, not here.

```
public Task<GetRolloverEnvelopeEntityResponse>
GetRolloverEnvelopeForMonthAsync(GetRolloverEnvelopeMonthRequest request)
```

Parameters

`request` GetRolloverEnvelopeMonthRequest

Returns

Task<GetRolloverEnvelopeEntityResponse>

UpdateRemainingAmountForMonthAsync(int?, int?, decimal?, MySqlTransaction, MySqlConnection)

DAO method to update the remaining amount of a specific month's rollover envelope instance

```
public Task<bool> UpdateRemainingAmountForMonthAsync(int? userId, int?
rolloverEnvelopeMonthId, decimal? amount, MySqlTransaction dbTransaction,
MySqlConnection connection)
```

Parameters

`userId` int?

`rolloverEnvelopeMonthId` int?

`amount` decimal?

`dbTransaction` MySqlTransaction

`connection` MySqlConnection

Returns

Task<bool>

UpdateRolloverEnvelopeAsync(UpdateRolloverEnvelopeRequest)

DAO method to update the persistent definition fields of a rollover envelope (name, category, default transfer target). Does not touch any month's amounts.

```
public Task<BaseIdResponse> UpdateRolloverEnvelopeAsync(UpdateRolloverEnvelopeRequest
envelope)
```

Parameters

`envelope` UpdateRolloverEnvelopeRequest

Returns

Task<BaseIdResponse>

Class SinkingFundEnvelopeDAO

Namespace: MakesCentsBackend.Services.DataAccessLayer

Assembly: MakesCentsBackend.dll

Data access object for sinking fund envelopes. Sinking fund envelopes are persistent - one row per envelope, with a live remaining_amount that carries forward month to month (no monthly instance table, unlike rollover envelopes).

```
public class SinkingFundEnvelopeDAO
```

Inheritance

object ← SinkingFundEnvelopeDAO

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

SinkingFundEnvelopeDAO(MySqlConnection, AuthorizationService)

Parameterized constructor to bring in DI variables

```
public SinkingFundEnvelopeDAO(MySqlConnection connection, AuthorizationService authService)
```

Parameters

`connection` MySqlConnection

`authService` AuthorizationService

Methods

CreateSinkingFundEnvelopeAsync(CreateSinkingFundEnvelopeRequest)

DAO method to create a sinking fund envelope

```
public Task<BaseIdResponse> CreateSinkingFundEnvelopeAsync(CreateSinkingFundEnvelopeRequest
envelope)
```

Parameters

`envelope` CreateSinkingFundEnvelopeRequest

Returns

Task<BaseIdResponse>

DeleteSinkingFundEnvelopeAsync(BaseIdRequest)

DAO method to delete a sinking fund envelope

```
public Task<BaseResponse> DeleteSinkingFundEnvelopeAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<BaseResponse>

GetRolloverDependentsAsync(BaseIdRequest)

DAO method to get the rollover envelopes that use this sinking fund envelope as their transfer target - these have to be pointed at a different sinking fund before this one can be deleted

```
public Task<GetSinkingFundDependentsResponse> GetRolloverDependentsAsync(BaseIdRequest
request)
```

Parameters

`request` BaseIdRequest

Returns

Task<GetSinkingFundDependentsResponse>

GetSinkingFundEnvelopeAsync(BaseIdRequest)

DAO method to get a single sinking fund envelope, including its transaction history

```
public Task<GetSinkingFundEnvelopeEntityResponse> GetSinkingFundEnvelopeAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<GetSinkingFundEnvelopeEntityResponse>

GetSinkingFundEnvelopeForUpdateAsync(BaseIdRequest)

DAO method to get a single sinking fund envelope without its transaction history (for update flows)

```
public Task<GetSinkingFundEnvelopeEntityResponse> GetSinkingFundEnvelopeForUpdateAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<GetSinkingFundEnvelopeEntityResponse>

UpdateRemainingAmountAsync(int?, int?, decimal?, MySqlTransaction, MySqlConnection)

DAO method to update the remaining amount of a sinking fund envelope by the amount

```
public Task<bool> UpdateRemainingAmountAsync(int? userId, int? sinkingFundEnvelopeId,
decimal? amount, MySqlTransaction dbTransaction, MySqlConnection connection)
```

Parameters

`userId` int?

`sinkingFundEnvelopeId` int?

`amount` decimal?

`dbTransaction` MySqlTransaction

`connection` MySqlConnection

Returns

Task<bool>

UpdateSinkingFundEnvelopeAsync(UpdateSinkingFundEnvelopeRequest)

DAO method to update a sinking fund envelope based on given fields

```
public Task<BaseIdResponse> UpdateSinkingFundEnvelopeAsync(UpdateSinkingFundEnvelopeRequest
envelope)
```

Parameters

`envelope` UpdateSinkingFundEnvelopeRequest

Returns

Task<BaseIdResponse>

Class TransactionDAO

Namespace: MakesCentsBackend.Services.DataAccessLayer

Assembly: MakesCentsBackend.dll

```
public class TransactionDAO
```

Inheritance

object ← TransactionDAO

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

TransactionDAO(MySqlConnection, AuthorizationService, AccountDAO, SinkingFundEnvelopeDAO, RolloverEnvelopeDAO)

Parameterized constructor to bring in DI variables. The account and envelope DAOs are needed so delete/restore can reverse or re-apply a transaction's balance effects.

```
public TransactionDAO(MySqlConnection connection, AuthorizationService
authService, AccountDAO accountDAO, SinkingFundEnvelopeDAO sinkingFundEnvelopeDAO,
RolloverEnvelopeDAO rolloverEnvelopeDAO)
```

Parameters

`connection` MySqlConnection

`authService` AuthorizationService

`accountDAO` AccountDAO

`sinkingFundEnvelopeDAO` SinkingFundEnvelopeDAO

`rolloverEnvelopeDAO` RolloverEnvelopeDAO

Methods

GetAllTransactionsAsync(BaseIdRequest)

DAO method to get every transaction (payment, account transfer, and envelope transfer) for a single budget month.

```
public Task<GetAllTransactionsEntityResponse> GetAllTransactionsAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task <GetAllTransactionsEntityResponse>

HardDeleteTransactionAsync(BaseIdRequest)

DAO method to permanently delete a transaction. If the transaction is still live, its effect is reversed first; if it was already soft-deleted, the effect was already reversed so only the row is removed.

```
public Task<BaseResponse> HardDeleteTransactionAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task <BaseResponse>

RestoreTransactionAsync(BaseIdRequest)

DAO method to restore a soft-deleted transaction. Re-applies the effect on envelopes and account balances that the soft delete removed.

```
public Task<BaseResponse> RestoreTransactionAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<BaseResponse>

SoftDeleteTransactionAsync(BaseIdRequest)

DAO method to soft delete a transaction. The row is kept (so it can be restored), but its effect on envelopes and account balances is reversed so deleted transactions don't count.

```
public Task<BaseResponse> SoftDeleteTransactionAsync(BaseIdRequest request)
```

Parameters

`request` BaseIdRequest

Returns

Task<BaseResponse>

UpdateTransactionAsync(UpdateTransactionRequest, MySqlTransaction)

Shared method (called by PaymentTransactionDAO/TransferTransactionDAO) to update the base transaction fields common to every transaction type. Unaffected by the schema split - transaction_date, total_amount, and notes are all still plain columns on transaction.

```
public Task<BaseIdResponse> UpdateTransactionAsync(UpdateTransactionRequest transaction,
MySqlTransaction dbTransaction)
```

Parameters

`transaction` UpdateTransactionRequest

`dbTransaction` MySqlTransaction

Returns

Task <BaseIdResponse>

Class TransferTransactionDAO

Namespace: MakesCentsBackend.Services.DataAccessLayer

Assembly: MakesCentsBackend.dll

```
public class TransferTransactionDAO
```

Inheritance

object ← TransferTransactionDAO

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

TransferTransactionDAO(MySqlConnection, AuthorizationService, AccountDAO, SinkingFundEnvelopeDAO, RolloverEnvelopeDAO, TransactionDAO)

Parameterized constructor to bring in DI variables

```
public TransferTransactionDAO(MySqlConnection connection, AuthorizationService authService,
AccountDAO accountDAO, SinkingFundEnvelopeDAO sinkingFundEnvelopeDAO, RolloverEnvelopeDAO
rolloverEnvelopeDAO, TransactionDAO transactionDAO)
```

Parameters

`connection` MySqlConnection

`authService` AuthorizationService

`accountDAO` AccountDAO

`sinkingFundEnvelopeDAO` SinkingFundEnvelopeDAO

`rolloverEnvelopeDAO` RolloverEnvelopeDAO

`transactionDAO` TransactionDAO

Methods

CreateTransferTransactionAsync(CreateTransferTransactionRequest)

DAO method to create a new transfer transaction (account-to-account or envelope-to-envelope)

```
public Task<CreateTransferTransactionResponse>
CreateTransferTransactionAsync(CreateTransferTransactionRequest transferTransaction)
```

Parameters

`transferTransaction` CreateTransferTransactionRequest

Returns

Task<CreateTransferTransactionResponse>

GetTransferTransactionAsync(BaseIdRequest)

DAO method to get a transfer transaction. For an envelope transfer, the two envelope sides are resolved from a follow-up transaction_split lookup rather than columns on transfer_transaction, since that table no longer has envelope columns.

```
public Task<GetTransferTransactionEntityResponse> GetTransferTransactionAsync(BaseIdRequest
request)
```

Parameters

`request` BaseIdRequest

Returns

Task<GetTransferTransactionEntityResponse>

UpdateTransferTransactionAsync(UpdateTransferTransactionRequest)

DAO method to update a transfer transaction. Reverses whatever the OLD effect was (old account balances if it was an account transfer, or old envelope remaining amounts if it was an envelope transfer) before applying the NEW effect, so edited amounts, re-assigned accounts/envelopes, or even switching between transfer types don't leave balances double-counted or stale.

```
public Task<BaseIdResponse> UpdateTransferTransactionAsync(UpdateTransferTransactionRequest transferTransaction)
```

Parameters

`transferTransaction` UpdateTransferTransactionRequest

Returns

Task<BaseIdResponse>

Class UserDAO

Namespace: MakesCentsBackend.Services.DataAccessLayer

Assembly: MakesCentsBackend.dll

Data Access Object for Users

```
public class UserDAO
```

Inheritance

object ← UserDAO

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

UserDAO(MySqlConnection)

Parameterized constructor for the User DAO Takes in DI parameters

```
public UserDAO(MySqlConnection connection)
```

Parameters

`connection` MySqlConnection

Methods

DeleteUserAsync(int)

Async method to delete a user based on a userId

```
public Task<BaseResponse> DeleteUserAsync(int userId)
```

Parameters

`userId` int

Returns

Task<BaseResponse>

FindUserByUsernameOrEmailAsync(string)

Finds a user based on a username/email

```
public Task<FindUserResponse> FindUserByUsernameOrEmailAsync(string usernameOrEmail)
```

Parameters

`usernameOrEmail` string

Returns

Task<FindUserResponse>

GetUserAsync(int)

Get a user from the database based on a users id

```
public Task<GetUserResponse> GetUserAsync(int userId)
```

Parameters

`userId` int

Returns

Task<GetUserResponse>

RegisterUserAsync(RegisterRequest)

Add a new user based on a user model

```
public Task<RegisterResponse> RegisterUserAsync(RegisterRequest user)
```

Parameters

`user` RegisterRequest

the data for the new user

Returns

Task<RegisterResponse>

The Id of the new user

UpdateUserAsync(EditUserRequest)

Async method to update a users information

```
public Task<BaseIdResponse> UpdateUserAsync(EditUserRequest user)
```

Parameters

`user` EditUserRequest

Returns

Task<BaseIdResponse>

Namespace MakesCentsBackend.Services.Mappers

Classes

MappingProfile

TransactionMapper

Class MappingProfile

Namespace: MakesCentsBackend.Services.Mappers

Assembly: MakesCentsBackend.dll

```
public class MappingProfile : Profile, IProfileExpression, IProfileConfiguration
```

Inheritance

object ← Profile ← MappingProfile

Implements

IProfileExpression, IProfileConfiguration

Inherited Members

Profile.DisableConstructorMapping() , Profile.CreateProjection<TSource, TDestination>() , Profile.CreateProjection<TSource, TDestination>(MemberList) , Profile.CreateMap<TSource, TDestination>() , Profile.CreateMap<TSource, TDestination>(MemberList) , Profile.CreateMap(Type, Type) , Profile.CreateMap(Type, Type, MemberList) , Profile.ClearPrefixes() , Profile.ReplaceMemberName(string, string) , Profile.RecognizePrefixes(params string[]) , Profile.RecognizePostfixes(params string[]) , Profile.RecognizeDestinationPrefixes(params string[]) , Profile.RecognizeDestinationPostfixes(params string[]) , Profile.AddGlobalIgnore(string) , Profile.IncludeSourceExtensionMethods(Type) , Profile.ProfileName , Profile.AllowNullDestinationValues , Profile.AllowNullCollections , Profile.ShouldMapProperty , Profile.ShouldMapField , Profile.ShouldMapMethod , Profile.ShouldUseConstructor , Profile.SourceMemberNamingConvention , Profile.DestinationMemberNamingConvention , Profile.ValueTransformers , Profile.PolymorphicProjectionsEnabled , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

MappingProfile()

Default constructor for the User Mapping Profile

```
public MappingProfile()
```

Class TransactionMapper

Namespace: MakesCentsBackend.Services.Mappers

Assembly: MakesCentsBackend.dll

```
public static class TransactionMapper
```

Inheritance

object ← TransactionMapper

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

ToSummaryResponse(SummaryTransactionEntityModel)

Mapper for summary transaction responses

```
public static SummaryTransactionDTOModel ToSummaryResponse(SummaryTransactionEntityModel db)
```

Parameters

`db` SummaryTransactionEntityModel

Returns

SummaryTransactionDTOModel

Namespace MakesCentsBackend.Services.Utilities

Classes

ClaimsPrincipalExtensions

DateOnlyHandler

Class to handle date only values when dealing with the database

JwtService

Service responsible for generating JWT tokens

PasswordHasher

Static class to handle password hashing

Class ClaimsPrincipalExtensions

Namespace: MakesCentsBackend.Services.Utilities

Assembly: MakesCentsBackend.dll

```
public static class ClaimsPrincipalExtensions
```

Inheritance

object ← ClaimsPrincipalExtensions

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

GetUserId(ClaimsPrincipal)

Get the UserId out of the JWT token

```
public static int GetUserId(this ClaimsPrincipal user)
```

Parameters

`user` ClaimsPrincipal

Returns

int

Exceptions

UnauthorizedAccessException

Class DateOnlyHandler

Namespace: MakesCentsBackend.Services.Utilities

Assembly: MakesCentsBackend.dll

Class to handle date only values when dealing with the database

```
public class DateOnlyHandler : SqlMapper.TypeHandler<DateOnly>, SqlMapper.ITypeHandler
```

Inheritance

object ← SqlMapper.TypeHandler<DateOnly> ← DateOnlyHandler

Implements

SqlMapper.ITypeHandler

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

Parse(object)

Method to take a DateTime object and return a DateOnly value

```
public override DateOnly Parse(object value)
```

Parameters

`value` object

Returns

DateOnly

SetValue(IDbDataParameter, DateOnly)

Method to take a DateOnly and set it to a DateTime

```
public override void SetValue(IDbDataParameter parameter, DateOnly value)
```

Parameters

`parameter` IDbDataParameter

`value` DateOnly

Class JwtService

Namespace: MakesCentsBackend.Services.Utilities

Assembly: MakesCentsBackend.dll

Service responsible for generating JWT tokens

```
public class JwtService
```

Inheritance

object ← JwtService

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

JwtService(IConfiguration)

Set readonly class variables

```
public JwtService(IConfiguration configuration)
```

Parameters

`configuration` IConfiguration

Exceptions

Exception

Methods

GenerateToken(int)

```
public string GenerateToken(int userId)
```

Parameters

`userId` int

Returns

string

Class PasswordHasher

Namespace: MakesCentsBackend.Services.Utilities

Assembly: MakesCentsBackend.dll

Static class to handle password hashing

```
public static class PasswordHasher
```

Inheritance

object ← PasswordHasher

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

HashPassword(string)

Hash a password and return the hash

```
public static string HashPassword(string password)
```

Parameters

`password` string

Returns

string

VerifyPassword(string, string)

Verify a hashed password with a plaintext one

```
public static bool VerifyPassword(string password, string hash)
```

Parameters

`password` string

`hash` string

Returns

bool